

Document Title	Specification of Update and Configuration Management
Document Owner	AUTOSAR
Document Responsibility	AUTOSAR
Document Identification No	888

Document Status	published
Part of AUTOSAR Standard	Adaptive Platform
Part of Standard Release	R25-11

Document Change History			
Date	Release	Changed by	Description
2025-11-27	R25-11	AUTOSAR Release Management	• Convert M2 modeling to APMC
2024-11-27	R24-11	AUTOSAR Release Management	• New suspend and resume requirements introduction • Progress monitoring refactoring
2023-11-23	R23-11	AUTOSAR Release Management	• Split UCM Master into SWS Vehicle Update Configuration Management
2022-11-24	R22-11	AUTOSAR Release Management	• Failing rollback clarifications • Campaign history type consolidated • Introduced production errors
2021-11-25	R21-11	AUTOSAR Release Management	• Renamed to SWS_UpdateAnd-ConfigurationManagement • UCM errors ordering • Vehicle State Manager API detailing
2020-11-30	R20-11	AUTOSAR Release Management	• Classic Platform update specification for UCM Master • Refactored UCM Master API • Simplified UCM Master State Machine • Detailed campaign history information





2019-11-28	R19-11	AUTOSAR Release Management	• Introduced UCM Master concept • Software Package state machine updated for processing while streaming • Reviewed UCM State Machine • Added new security analysis appendix • Changed Document Status from Final to published
2019-03-29	19-03	AUTOSAR Release Management	• Updating Package Management state machine • New requirements for robustness against reset • Improving specification item atomicity • Fixing errors in chapter Service Interfaces
2018-10-31	18-10	AUTOSAR Release Management	• Updated interaction other functional clusters like PER and EMO/SM • Introduction of vehicle package distribution
2018-03-29	18-03	AUTOSAR Release Management	• Extended and updated service interface • Introduction of Software Package • Introduction to securing update process
2017-10-27	17-10	AUTOSAR Release Management	• Initial release

Disclaimer

This work (specification and/or software implementation) and the material contained in it, as released by AUTOSAR, is for the purpose of information only. AUTOSAR and the companies that have contributed to it shall not be liable for any use of the work.

The material contained in this work is protected by copyright and other types of intellectual property rights. The commercial exploitation of the material contained in this work requires a license to such intellectual property rights.

This work may be utilized or reproduced without any modification, in any form or by any means, for informational purposes only. For any other purpose, no part of the work may be utilized or reproduced, in any form or by any means, without permission in writing from the publisher.

The work has been developed for automotive applications only. It has neither been developed, nor tested for non-automotive applications.

The word AUTOSAR and the AUTOSAR logo are registered trademarks.

Table of Contents

1	Introduction and functional overview	10
2	Acronyms and abbreviations	11
3	Related documentation	13
3.1	Input documents & related standards and norms	13
3.2	Related specification	13
3.3	Further applicable specification	14
4	Constraints and assumptions	15
4.1	Known Limitations	15
4.2	Applicability to car domains	16
5	Dependencies to other functional clusters	17
5.1	Provided Interfaces	17
5.2	Required Interfaces	18
5.3	Interfaces to Adaptive State Management	20
6	Requirements Tracing	21
7	Functional specification	26
7.1	Software Cluster lifecycle	26
7.2	Technical Overview	28
7.2.1	UCM Diagnostic Application	29
7.2.2	Software Package Management	30
7.2.2.1	Software Package	31
7.2.2.2	Content of a Software Package	32
7.2.2.3	Applications Persisted Data	34
7.2.3	Runtime dependencies	35
7.2.4	Update scope and State Management	35
7.3	Preparation Phase	37
7.3.1	Transferring Software Packages	37
7.3.1.1	Error handling in TransferStart	40
7.3.1.2	Error handling in TransferData	40
7.3.1.3	Error handling in TransferExit	43
7.3.1.4	Error handling in DeleteTransfer	45
7.3.2	Processing of Software Packages from a stream	45
7.3.3	Processing Software Packages	46
7.3.3.1	Error handling during Processing Software Packages	48
7.3.3.2	Error handling for Cancel	51
7.3.3.3	Error handling for RevertProcessedSwPackages	51
7.3.3.4	Error handling for GetSwProcessProgress	52
7.4	Activation Phase	52
7.4.1	Activation	52

7.4.1.1 Error handling for <code>Activate</code>	54
7.4.2 Rollback	55
7.4.2.1 Error handling for <code>Rollback</code>	57
7.4.3 Boot options	57
7.5 Cleanup Phase	58
7.5.1 Cleanup	58
7.6 Status Reporting	58
7.6.1 Preparation phase of Package Management	60
7.6.2 Activation phase of Package Management	60
7.6.3 Cleaning up phase of Package Management	64
7.6.4 Suspend and resume	65
7.7 Robustness against reset	66
7.7.1 Boot monitoring	67
7.8 History	67
7.9 Version Reporting	68
7.10 Securing Software Updates	69
7.11 Functional cluster lifecycle	69
7.11.1 Startup	69
7.11.2 Shutdown	70
7.12 Reporting	71
7.12.1 Security Events	71
7.12.2 Log Messages	73
7.12.2.1 Standardized Logging	73
7.12.3 Violation Messages	78
7.12.4 Production Errors	79
7.12.4.1 UCM ROLLBACK FAILED	79
7.12.4.2 HISTORY RECORD FAILED	79
7.12.4.3 CANCEL FAILED	79
7.12.4.4 MISSING DEPENDENCIES	80
7.12.4.5 OLD VERSION PACKAGE	80
7.12.4.6 PREPAREUPDATE FAILED	81
7.12.4.7 PREPAREUPDATE Rejected	81
7.12.4.8 UPDATE SESSION FAILED	81
7.12.4.9 UPDATE SESSION REJECTED	82
7.12.4.10 VERIFICATION FAILED	82
7.12.4.11 VERIFICATION REJECTED	82
7.12.4.12 PREPAREROLLBACK FAILED	83
7.12.4.13 PREPAREROLLBACK REJECTED	83
7.12.4.14 CLEANUP FAILED	83
8 API specification	84

9	Service Interfaces	85
9.1	Type definitions	85
9.1.1	UCMIdentifierType	88
9.1.2	UCMIdentifierAndVersionType	88
9.1.3	TransferIdType	88
9.1.4	SwPackageNameType	89
9.1.5	ProcessingStateType	89
9.1.6	TransferStateType	89
9.1.7	SwClusterNameType	90
9.1.8	StrongRevisionLabelString	90
9.1.9	SwNameVersionType	91
9.1.10	ProgressInformationType	91
9.1.11	ByteVectorType	92
9.1.12	SwPackageStateType	92
9.1.13	SwPackageInfoType	92
9.1.14	SwPackageInfoVectorType	93
9.1.15	SwClusterStateType	93
9.1.16	SwClusterInfoType	94
9.1.17	SwClusterInfoVectorType	94
9.1.18	CurrentStatusType	95
9.1.19	UpdateStateType	95
9.1.20	RunningStateType	96
9.1.21	ActionType	96
9.1.22	ResultType	97
9.1.23	HistoryType	97
9.1.24	HistoryVectorType	98
9.1.25	SwClusterManifestInfoType	98
9.1.26	DependencyType	98
9.1.27	DependencyVectorType	99
9.1.28	DependencyRoleType	99
9.1.29	LogicalOperationType	100
9.1.30	DependencyCompareConditionType	100
9.1.31	DependencyOperatorType	101
9.2	Provided Service Interfaces	101
9.2.1	Package Management	101
9.3	Required Interface	113
9.3.1	State Management Update Request	113
9.4	Application Errors	114
9.4.1	Application Error Domain	114
9.4.1.1	UCMErrorDomain	114

10	Configuration	116
10.1	Configuration Specification	116
10.1.1	Ucm Subordinate Configuration	117
10.1.2	General Configuration	120
10.1.3	SoftwareCluster	122
10.1.4	Relation to Diagnostics	130
10.1.5	ArtifactLocator	131
10.1.6	ArtifactChecksum	132
10.1.7	Correspondence Table	133
10.2	Default Values	134
10.3	Semantic Constraints	135
11	Sequence diagrams	136
11.1	Update process	136
11.2	Data transmission	136
11.3	Package processing	137
11.4	Activation	138
11.5	Failing activation	139
11.6	Failing rollback	141
11.7	V-UCM simplified vehicle update	143
A	Mentioned Manifest Elements	144
B	Mentioned APMC Elements from other Functional Clusters	152
C	Demands and constraints on Base Software	153
D	Interfaces to other Functional Clusters (informative)	154
D.1	Overview	154
D.2	Interfaces Tables	154
D.2.1	UCM update notification	154
E	Security Analysis of Installation and Update	155
E.1	Securing Software Package	155
E.2	Securing Calls to UCM	155
E.3	Suppressing Call to UCM	156
E.4	Resource Starvation	156
E.5	Zombie Sessions	156
F	History of Constraints and Specification Items	157
F.1	Constraint and Specification Item History of this document according to AUTOSAR Release R19-11.	157
F.1.1	Added Specification Items in R19-11	157
F.1.2	Changed Specification Items in R19-11	160
F.1.3	Deleted Specification Items in R19-11	161
F.1.4	Added Constraints in R19-11	161
F.1.5	Changed Constraints in R19-11	161

F.1.6 Deleted Constraints in R19-11	161
F.2 Constraint and Specification Item History of this document according to AUTOSAR Release R20-11.	161
F.2.1 Added Specification Items in R20-11	161
F.2.2 Changed Specification Items in R20-11	165
F.2.3 Deleted Specification Items in R20-11	167
F.2.4 Added Constraints in R20-11	169
F.2.5 Changed Constraints in R20-11	169
F.2.6 Deleted Constraints in R20-11	169
F.3 Constraint and Specification Item History of this document according to AUTOSAR Release R21-11.	169
F.3.1 Added Specification Items in R21-11	169
F.3.2 Changed Specification Items in R21-11	170
F.3.3 Deleted Specification Items in R21-11	174
F.3.4 Added Constraints in R21-11	175
F.3.5 Changed Constraints in R21-11	175
F.3.6 Deleted Constraints in R21-11	175
F.4 Constraint and Specification Item History of this document according to AUTOSAR Release R22-11.	175
F.4.1 Added Specification Items in R22-11	175
F.4.2 Changed Specification Items in R22-11	176
F.4.3 Deleted Specification Items in R22-11	181
F.4.4 Added Constraints in R22-11	181
F.4.5 Changed Constraints in R22-11	181
F.4.6 Deleted Constraints in R22-11	181
F.5 Constraint and Specification Item History of this document according to AUTOSAR Release R23-11.	182
F.5.1 Added Specification Items in R23-11	182
F.5.2 Changed Specification Items in R23-11	182
F.5.3 Deleted Specification Items in R23-11	183
F.5.4 Added Constraints in R23-11	187
F.5.5 Changed Constraints in R23-11	187
F.5.6 Deleted Constraints in R23-11	187
F.6 Constraint and Specification Item History of this document according to AUTOSAR Release R24-11.	188
F.6.1 Added Specification Items in R24-11	188
F.6.2 Changed Specification Items in R24-11	190
F.6.3 Deleted Specification Items in R24-11	193
F.6.4 Added Constraints in R24-11	193
F.6.5 Changed Constraints in R24-11	193
F.6.6 Deleted Constraints in R24-11	193
F.7 Constraint and Specification Item History of this document according to AUTOSAR Release R25-11.	194
F.7.1 Added Specification Items in R25-11	194

F.7.2	Changed Specification Items in R25-11	195
F.7.3	Deleted Specification Items in R25-11	195
F.7.4	Added Constraints in R25-11	196
F.7.5	Changed Constraints in R25-11	196
F.7.6	Deleted Constraints in R25-11	196

1 Introduction and functional overview

This software specification contains the functional description and interfaces of the functional cluster `Update and Configuration Management` which belongs to the `AUTOSAR Adaptive Platform Services`. `Update and Configuration Management` has the responsibility of installing, updating and removing software on an `AUTOSAR Adaptive Platform` in a safe and secure way while not sacrificing the dynamic nature of the `AUTOSAR Adaptive Platform`.

The `Update and Configuration Management` functional cluster is responsible for:

- Version reporting of the software present in the `AUTOSAR Adaptive Platform`
- Receiving and buffering software updates
- Checking that enough resources are available to ensure a software update
- Performing software updates and providing log messages and progress information
- Validating the outcome of a software update
- Providing rollback functionality to restore a known functional state in case of failure

In addition to updating and changing software on the `AUTOSAR Adaptive Platform`, the `Update and Configuration Management` is also responsible for updates and changes to the `AUTOSAR Adaptive Platform` itself, including all functional clusters, the underlying POSIX OS and its kernel with the responsibilities defined above.

In order to allow flexibility in how `Update and Configuration Management` is used, it will expose its functionality via `ara::com` service interfaces, not direct APIs. This ensures that the user of the functional cluster `Update and Configuration Management` does not have to be located on the same ECU.

2 Acronyms and abbreviations

The glossary below includes acronyms and abbreviations relevant to the UCM module that are not included in the [1, AUTOSAR glossary].

Abbreviation / Acronym:	Description:
Application Error	Errors returned by UCM
Backend	Backend is a server hosting Software Packages
Boot options	Boot Manager Configuration
Dependency check	Verification method proving that all configured dependencies will be fulfilled after finishing the activation.
DM	AUTOSAR Adaptive Diagnostic Management
D-PDU API	Diagnostic Protocol Data Unit Application Programming Interface
Integrity check	Verification method proving there has not been any alteration of the artefact content
MDF	Module Description File
MVCI	Modular Vehicle Communication Interface
OTA Client	OTA Client is an Adaptive Application in communication with Backend Over The Air
RDF	Root Description File
UCM	Update and Configuration Management
VCI	Vehicle Communication Interface
Vehicle Driver Application	Vehicle Driver Application is an Adaptive Application in communication with Vehicle Driver Human to Machine Interface
V-UCM	V-UCM is distributing packages and coordinating an update campaign in a vehicle
update cycle	Refers to the recurring traversal of the Preparation, Activation and Cleanup Phases of a software (one or more Software Clusters) update as specified in this document.
UCM Client	The application using the provided Service Interface of UCM e.g. OTA Client or V-UCM.

Table 2.1: Acronyms and abbreviations used in the scope of this Document

Some technical terms used in this document are already defined in the corresponding document mentioned in the table below. This is to avoid duplicate definition of the technical term. And to refer to the correct document.

Term	Description
Adaptive Application	see [1] AUTOSAR Glossary
AUTOSAR Adaptive Platform	see [1] AUTOSAR Glossary
AUTOSAR Classic Platform	see [1] AUTOSAR Glossary
Communication Management	see [2] AUTOSAR Communication Management
Electronic Control Unit	see [1] AUTOSAR Glossary
Executable	see [1] AUTOSAR Glossary
Execution Management	see [3] AUTOSAR Execution Management
Function Group	see [4] AUTOSAR State Management
Functional Cluster	see [1] AUTOSAR Glossary
Machine	see [1] AUTOSAR Glossary
MachineFG	see [3] AUTOSAR Execution Management
Manifest	see [1] AUTOSAR Glossary
Platform Health Management	see [5] AUTOSAR Platform Health Management

Service	see [1] AUTOSAR Glossary
Service Discovery	see [1] AUTOSAR Glossary
Service Interface	see [1] AUTOSAR Glossary
Software Cluster	see [1] AUTOSAR Glossary
Software Package	see [1] AUTOSAR Glossary
State Management	see [4] AUTOSAR State Management
Vehicle Package	see [1] AUTOSAR Glossary
Vehicle State Manager	see [1] AUTOSAR Glossary

Table 2.2: Reference to Technical Terms

3 Related documentation

3.1 Input documents & related standards and norms

- [1] Glossary
AUTOSAR_FO_TR_Glossary
- [2] Specification of Communication Management
AUTOSAR_AP_SWS_CommunicationManagement
- [3] Specification of Execution Management
AUTOSAR_AP_SWS_ExecutionManagement
- [4] Specification of State Management
AUTOSAR_AP_SWS_StateManagement
- [5] Specification of Platform Health Management
AUTOSAR_AP_SWS_PlatformHealthManagement
- [6] General Requirements specific to Adaptive Platform
AUTOSAR_AP_RS_General
- [7] Explanation of Adaptive Platform Software Architecture
AUTOSAR_AP_EXP_SWArchitecture
- [8] Requirements on Update and Configuration Management
AUTOSAR_AP_RS_UpdateAndConfigurationManagement
- [9] Explanation of Adaptive Platform Design
AUTOSAR_AP_EXP_PlatformDesign
- [10] Specification of Vehicle Update and Configuration Management
AUTOSAR_AP_SWS_VehicleUpdateAndConfigurationManagement
- [11] Specification of Manifest
AUTOSAR_AP_TPS_ManifestSpecification
- [12] Specification of Persistency
AUTOSAR_AP_SWS_Persistency

3.2 Related specification

See chapter [3.1](#).

3.3 Further applicable specification

AUTOSAR provides a general specification [6] which is also applicable for [UCM](#). The specification RS General shall be considered as additional and required specification for implementation of [UCM](#).

4 Constraints and assumptions

4.1 Known Limitations

UCM is not responsible to initiate the update process. UCM realizes a service interface to achieve this operation. The user of this service interface is responsible to verify that the vehicle is in a updatable state before executing a software update procedure on demand. It is also in the responsibility of the user to communicate with other AUTOSAR Adaptive Platforms or AUTOSAR Classic Platforms within the vehicle.

The UCM receives a locally available software package for processing. The software package is usually downloaded from the OEM backend. The download of the software packages has to be done by another application, i.e. UCM does not manage the connection to the OEM backend. Prior to triggering their processing, the software packages have to be transferred to UCM by using the provided `ara::com` interface.

The UCM update process is designed to cover updates on use case with single AUTOSAR Adaptive Platform. UCM can update Adaptive Applications, the AUTOSAR Adaptive Platform itself, including all functional clusters and the underlying OS.

The UCM is not responsible for enforcing authentication and access control to the provided interfaces. The document currently does not provide any mechanism for the confidentiality protection as well as measures against denial of service attacks. The assumption is that the platform preserves the integrity of parameters exchanged between UCM and its user.

The possibility to restart a specific application instead of a Machine reboot depends of the kind of update and application, is therefore implementation specific and is defined in the Software Package manifest.

UCM does only support updates through its ARA::COM service interface.

The Software Cluster metadata (like versions, etc.) managed by UCM can get out of sync if Software Clusters are updated without involvement of UCM, the information reported by UCM is then inconsistent. Moreover, UCM cannot protect against downgrading of a Software Cluster if there are alternative ways to update this Software Cluster.

UCM does not explicitly support the individual change of configuration, calibration, coding or any other parameters. Such changes are only implicitly supported by updating Software Clusters. Thus, the limitation at this point is that even if only a single parameter changes during an update, it always requires the full process of transferring, processing and activation of a Software Package, leading to a new Software Cluster version.

4.2 Applicability to car domains

No restrictions to applicability.

5 Dependencies to other functional clusters

This chapter provides an overview of the dependencies to other Functional Clusters in the [AUTOSAR Adaptive Platform](#). Section 5.1 “[Provided Interfaces](#)” lists the interfaces provided by Update and Configuration Management to other Functional Clusters. Section 5.2 “[Required Interfaces](#)” lists the interfaces required by Update and Configuration Management.

A detailed technical architecture documentation of the [AUTOSAR Adaptive Platform](#) is provided in [7].

5.1 Provided Interfaces

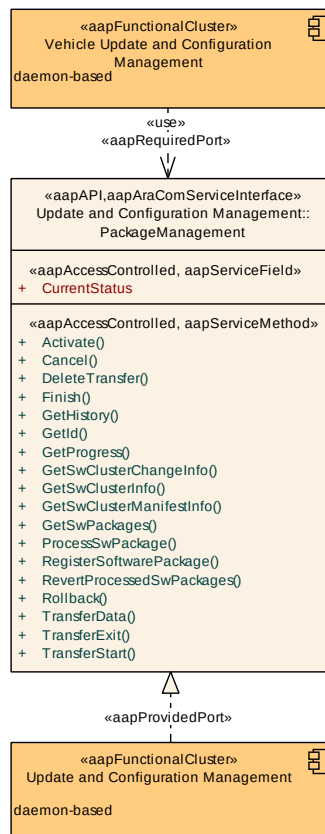


Figure 5.1: Interfaces provided by Update and Configuration Management to other Functional Clusters

Figure 5.1 shows interfaces provided by Update and Configuration Management to other Functional Clusters within the [AUTOSAR Adaptive Platform](#). [Table 5.1](#) provides a complete list of interfaces provided to other Functional Clusters within the [AUTOSAR Adaptive Platform](#).

Interface	Functional Cluster	Purpose
PackageManagement	Vehicle Update and Configuration Management	This interface is used to control different Update and Configuration Management instances and e.g., applications implementing the same interface located within the vehicle that act as an adapter to install software packages on third-party systems. Vehicle Update and Configuration Management is able to differentiate between the service instances by matching the result of GetId with an ID provided in a Software Package.

Table 5.1: Interfaces provided to other Functional Clusters

5.2 Required Interfaces



Figure 5.2: Interfaces required by Update and Configuration Management from Functional Clusters Persistency and Time Synchronization

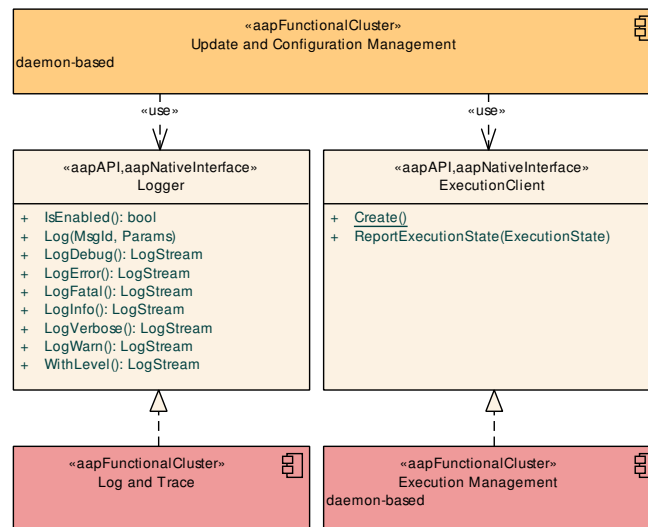


Figure 5.3: Interfaces required by Update and Configuration Management from Functional Clusters Log and Trace and Execution Management

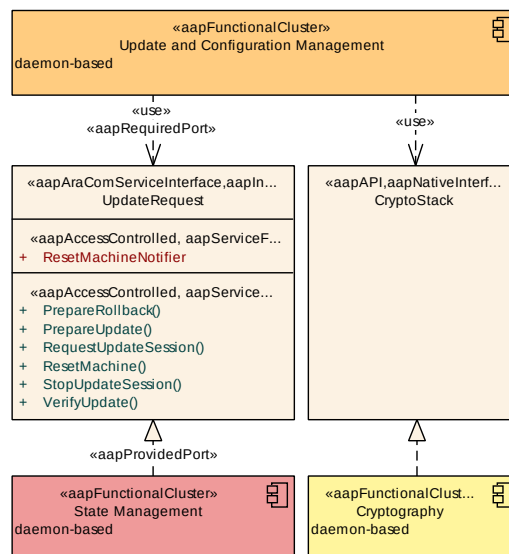


Figure 5.4: Interfaces required by Update and Configuration Management from Functional Clusters State Management and Cryptography

Figures 5.2, 5.3, and 5.4 show interfaces required by Update and Configuration Management from other Functional Clusters within the AUTOSAR Adaptive Platform. Table 5.2 provides a complete list of required interfaces from other Functional Clusters within the AUTOSAR Adaptive Platform.

Functional Cluster	Interface	Purpose
Cryptography	CryptoStack	This interface may be used e.g., to verify the integrity and authenticity of Software Packages.
Execution Management	ExecutionClient	This interface shall be used by the daemon process(es) inside Update and Configuration Management to report their execution state to Execution Management.
Log and Trace	Logger	Update and Configuration Management shall use this interface to log standardized messages.
Persistency	FileStorage	Used to store files of received software packages.
Persistency	KeyValueStorage	Used to store the internal state of Update and Configuration Management.
Platform Health Management	SupervisedEntity	This interface should be used to supervise the daemon process(es) of Update and Configuration Management.
State Management	UpdateRequest	This interface is used to interact with State Management of the Adaptive Platform during an update.
Time Synchronization	SynchronizedTimeBaseConsumer	Update and Configuration Management shall use this interface to get latest timestamp.

Table 5.2: Interfaces required from other Functional Clusters

5.3 Interfaces to Adaptive State Management

UCM relies on [State Management](#) and its provided `UpdateRequest` Service Interface to perform the necessary `Function Group` state changes needed to activate the newly installed, updated or removed software.

Certain applications can conflict with the update process or the newly updated package, and they need to be stopped during the update process. This could be achieved by putting the machine to a safe `Machine` state, by activating a combination of suitable `Function Groups` and its states. It is the responsibility of the platform integrator to define this state or `Function Groups`. The [Adaptive Application](#) accessing the UCM, should make sure that the platform is switched to this state (using interfaces from [State Management](#)), before starting the update.

6 Requirements Tracing

The following tables reference the requirements specified in [8] and links to the fulfillment of these. Please note that if column “Satisfied by” is empty for a specific requirement this means that this requirement is not fulfilled by this document.

Requirement	Description	Satisfied by
[RS_EM_00014]	Execution Management shall support a Trusted Platform.	[SWS_UCM_00202]
[RS_Ids_00810]	Basic SW security events	[SWS_UCM_00399] [SWS_UCM_00400] [SWS_UCM_00401] [SWS_UCM_00402] [SWS_UCM_00403] [SWS_UCM_00404] [SWS_UCM_00405] [SWS_UCM_00407] [SWS_UCM_00408]
[RS_SM_00001]	State Management shall coordinate and control multiple sets of Applications.	[SWS_UCM_00242]
[RS_UCM_00001]	UCM shall support installing new software on AUTOSAR Adaptive Platform	[SWS_UCM_00017] [SWS_UCM_00073] [SWS_UCM_00099] [SWS_UCM_00131] [SWS_UCM_00137] [SWS_UCM_00165] [SWS_UCM_00240] [SWS_UCM_00266] [SWS_UCM_00305] [SWS_UCM_00343] [SWS_UCM_00391] [SWS_UCM_00392]
[RS_UCM_00002]	UCM shall support reporting version information for an AUTOSAR Adaptive Platform	[SWS_UCM_00039] [SWS_UCM_00040] [SWS_UCM_00071] [SWS_UCM_00077] [SWS_UCM_00078] [SWS_UCM_00079] [SWS_UCM_00112] [SWS_UCM_00130] [SWS_UCM_00131] [SWS_UCM_00175] [SWS_UCM_00176] [SWS_UCM_00311] [SWS_UCM_00312] [SWS_UCM_00319] [SWS_UCM_00330] [SWS_UCM_00343] [SWS_UCM_00357] [SWS_UCM_00358] [SWS_UCM_00362] [SWS_UCM_CONSTR_00001] [SWS_UCM_CONSTR_00002] [SWS_UCM_CONSTR_00014]
[RS_UCM_00003]	UCM shall support updating installed software on Adaptive Platform	[SWS_UCM_00017] [SWS_UCM_00073] [SWS_UCM_00165] [SWS_UCM_00190] [SWS_UCM_00257] [SWS_UCM_00306] [SWS_UCM_00342]
[RS_UCM_00004]	UCM shall support uninstalling software on AUTOSAR Adaptive Platform	[SWS_UCM_00073] [SWS_UCM_00137] [SWS_UCM_00165] [SWS_UCM_00184] [SWS_UCM_00266] [SWS_UCM_00273] [SWS_UCM_00343] [SWS_UCM_00391] [SWS_UCM_00392]
[RS_UCM_00005]	UCM shall make sure that persistent data owned by uninstalled software is deleted	[SWS_UCM_00184] [SWS_UCM_00273] [SWS_UCM_00349]
[RS_UCM_00006]	UCM shall verify Software Package authenticity and integrity using strong cryptographic techniques	[SWS_UCM_00039] [SWS_UCM_00040] [SWS_UCM_00077] [SWS_UCM_00079] [SWS_UCM_00092] [SWS_UCM_00098] [SWS_UCM_00136] [SWS_UCM_00200] [SWS_UCM_00393] [SWS_UCM_00394]
[RS_UCM_00007]	UCM shall check that software dependencies are fulfilled	[SWS_UCM_00026] [SWS_UCM_00136] [SWS_UCM_00161] [SWS_UCM_00231] [SWS_UCM_00260] [SWS_UCM_00313] [SWS_UCM_00314] [SWS_UCM_00315] [SWS_UCM_00316] [SWS_UCM_00317] [SWS_UCM_00318] [SWS_UCM_00363]





Requirement	Description	Satisfied by
[RS_UCM_00008]	UCM shall support a recovery mechanism in case of failed activation	[SWS_UCM_00024] [SWS_UCM_00107] [SWS_UCM_00110] [SWS_UCM_00111] [SWS_UCM_00126] [SWS_UCM_00127] [SWS_UCM_00131] [SWS_UCM_00146] [SWS_UCM_00155] [SWS_UCM_00162] [SWS_UCM_00163] [SWS_UCM_00164] [SWS_UCM_00264] [SWS_UCM_00282] [SWS_UCM_00299] [SWS_UCM_00302] [SWS_UCM_00353] [SWS_UCM_00371] [SWS_UCM_00374]
[RS_UCM_00010]	UCM shall support reporting of Software Packages downloaded for AUTOSAR Adaptive Platform	[SWS_UCM_00039] [SWS_UCM_00040] [SWS_UCM_00077] [SWS_UCM_00079] [SWS_UCM_00131] [SWS_UCM_00330] [SWS_UCM_00343] [SWS_UCM_00393] [SWS_UCM_00394] [SWS_UCM_CONSTR_00001] [SWS_UCM_CONSTR_00002]
[RS_UCM_00011]	UCM shall support reporting software versions which have been installed and will be activated when new versions are activated	[SWS_UCM_00030] [SWS_UCM_00039] [SWS_UCM_00040] [SWS_UCM_00077] [SWS_UCM_00078] [SWS_UCM_00079] [SWS_UCM_00131] [SWS_UCM_00191] [SWS_UCM_00192] [SWS_UCM_00193] [SWS_UCM_00194] [SWS_UCM_00195] [SWS_UCM_00196] [SWS_UCM_00197] [SWS_UCM_00198] [SWS_UCM_00199] [SWS_UCM_00286] [SWS_UCM_00287] [SWS_UCM_00311] [SWS_UCM_00312] [SWS_UCM_00356] [SWS_UCM_00359] [SWS_UCM_00360] [SWS_UCM_00393] [SWS_UCM_00394] [SWS_UCM_CONSTR_00001] [SWS_UCM_CONSTR_00002] [SWS_UCM_CONSTR_00014]
[RS_UCM_00012]	UCM shall check the consistency of transferred Software Package	[SWS_UCM_00029] [SWS_UCM_00039] [SWS_UCM_00040] [SWS_UCM_00077] [SWS_UCM_00079] [SWS_UCM_00092] [SWS_UCM_00104] [SWS_UCM_00136] [SWS_UCM_00207] [SWS_UCM_00213] [SWS_UCM_00267] [SWS_UCM_00393] [SWS_UCM_00394] [SWS_UCM_CONSTR_00012]
[RS_UCM_00013]	UCM shall check that it has enough resources to receive, process and store the Software Package and associated data	[SWS_UCM_00007] [SWS_UCM_00008] [SWS_UCM_00010] [SWS_UCM_00087] [SWS_UCM_00088] [SWS_UCM_00136] [SWS_UCM_00140] [SWS_UCM_00145] [SWS_UCM_00206] [SWS_UCM_00217] [SWS_UCM_00243] [SWS_UCM_00275] [SWS_UCM_00276] [SWS_UCM_00283] [SWS_UCM_00289] [SWS_UCM_00305] [SWS_UCM_00329]
[RS_UCM_00014]	UCM shall check that correct amount of data has been transferred for the Software Package	[SWS_UCM_00136] [SWS_UCM_00204] [SWS_UCM_00205] [SWS_UCM_00243]
[RS_UCM_00015]	UCM shall remove all unneeded data after Software Package processing has finished	[SWS_UCM_00020] [SWS_UCM_00131] [SWS_UCM_00285] [SWS_UCM_00331] [SWS_UCM_00349] [SWS_UCM_00354] [SWS_UCM_00409] [SWS_UCM_00410]
[RS_UCM_00018]	UCM shall announce when an application has been installed, updated or uninstalled	[SWS_UCM_00021] [SWS_UCM_00131] [SWS_UCM_00259] [SWS_UCM_00356] [SWS_UCM_00359] [SWS_UCM_00360] [SWS_UCM_00361]





Requirement	Description	Satisfied by
[RS_UCM_00019]	UCM shall support simultaneous transfers multiple Software Packages	[SWS_UCM_00007] [SWS_UCM_00008] [SWS_UCM_00010] [SWS_UCM_00031] [SWS_UCM_00075] [SWS_UCM_00087] [SWS_UCM_00088] [SWS_UCM_00098] [SWS_UCM_00140] [SWS_UCM_00145] [SWS_UCM_00148] [SWS_UCM_00203] [SWS_UCM_00204] [SWS_UCM_00205] [SWS_UCM_00206] [SWS_UCM_00208] [SWS_UCM_00212] [SWS_UCM_00214] [SWS_UCM_00215] [SWS_UCM_00216] [SWS_UCM_00275] [SWS_UCM_00276] [SWS_UCM_00283] [SWS_UCM_00344] [SWS_UCM_00345] [SWS_UCM_00346] [SWS_UCM_00347] [SWS_UCM_00348]
[RS_UCM_00020]	UCM shall support cancellation of an update or install operation	[SWS_UCM_00003] [SWS_UCM_00167] [SWS_UCM_00234] [SWS_UCM_00235] [SWS_UCM_00236] [SWS_UCM_00237] [SWS_UCM_00239] [SWS_UCM_00278] [SWS_UCM_00279] [SWS_UCM_00351] [SWS_UCM_00372]
[RS_UCM_00021]	UCM shall support atomic activation of installed or updated Software Clusters	[SWS_UCM_00022] [SWS_UCM_00094] [SWS_UCM_00131] [SWS_UCM_00241] [SWS_UCM_00259] [SWS_UCM_00260] [SWS_UCM_00280] [SWS_UCM_00352]
[RS_UCM_00023]	UCM shall provide an interface to read progress of the update	[SWS_UCM_00131] [SWS_UCM_00341]
[RS_UCM_00024]	UCM shall provide an interface to read the state of UCM	[SWS_UCM_00019] [SWS_UCM_00038] [SWS_UCM_00044] [SWS_UCM_00080] [SWS_UCM_00081] [SWS_UCM_00084] [SWS_UCM_00085] [SWS_UCM_00131] [SWS_UCM_00147] [SWS_UCM_00149] [SWS_UCM_00150] [SWS_UCM_00152] [SWS_UCM_00153] [SWS_UCM_00154] [SWS_UCM_00166] [SWS_UCM_00168] [SWS_UCM_00169] [SWS_UCM_00258] [SWS_UCM_00293] [SWS_UCM_00300] [SWS_UCM_00301] [SWS_UCM_00341] [SWS_UCM_00361] [SWS_UCM_00364] [SWS_UCM_00395] [SWS_UCM_00396]
[RS_UCM_00025]	UCM shall support receiving of Software Package data	[SWS_UCM_00007] [SWS_UCM_00008] [SWS_UCM_00010] [SWS_UCM_00031] [SWS_UCM_00032] [SWS_UCM_00087] [SWS_UCM_00088] [SWS_UCM_00098] [SWS_UCM_00131] [SWS_UCM_00140] [SWS_UCM_00145] [SWS_UCM_00165] [SWS_UCM_00166] [SWS_UCM_00167] [SWS_UCM_00168] [SWS_UCM_00169] [SWS_UCM_00206] [SWS_UCM_00217] [SWS_UCM_00219] [SWS_UCM_00243] [SWS_UCM_00272] [SWS_UCM_00275] [SWS_UCM_00276] [SWS_UCM_00283] [SWS_UCM_00294] [SWS_UCM_00344] [SWS_UCM_00345] [SWS_UCM_00346] [SWS_UCM_00347] [SWS_UCM_00348]





Requirement	Description	Satisfied by
[RS_UCM_00026]	UCM shall process installation of new Software Packages, updates and removal of existing Software Packages sequentially	[SWS_UCM_00017] [SWS_UCM_00044] [SWS_UCM_00122] [SWS_UCM_00184] [SWS_UCM_00218] [SWS_UCM_00219] [SWS_UCM_00240] [SWS_UCM_00257] [SWS_UCM_00258] [SWS_UCM_00261] [SWS_UCM_00262] [SWS_UCM_00263] [SWS_UCM_00273] [SWS_UCM_00277] [SWS_UCM_00281] [SWS_UCM_00288] [SWS_UCM_00349] [SWS_UCM_00350] [SWS_UCM_00364] [SWS_UCM_00373] [SWS_UCM_00396]
[RS_UCM_00027]	UCM shall be able to safely recover from unexpected interruption.	[SWS_UCM_00157] [SWS_UCM_00158] [SWS_UCM_00270] [SWS_UCM_00302]
[RS_UCM_00028]	UCM shall support updating Functional Clusters	[SWS_UCM_00100] [SWS_UCM_00245] [SWS_UCM_00306] [SWS_UCM_00342]
[RS_UCM_00029]	UCM shall support updating the underlying Operating System	[SWS_UCM_00101] [SWS_UCM_00245] [SWS_UCM_00342]
[RS_UCM_00030]	UCM shall be able to verify the updated software during activation	[SWS_UCM_00107] [SWS_UCM_00111] [SWS_UCM_00126] [SWS_UCM_00127] [SWS_UCM_00146] [SWS_UCM_00155] [SWS_UCM_00162] [SWS_UCM_00163] [SWS_UCM_00164] [SWS_UCM_00260] [SWS_UCM_00264] [SWS_UCM_00352] [SWS_UCM_00370] [SWS_UCM_00374]
[RS_UCM_00031]	UCM shall prevent installation of arbitrary previous version of an Adaptive Application or the Adaptive Platform	[SWS_UCM_00103] [SWS_UCM_00190]
[RS_UCM_00032]	UCM shall provide an interface to return UCM's action history	[SWS_UCM_00115] [SWS_UCM_00131] [SWS_UCM_00132] [SWS_UCM_00133] [SWS_UCM_00134] [SWS_UCM_00135] [SWS_UCM_00160] [SWS_UCM_00271] [SWS_UCM_00292] [SWS_UCM_00355]
[RS_UCM_00044]	UCM Initialization	[SWS_UCM_00274]
[RS_UCM_00045]	UCM shall report production errors	[SWS_UCM_00302] [SWS_UCM_00303] [SWS_UCM_00320] [SWS_UCM_00321] [SWS_UCM_00322] [SWS_UCM_00323] [SWS_UCM_00324] [SWS_UCM_00325] [SWS_UCM_00326] [SWS_UCM_00327] [SWS_UCM_00366] [SWS_UCM_00367] [SWS_UCM_00368] [SWS_UCM_00369] [SWS_UCM_00375] [SWS_UCM_00409] [SWS_UCM_00410]
[RS_UCM_00046]	UCM shall support standardized trace points	[SWS_UCM_00332] [SWS_UCM_00333] [SWS_UCM_00334] [SWS_UCM_00335] [SWS_UCM_00336] [SWS_UCM_00337] [SWS_UCM_00338] [SWS_UCM_00339] [SWS_UCM_00340] [SWS_UCM_00376] [SWS_UCM_00377] [SWS_UCM_00378] [SWS_UCM_00379] [SWS_UCM_00380] [SWS_UCM_00381] [SWS_UCM_00382] [SWS_UCM_00383] [SWS_UCM_00384]
[RS_UCM_00047]	UCM Support for Update Suspend and Resume.	[SWS_UCM_00385] [SWS_UCM_00386] [SWS_UCM_00387] [SWS_UCM_00388] [SWS_UCM_00389] [SWS_UCM_00390] [SWS_UCM_00397] [SWS_UCM_00398]
[RS_VUCM_00035]	V-UCM Cross-ECU Updates	[SWS_UCM_00309]
[RS_VUCM_00036]	V-UCM Interaction with UCMS	[SWS_UCM_00173]





Requirement	Description	Satisfied by
[RS_VUCM_00037]	V-UCM Safe Software Modification	[SWS_UCM_00313] [SWS_UCM_00314] [SWS_UCM_00315] [SWS_UCM_00316] [SWS_UCM_00317] [SWS_UCM_00318]
[RS_VUCM_00039]	V-UCM Vehicle Packages Validation	[SWS_UCM_00200]

Table 6.1: Requirements Tracing

7 Functional specification

7.1 Software Cluster lifecycle

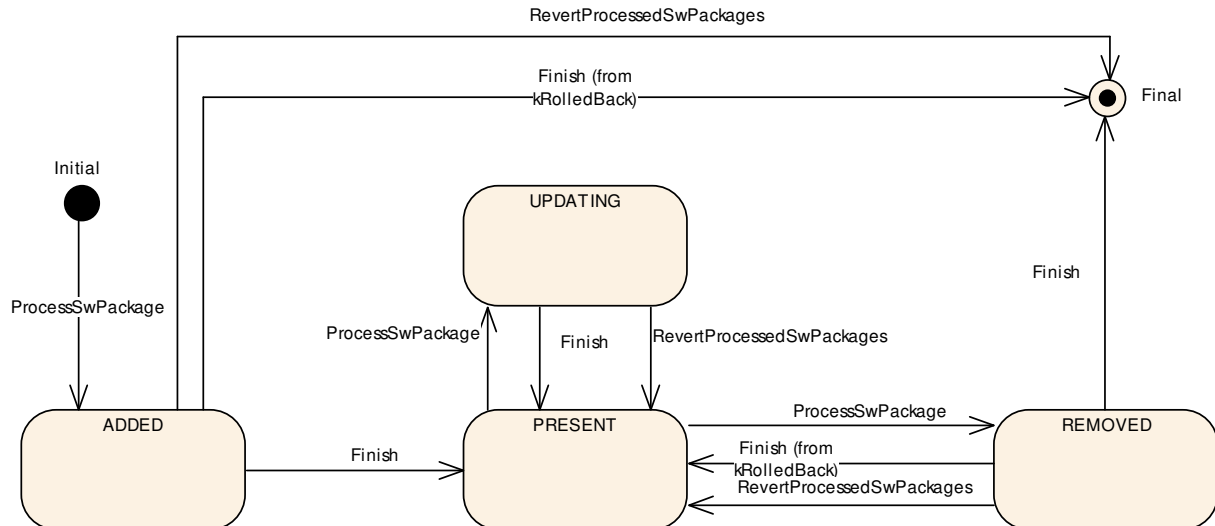


Figure 7.1: State Machine for a Software Cluster

The state machine in Fig. 7.1 describes the life-cycle states of a **Software Cluster**. These states are reported with **GetSwClusterChangeInfo** method.

[SWS_UCM_00191] **Software Cluster** life-cycle state **kAdded**

Upstream requirements: **RS_UCM_00011**

[A **Software Cluster** state shall be **kAdded** after the **Software Cluster** is successfully processed with **ProcessSwPackage** method call on the **AUTOSAR Adaptive Platform** and if it was not previously present in the **AUTOSAR Adaptive Platform** and before activation is finished.]

[SWS_UCM_00192] **Software Cluster** life-cycle state transition from **kAdded** to **kPresent**

Upstream requirements: **RS_UCM_00011**

[A **Software Cluster** state shall change from **kAdded** to **kPresent** after a successful activation of a newly added **Software Cluster** with **Finish** method call.]

[SWS_UCM_00195] **Software Cluster** life-cycle state **kUpdating**

Upstream requirements: **RS_UCM_00011**

[A **Software Cluster** state shall be **kUpdating** after a successful processing of the updated **Software Cluster** with **ProcessSwPackage** method call and before activation is finished.]

[SWS_UCM_00193] Software Cluster life-cycle state transition from kUpdating to kPresent

Upstream requirements: RS_UCM_00011

[A Software Cluster state shall change from kUpdating to kPresent after a successful activation of the updated Software Cluster with Finish method call, or after reverting the Software Cluster update with a RevertProcessedSwPackages method call.]

[SWS_UCM_00196] Software Cluster life-cycle state kRemoved

Upstream requirements: RS_UCM_00011

[A Software Cluster state shall be kRemoved after successful completion of method ProcessSwPackage which involves the removal of the existed Software Cluster and before activation is finished.]

[SWS_UCM_00194] Software Cluster life-cycle state transition from kRemoved to kPresent in case of RevertProcessedSwPackages call

Upstream requirements: RS_UCM_00011

[A Software Cluster state shall change from kRemoved to kPresent after a successful call to RevertProcessedSwPackages method in case the Software Cluster was previously requested to be removed by ProcessSwPackage method call.]

[SWS_UCM_00286] Software Cluster life-cycle state transition from kRemoved to kPresent in case of Finish call

Upstream requirements: RS_UCM_00011

[A Software Cluster state shall change from kRemoved to kPresent after a successful call to Finish method in case a Software Cluster being removed has to be rolled back after a failing activation.]

[SWS_UCM_00197] End of Software Cluster life-cycle state from state kAdded in case of RevertProcessedSwPackages call

Upstream requirements: RS_UCM_00011

[A Software Cluster shall reach the end of its life-cycle from kAdded after a successful removal of a newly added Software Cluster with RevertProcessedSwPackages method call in case the Software Cluster was previously requested to be added by ProcessSwPackage method call.]

[SWS_UCM_00287] End of Software Cluster life-cycle state from state kAdded in case of Finish call

Upstream requirements: RS_UCM_00011

[A Software Cluster shall reach the end of its life-cycle from kAdded after a successful removal of a newly added Software Cluster with Finish method call in case the newly added Software Cluster has to be rolled back after a failing activation.]

[SWS_UCM_00198] End of `Software Cluster` life-cycle state from state `kRemoved`

Upstream requirements: `RS_UCM_00011`

[A `Software Cluster` shall reach the end of its life-cycle if it is successfully removed with a `Finish` method call and the `Software Cluster` is in state `kRemoved`.]

[SWS_UCM_00199] Reporting of `Software Cluster` reaching end of life-cycle

Upstream requirements: `RS_UCM_00011`

[Any `Software Cluster` reaching the end of its life-cycle shall not be reported by UCM any more.]

The multiplicity of `SoftwareCluster.design` has been set to 0..* to support the following use cases:

- Both an OEM and a Tier-1 supplier want to provide a design for a `SoftwareCluster`. But the OEM does not let the supplier extend their design.

The consequence is that the supplier would create an own `SoftwareCluster-Design` and let the `SoftwareCluster` refer to it. This means that the `SoftwareCluster` has two references in the role `SoftwareCluster.design`.

- An integrator decides to implement several `SoftwareClusterDesigns` with one `SoftwareCluster`.

7.2 Technical Overview

One of the declared goals of `AUTOSAR Adaptive Platform` is the ability to flexibly update the software and its configuration through over-the-air updates. During the life-cycle of an `AUTOSAR Adaptive Platform`, UCM is responsible to perform software modifications on the machine and to retain consistency of the whole system.

The `UCM Functional Cluster` provides a service interface that exposes its functionality to retrieve `AUTOSAR Adaptive Platform` software information and consistently execute software updates. Since `ara:com` is used, the client using the UCM service interface can be located on the same `AUTOSAR Adaptive Platform`, but also remote clients are possible.

The service interface has been primarily designed with the goal to make it possible to use standard diagnostic services for downloading and installing software updates for the `AUTOSAR Adaptive Platform`. However, the methods and fields in the service interface are designed in such a way that they can be used in principle by any `Adaptive Application`. UCM does not impose any specific protocol on how data is transferred to the `AUTOSAR Adaptive Platform` and how package processing is controlled. In particular UCM does not expose diagnostic services.

It is not possible for UCM to identify its clients and there could be several clients involved in one update. Therefore, clients have the responsibility to avoid conflicting interaction

on the same UCM service interface. There might be a list of clients (handled by IAM) which are allowed to access the server.

As shown in Figure 7.2, whether the use case is an over-the-air update or garage update done through diagnostics, it is not visible to the UCM. The UCM Client abstracts the use case from the UCM and forwards the data stream and sequence control commands to the UCM. Later in this document, the term UCM Client is used to describe an *Adaptive Application* that consumes UCM PackageManagement services through UCM ara::com API. Diagnostic Application and V-UCM are two examples of such UCM Clients.

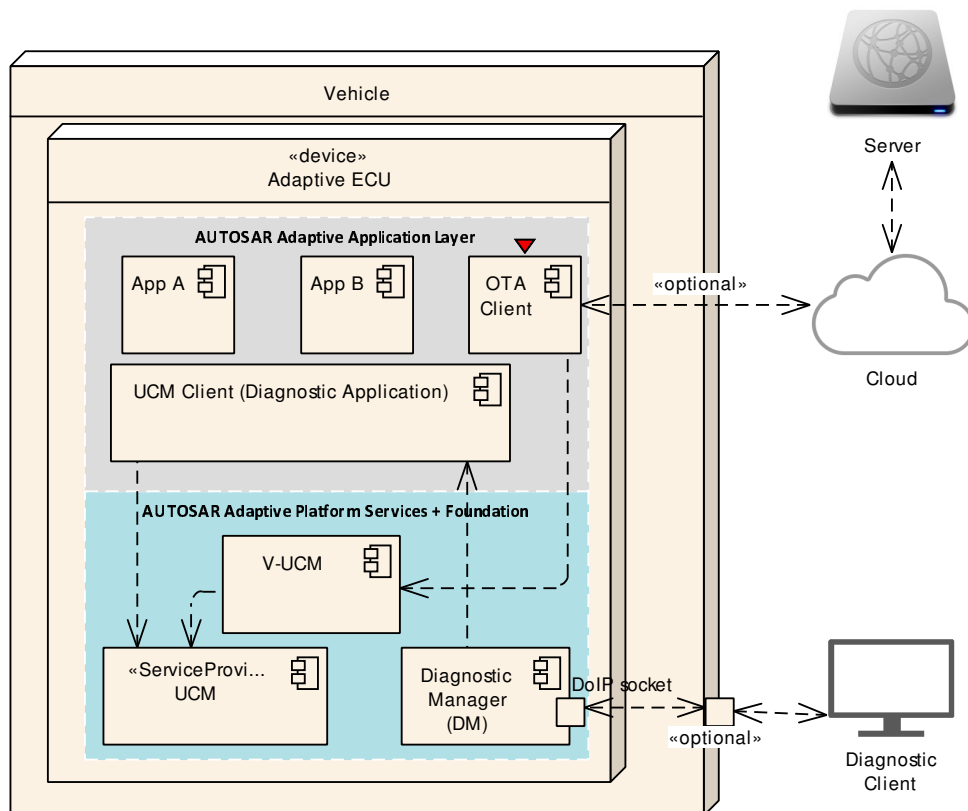


Figure 7.2: Architecture overview for diagnostic use case

7.2.1 UCM Diagnostic Application

	Diagnostic Application as UCM Client	Diagnostic Application as V-UCM Client
Purpose	Update standalone ECU/Machine without involvement of V-UCM	Update ECU/Machine as part of vehicle update through V-UCM
Flow	Diagnostic Tool -> Diagnostic Manager -> Diagnostic App -> UCM	Diagnostic Tool -> Diagnostic Manager -> Diagnostic App -> V-UCM
Instance	One instance per standalone Adaptive ECU/Machine	One instance per vehicle
Artifacts handled	Receives Software Packages	Receives Vehicle Packages and Software Packages

	Diagnostic Application as UCM Client	Diagnostic Application as V-UCM Client
UCM API (Service)	Package Management	Vehicle Package Management
ECUs/Machines being updated	Adaptive only (incl. Classic Machines on Adaptive ECU, if needed)	Any ECU (Adaptive, Classic, Proprietary)
Implemented by	ECU Vendor and/or OEM	OEM
References	• Figure 7.2, Figure 11.1, Figure 11.2, Figure 11.4 in this document • Figure "Vehicle Update Architecture" in AUTOSAR_EXP_PlatformDesign [9] • Figure "Interfaces of UCM" in AUTOSAR_EXP_SWArchitecture [7]	• Figure "Example of V-UCM architecture overview within a vehicle" and Figure "Classic platform update with V-UCM and diagnostic tool" in [10] document, Figure 11.2 in this document

Table 7.1: The usage of UCM Diagnostic Application

7.2.2 Software Package Management

As Software Packages are the starting point of any update performed by UCM the management of these is a essential part of the specification. Getting from transferred Software Packages to activated Software Clusters in an [update cycle](#) is done in three phases:

- **Preparation phase:** The phase in which [Software Packages](#) can be transferred from the [UCM Client](#) to the UCM and processed to alter (Install, Update or Remove) each relevant [Software Cluster](#) included in the actual update. For detailed informations see chapter 7.3.
- **Activation phase:** The critical phase of an [update cycle](#) in which the UCM performs dependency checks of the involved [Software Clusters](#) (compare chapter 7.2.3) then the activation of [Software Clusters](#) and finally the verification of the installation, with the help of [State Management](#), prior to finishing the update. For further informations see chapter 7.4.
- **Cleanup phase:** The phase in which the system is restored into a clean state with all unnecessary artifacts, like [Software Packages](#), removed and prepared to be ready for the next update cycle. More information can be found in chapter 7.5.

7.2.2.1 Software Package

[SWS_UCM_00122] Software Package utilization

Upstream requirements: RS_UCM_00026

[The unit for deployment that the UCM shall take as input is called `Software Package`, see [1]. Each `Software Package` shall address a single `SoftwareCluster`.]

A `SoftwareCluster` can act in two roles:

- ‘Sub’-`SoftwareCluster` : It is a `SoftwareCluster` without diagnostic target address, containing processes, executables and further elements
- ‘Root’-`SoftwareCluster` : It is a `SoftwareCluster` with a diagnostic target address that may reference several other ‘Sub’-`SoftwareClusters`, which thus form a logical group.

A `SoftwareCluster` can be of the following categories expressed by the attribute `SoftwareCluster.SoftwareClusterCategory`:

- `APPLICATION_LAYER`: the `SoftwareCluster` can be removed by UCM.
- `PLATFORM_CORE`: the `SoftwareCluster` cannot be removed as it would break the system.
- `PLATFORM`: the `SoftwareCluster` is part of the platform software and can be removed.

[SWS_UCM_00245] Software Cluster category

Upstream requirements: RS_UCM_00028, RS_UCM_00029

[UCM shall not remove a `SoftwareCluster` where the corresponding `SoftwareClusterDesign` has attribute `installationBehavior` set to value `cannotBeRemoved`. In case of such an attempt, UCM shall raise `ApplicationError kSwclRemovalDenied`.]

A `Software Package` has to be modelled as a so-called `SoftwareCluster` which describes the content of a `Software Package` that is downloaded or uploaded to the AUTOSAR Adaptive Platform, see [11].

The term `Software Package` is used for the "physical", uploadable `Software Package` that is processed by UCM whereas the term `SoftwareCluster` is used for the modeling element. In the model, the content of a `SoftwareCluster` is defined by references to all required model elements. The `SoftwareCluster` and the related model elements define the content of the manifest that is part of the `Software Package`. The `Software Package` format and the update scope are described in chapter "Content of a `Software Package`" as well as in [9].

[SWS_UCM_CONSTR_00012]

Upstream requirements: [RS_UCM_00012](#)

[The [SoftwareCluster](#) aggregation of [ArtifactChecksum](#) shall not include the [ArtifactChecksum.URI](#) of this same [SoftwareCluster](#) manifest.]

The [ArtifactChecksum.URI](#) attribute in [ArtifactChecksum](#) is referring to the artifact contained in the [SoftwareCluster](#).

7.2.2.2 Content of a Software Package

Each [Software Package](#) addresses a single [SoftwareCluster](#) and contains manifests, executables and further data (depending on the role of the [SoftwareCluster](#)) as the example sketched in Figure 7.3.

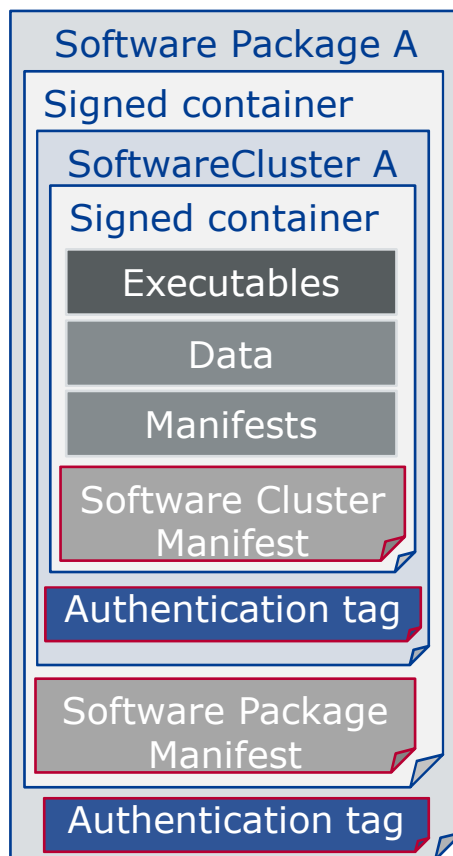


Figure 7.3: [Software Package](#) content description

A single [Software Package](#) is designed in a way that it could contain one or several executables of [Adaptive Applications](#), kernel or firmware updates, or updated configuration and calibration data to be deployed on the [AUTOSAR Adaptive Platform](#).

The [Software Package](#) manifest is recommended to be sent at the beginning in order for [UCM](#) to have early information of for instance memory usage or streaming.

An exemplary implementation of the adaptive workflow with `Software Packages` can be seen in chapter Methodology and Manifest in [9]. For more details on the `Software Package` class, you can refer to `SoftwarePackage`.

[SWS_UCM_00112] `Software Cluster` and version

Upstream requirements: `RS_UCM_00002`

[`SoftwareCluster`'s manifest shall include a name and a `SoftwareClusterVersion` following description of `StrongRevisionLabelString`.]

[SWS_UCM_00319] Semantic versioning

Upstream requirements: `RS_UCM_00002`

[`UCM` shall compute `SoftwareCluster` dependency check comparing only `MajorVersion` and `MinorVersion`.]

[SWS_UCM_CONSTR_00001]

Upstream requirements: `RS_UCM_00002`, `RS_UCM_00010`, `RS_UCM_00011`

[If any content (for instance an executable or persistent data) of an already installed `SoftwareCluster` is modified by an incoming `Software Package`, then the `SoftwareClusterVersion` number of the incoming `SoftwareCluster` indicated in the `Software Package` shall be higher than the `SoftwareClusterVersion` number of the already installed `SoftwareCluster`.]

If the constraint is violated, an error will be raised according to [SWS_UCM_00103].

A higher version number is achieved by an increment of any of the `MajorVersion`, the `MinorVersion`, or the `PatchVersion`.

For `SoftwareCluster` dependency check [SWS_UCM_00319] or `Software Package` compatibility against `UCM` [SWS_UCM_00161], `PatchVersion` and additional labels of `StrongRevisionLabelString` are not considered.

If there is a need to downgrade a failing `SoftwareCluster` (for instance, malfunction in the field that was not detected at activation), it will therefore be needed to repackage the same old `SoftwareCluster` that was properly working with an higher `SoftwareClusterVersion` number.

[SWS_UCM_00130] `Software Cluster` and version error

Upstream requirements: `RS_UCM_00002`

[If `SoftwareCluster`'s manifest does not contain any `SoftwareClusterVersion` following description of `StrongRevisionLabelString`, `UCM` shall raise the `ApplicationError kPackageManifestInvalid`.]

[SWS_UCM_CONSTR_00014] Software Package and Software Cluster short-Names

Upstream requirements: [RS_UCM_00002](#), [RS_UCM_00011](#)

[[SoftwarePackage](#) and the referenced [SoftwareCluster](#) shall share the same [shortName](#) in order to be able to compare their versions.]

The [shortName](#) of [SoftwareCluster](#) is by definition unique in its context as described into TPS Manifest document [11]. The applicable context for a [SoftwareCluster](#) is the complete vehicle, otherwise there would be conflicts of [SoftwareCluster shortNames](#) within a dependency model for instance. It is responsibility of integrator and tooling to make sure about [shortName](#) uniqueness within vehicle and it is typically applied by adding information to [SoftwareCluster shortName](#) like uri or architectural tree position (example: VirtualMachinename-SWCLshortName or filesystemUri-SWCLshortName)

7.2.2.3 Applications Persisted Data

Updating and rolling back of persisted data is handled completely by the application using persistency without involvement of [UCM](#). A detailed explanation can be found in the Persistency Specification [12]. An exception here is the removal of persistent data after a [SoftwareCluster](#) is removed.

[SWS_UCM_00184] Persistent data clean-up after Software Cluster removal

Upstream requirements: [RS_UCM_00026](#), [RS_UCM_00005](#), [RS_UCM_00004](#)

[[UCM](#) shall remove persistent data of a removed [SoftwareCluster](#) by using the information given in the application manifest, namely [deploymentUri.uri](#) and [persistencyCentralStorageURI](#), in order to leave the [AUTOSAR Adaptive Platform](#) and the file system clean.]

[SWS_UCM_00273] Persistent data clean-up after Software Cluster update that removes a process

Upstream requirements: [RS_UCM_00026](#), [RS_UCM_00005](#), [RS_UCM_00004](#)

[[UCM](#) shall remove persistent data of a removed process by using the information given in the execution manifest, namely [deploymentUri.uri](#) and [persistencyCentralStorageURI](#) in order to leave the [AUTOSAR Adaptive Platform](#) and the file system clean.]

Persistent data can include administrative and backup data.

[SWS_UCM_00305] Persistent data uri at [Software Cluster](#) installation

Upstream requirements: [RS_UCM_00001](#), [RS_UCM_00013](#)

[In the case of installation of a [Software Cluster](#), [UCM](#) shall create the new uris following content of execution manifest, namely [deploymentUri.uri](#) and [persistencyCentralStorageURI](#).]

[SWS_UCM_00306] Persistent data uri change at update

Upstream requirements: [RS_UCM_00003](#), [RS_UCM_00028](#)

[In the case of uri change of persistent storages, UCM shall move the existing folders containing the persistent storages from the old uri to the new uri following content of execution manifest, namely `deploymentUri.uri` and `persistencyCentralStorageURI`.]

[SWS_UCM_00329] Activate `kPersistencyAllocationFailed`

Upstream requirements: [RS_UCM_00013](#)

[After `Activate` method is called and if the sum of all `Software Clusters maximumAllowedSize` exceeds available size defined by `maxAvailablePersistencyStorageSpace` which is reserved for persistency, then UCM shall raise `ApplicationError kPersistencyAllocationFailed`.]

7.2.3 Runtime dependencies

Processes within a `SoftwareCluster` can have functional dependencies toward other `SoftwareClusters`.

Dependencies are described in the `SoftwareCluster` metamodel, see [11]. This dependency model allows to confirm for instance if there are missing or conflicting services within `Machine` or within the whole vehicle, or if required libraries located in another `Software Cluster` would be missing or conflicting with the being updated `Software Cluster`.

The rationale is, if UCM has to process several `Software Packages`, then execution dependencies may not be fulfilled at all times during the `Software Packages` process but must be fulfilled before changes can be activated.

At activation, UCM is starting a session with `State Management` (SM) which is requesting `Execution Management` (EM) to change states of `FunctionGroups` (Sequence diagram 10.4). `Execution Management` uses `Execution Manifest` which contains modelled execution dependencies. If those dependencies are not met at **Verify** state, `Execution Management` will report an error to `State Management` forwarding this error to UCM which will rollback.

7.2.4 Update scope and State Management

`Software Package` processed by UCM can contain `Adaptive Applications`, updates to `AUTOSAR Adaptive Platform` itself or to the underlying OS. Update type depends on the content of the `Software Package`.

[SWS_UCM_00099] Update of **Adaptive Application**

Upstream requirements: [RS_UCM_00001](#)

[UCM shall be able to update **Adaptive Applications**]

[SWS_UCM_00100] Update of **Functional Clusters**

Upstream requirements: [RS_UCM_00028](#)

[UCM shall be able to update all **Functional Clusters**, including UCM itself.]

[SWS_UCM_00101] Update of **Host**

Upstream requirements: [RS_UCM_00029](#)

[UCM shall be able to update the underlying OS hosting the **AUTOSAR Adaptive Platform**.]

Definition of an updatable state with respect to the system setup is the OEM responsibility. Based on the system setup and the application, the system might need to be switched into a predefined state, to free resource to speed up the update, to block normal usage of software which might cause interruptions to update process and to block using functionality which might be interrupted by the update sequence.

[SWS_UCM_00257] Update session

Upstream requirements: [RS_UCM_00026](#), [RS_UCM_00003](#)

[To confirm the system is in an updatable state, UCM shall start an update session by calling **State Management** `UpdateRequest` **Service Interface** `RequestUpdateSession` method after its dependency check triggered by **Activate** method call successfully completes.]

[SWS_UCM_00258] Update session rejected

Upstream requirements: [RS_UCM_00026](#), [RS_UCM_00024](#)

[If **State Management** `UpdateRequest` **Service Interface** `RequestUpdateSession` method call raises error `kRejected`, UCM shall transition from `kActivating` to `kPreparing` state, report FAILED to the `UCM_UPDATE_SESSION_REJECTED` production error and **Activate** method call shall return `ApplicationError kUpdateSessionRejected`. When the update session accepted, a PASSED shall be reported alternatively.]

If update session could be recurrently rejected, it is up to implementer to cache the dependency check result in order to avoid unnecessary computation and compute it only once.

During the update session, the minimum applications required for the Update process should be executed. This way system is more robust, more resources are free and user is blocked from using applications, of which failure could cause safety risk to the user.

Update of some components require a Machine reset to be performed. These components should be configured to be part of [Function Group MachineFG](#), as the update sequence of [Function Group MachineFG](#) includes a [Machine](#) reset. [Execution Management](#), [State Management](#), [Communication Management](#) and [UCM](#) itself are good examples which probably require a Machine reset to activate the update. Other such components could be applications involved in the update sequence or applications involved in safety monitoring. Further details on [Function Group MachineFG](#) can be found in [State Management](#).

7.3 Preparation Phase

The Preparation phase includes transferring and processing of [Software Packages](#). It is possible to only transfer (see chapter [7.3.1](#)) and perform the processing of a Software Package (see chapter [7.3.3](#)) in a later point of time, or combine both steps and process directly from a stream, as described in chapter [7.3.2](#).

7.3.1 Transferring Software Packages

In the Preparation phase, it is possible to transfer [Software Packages](#) decoupled from the processing. This section describes requirements for initiation of a data transfer, the data transmission and ending of the data transmission.

Each [Software Package](#) gets two states assigned as soon as the transfer has been started. The two states are defined by the [TransferStateType](#) and the [ProcessingStateType](#). With the combination of both and the information returned from the method [GetSwPackages](#) a client of [UCM](#) has access to the current progress and status of a [Software Package](#). The state machines in [Fig. 7.4](#) are showing the lifecycle of a [Software Package](#) that is transferred to and processed by [UCM](#). During this lifecycle, a [Software Package](#) is uniquely identified with an [id](#) that [UCM](#) provides to the client.

The [UCM](#) has the possibility to keep the [Software Package](#) in [kTransferred](#) state in case it failed the processing of the [Software Package](#) and retry later: transferring [Software Package](#) can be costly, for instance if it is authenticated, there could be no reason to delete it if the update has failed.

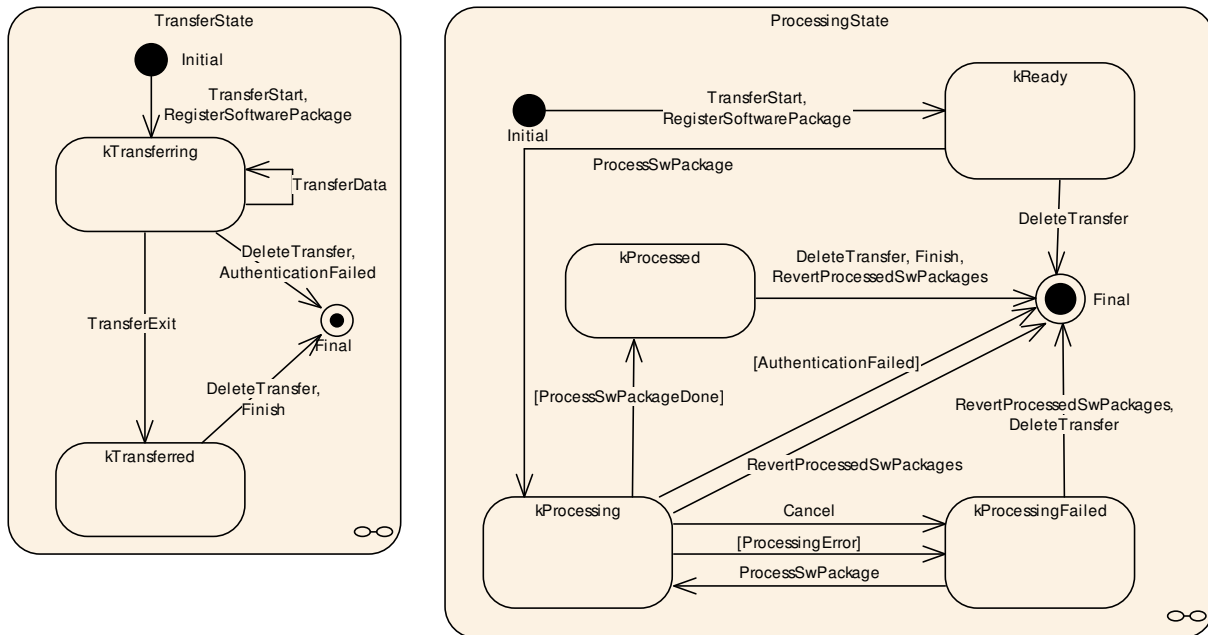


Figure 7.4: State machines for transferring and processing state which, in combination, are representing the lifecycle of **Software Packages. This diagram only represents the life cycle without storing a Software Package (usually referred to as **streaming**).**

A **Software Package** can be processed again if it is stored and reachable to UCM, and if it does not break the Old Version protection [SWS_UCM_00103] (it did not have successful activation).

[SWS_UCM_00007] Data transfer at any time

Upstream requirements: RS_UCM_00013, RS_UCM_00019, RS_UCM_00025

[UCM shall provide support to transfer **Software Packages** at any time when UCM is running. Transferring is decoupled from the UCM Package Management states.]

[SWS_UCM_00272] Transfer block size

Upstream requirements: RS_UCM_00025

[TransferStart shall return blockSize parameter to indicate the maximum block size (unit: bytes, as defined by MaxBlockSize) to be allowed to transfer in one TransferData method call.]

The block size should be aligned to flashing capability in case of Classic Platform capability for instance.

[SWS_UCM_00088] Preparation of data transfer

Upstream requirements: RS_UCM_00013, RS_UCM_00019, RS_UCM_00025

[Data transfer shall be prepared with the method TransferStart or RegisterSoftwarePackage. In the preparation step the number of bytes to be transferred is provided by the client and UCM assigns an id for the **Software Package** to be trans-

ferred. The Transfer State of a `Software Package` (`transferState`) shall be set to `kTransferring`.]

While a `Software Package` is being transferred, if UCM receives a subsequent `TransferStart` call targeting another `Software Package`, UCM should make sure that the sum of the size of both `Software Packages` (the one being transferred and the one requested to be transferred) does not exceed the size of the UCM buffer. Otherwise, the `TransferStart` should raise the `ApplicationError kMemoryInsufficient` and the newly requested transmission should be rejected as described above.

[SWS_UCM_00008] Executing the data transfer

Upstream requirements: `RS_UCM_00013`, `RS_UCM_00019`, `RS_UCM_00025`

[After successful call of `TransferStart` method, the transmission of the `Software Package` block-wise shall be supported by the method `TransferData`. The Transfer State of a `Software Package` (`transferState`) shall stay in `kTransferring`.]

[SWS_UCM_00145] Sequential order of data transfer

Upstream requirements: `RS_UCM_00013`, `RS_UCM_00019`, `RS_UCM_00025`

[The method `TransferData` shall support the parameter `blockCounter` that shall start with 0x01 and be incremented by one for each subsequent block.]

[SWS_UCM_00010] End of data transfer

Upstream requirements: `RS_UCM_00013`, `RS_UCM_00019`, `RS_UCM_00025`

[After transmission of a `Software Package` is completed, the transmission can be finished with method `TransferExit`. The Transfer State of a `Software Package` (`transferState`) shall be set to `kTransferred`.]

`Software Package` contains authentication and integrity tags, which are used during the transfer sequence to authenticate the content of the `Software Package`.

[SWS_UCM_00075] Multiple data transfers in parallel

Upstream requirements: `RS_UCM_00019`

[Handling of multiple data transfers in parallel shall be supported by UCM.]

If UCM provide enough buffering resources for `Software Packages`, several packages could be transferred (in parallel) before they are processed one after the other. The processing (i.e. unpacking and actually applying changes to the `AUTOSAR Adaptive Platform`) of `Software Packages` described by the state `kPreparing` is further detailed in Sect. 7.3.3.

[SWS_UCM_00021] Deleting transferred `Software Packages`

Upstream requirements: `RS_UCM_00018`

[UCM shall provide a method `DeleteTransfer` that shall delete the targeted `Software Package` and free the resources reserved to store that `Software Package`.]

Returned value of `GetSwPackages` can be used to monitor changes in the `consecutiveBytesReceived` to determine any progress the transferring of a `Software Package` is making.

At the invocation of method `GetSwPackages` of the service interface `PackageManagement`, UCM returns the `Software Packages`' identifiers, names, versions, states, consecutive bytes received and consecutive blocks received.

[SWS_UCM_00330] `GetSwPackages` method at `Software Packages` `kTransferring` state

Upstream requirements: `RS_UCM_00010`, `RS_UCM_00002`

[When `Software Package` is in `kTransferring` state, `GetSwPackages` should return empty values except for `TransferID`, `ConsecutiveBytesReceived` and `ConsecutiveBlocksReceived`.]

When `Software Package` is in `kTransferring` state, it is not possible to get versions or names as manifest could not be complete or accessible.

[SWS_UCM_00216] Validity of `TransferId`

Upstream requirements: `RS_UCM_00019`

[The `TransferId` of a `Software Package` shall be invalidated for further use when it reaches final lifecycle state.]

7.3.1.1 Error handling in `TransferStart`

`TransferStart` allocates resources for the client transfer.

[SWS_UCM_00140] UCM insufficient memory

Upstream requirements: `RS_UCM_00013`, `RS_UCM_00019`, `RS_UCM_00025`

[`TransferStart` method shall raise the `ApplicationError` `kMemoryInsufficient` if the UCM buffer has not enough resources to store the corresponding `Software Package`.]

7.3.1.2 Error handling in `TransferData`

`TransferData` executes the following checks. It is recommended to follow the specified order.

[SWS_UCM_00275] `TransferData` error handling order

Upstream requirements: `RS_UCM_00013`, `RS_UCM_00019`, `RS_UCM_00025`

[`TransferData` method shall check the following error conditions and return the respective error code.

1. [SWS_UCM_00208]
2. [SWS_UCM_00203]
3. [SWS_UCM_00204]
4. [SWS_UCM_00243]
5. [SWS_UCM_00205]
6. [SWS_UCM_00206]
7. [SWS_UCM_00289]
8. [SWS_UCM_00207]
9. [SWS_UCM_00294]
10. [SWS_UCM_00098]
11. [SWS_UCM_00092]
12. [SWS_UCM_00245]
13. [SWS_UCM_00103]

]

[SWS_UCM_00208] **TransferData** OperationNotPermitted

Upstream requirements: RS_UCM_00019

[Calling **TransferData** after calling **TransferExit** for a specific TransferId shall raise the error **ApplicationError kOperationNotPermitted**]

[SWS_UCM_00203] **TransferData** InvalidTransferId

Upstream requirements: RS_UCM_00019

[**TransferData** shall raise the error **ApplicationError kTransferIdInvalid** in case an invalid TransferId (An ID that was not initiated by **TransferStart** or marked invalid by **DeleteTransfer**) is sent by the client.]

[SWS_UCM_00204] **TransferData** IncorrectBlock

Upstream requirements: RS_UCM_00014, RS_UCM_00019

[**TransferData** shall raise **ApplicationError kBlockIncorrect** upon receipt of a block counter value that is successfully transmitted to UCM before or upon receipt of an unexpected block counter value.]

[SWS_UCM_00243] **Too big block size received by UCM**

Upstream requirements: RS_UCM_00013, RS_UCM_00014, RS_UCM_00025

[In the case the received block size with **TransferData** exceeds the block size returned by **TransferStart** for the same TransferId, UCM shall raise the **ApplicationError kBlockSizeIncorrect**.]

[SWS_UCM_00205] **TransferData IncorrectSize**

Upstream requirements: RS_UCM_00014, RS_UCM_00019

[In case the transferred Software package size exceeds the provided size in `TransferStart`, `TransferData` shall raise `ApplicationError kSizeIncorrect`]

[SWS_UCM_00206] **TransferData InsufficientMemory**

Upstream requirements: RS_UCM_00013, RS_UCM_00019, RS_UCM_00025

[`TransferData` shall raise the error `ApplicationError kMemoryInsufficient` if resources to store the `Software Package` ceased to exist during the transfer operation.]

[SWS_UCM_00289] **TransferData TransferFailed**

Upstream requirements: RS_UCM_00013

[`TransferData` shall raise the error `ApplicationError kTransferFailed` if UCM cannot persist transferred block.]

[SWS_UCM_00207] **TransferData BlockInconsistent**

Upstream requirements: RS_UCM_00012

[If UCM checks consistency of Block for each `TransferData`, UCM shall raise the error `ApplicationError kBlockInconsistent` in case Consistency check for transferred block fails.]

The `kBlockInconsistent` error is intended to be used by the Flashing Adapter. The Flashing Adapter can calculate additional consistency information for each block internally, e.g. a CRC32 checksum. It can then use UDS protocol to send block data and checksum to the target ECU. In case checksum verification fails, the Flashing Adapter can report the `kBlockInconsistent` error to the V-UCM or diagnostic client application.

As described in section 7.2.2.2 and [11], each `Software Package` has an authentication tag `CryptoServiceCertificate` which protects integrity and authenticity. Therefore additional consistency check information is not needed. If authentication check fails, `kAuthenticationFailed` error is intended to be used instead.

[SWS_UCM_00294] **Unsupported package format for UCM**

Upstream requirements: RS_UCM_00025

[In the case the `Software Package` archiving format is not supported, UCM `TransferData` method shall return `ApplicationError kPackageFormatUnsupported`.]

[SWS_UCM_00098] **Software Package Authentication failure**

Upstream requirements: RS_UCM_00006, RS_UCM_00019, RS_UCM_00025

[UCM shall raise the `ApplicationError kAuthenticationFailed`, if the `Software Package` authentication check fails.]

This error can happen when `TransferData`, `TransferExit` and `ProcessSwPackage` methods are called. When `kAuthenticationFailed` error is raised, it is up to client to decide if a `DeleteTransfer` will be called or not. The behaviour may vary depending on the life cycle, meaning R&D phase or on the field phase.

`TransferData` checks the package version format in accordance to [SWS_UCM_00161] (`kPackageVersionIncompatible`).

`TransferData` checks if the `Software Cluster` to be removed has the corresponding `SoftwareClusterDesign` attribute `installationBehavior` set to `cannotBeRemoved`. If this is the case, UCM shall not remove it in accordance to [SWS_UCM_00245].

`TransferData` checks if the `Software Cluster` version being updated is older than currently present in `Machine` in accordance to [SWS_UCM_00103] (`kOldVersion`).

7.3.1.3 Error handling in `TransferExit`

[SWS_UCM_00276] `TransferExit` error handling order

Upstream requirements: RS_UCM_00013, RS_UCM_00019, RS_UCM_00025

[`TransferExit` method shall check the following error conditions and return the respective error code.

1. [SWS_UCM_00148]
2. [SWS_UCM_00212]
3. [SWS_UCM_00087]
4. [SWS_UCM_00294]
5. [SWS_UCM_00098]
6. [SWS_UCM_00092]
7. [SWS_UCM_00161]
8. [SWS_UCM_00213]
9. [SWS_UCM_00245]
10. [SWS_UCM_00103]

]

[SWS_UCM_00148] Transfer sequence order

Upstream requirements: [RS_UCM_00019](#)

[Calling [TransferExit](#) without calling [TransferData](#) at least once or after [TransferExit](#) is called for a specific TransferID, shall raise the [ApplicationError kOperationNotPermitted](#).]

[SWS_UCM_00212] [TransferExit](#) InvalidTransferId

Upstream requirements: [RS_UCM_00019](#)

[[TransferExit](#) shall raise the error [ApplicationError kTransferIdInvalid](#) in case an invalid TransferId is sent by the client.]

[SWS_UCM_00087] Insufficient amount of data transferred

Upstream requirements: [RS_UCM_00013](#), [RS_UCM_00019](#), [RS_UCM_00025](#)

[When [TransferExit](#) method is called, UCM shall check if all blocks of the [Software Package](#) have been transferred according to the [size](#) parameter of [TransferStart](#). If not UCM shall return [ApplicationError kDataInsufficient](#).]

[TransferExit](#) checks if the [Software Package](#) archiving format is supported in accordance to [\[SWS_UCM_00294\]](#) ([kPackageFormatUnsupported](#)).

[TransferExit](#) checks authentication in accordance to [\[SWS_UCM_00098\]](#) ([kAuthenticationFailed](#)).

[SWS_UCM_00092] Software Package integrity

Upstream requirements: [RS_UCM_00012](#), [RS_UCM_00006](#)

[When [TransferData](#) or [TransferExit](#) method is called, UCM shall raise the [ApplicationError kPackageInconsistent](#) if the [Software Package integrity check](#) fails. This Software Package integrity check may be realized by the UCM via a Software Package Checksum check or via other mechanisms.]

[TransferExit](#) checks the package version format in accordance to [\[SWS_UCM_00161\]](#) ([kPackageVersionIncompatible](#)).

[SWS_UCM_00213] [TransferExit kPackageManifestInvalid](#)

Upstream requirements: [RS_UCM_00012](#)

[[TransferExit](#) shall raise the error [ApplicationError kPackageManifestInvalid](#) upon receipt of an invalid manifest.]

[TransferExit](#) checks if the [Software Cluster](#) to be removed has the corresponding [SoftwareClusterDesign](#) attribute [installationBehavior](#) set to [cannotBeRemoved](#). If this is the case, UCM shall not remove it in accordance to [\[SWS_UCM_00245\]](#).

[TransferExit](#) checks if the [Software Cluster](#) version being updated is older than currently present in [Machine](#) in accordance to [\[SWS_UCM_00103\]](#) ([kOldVersion](#)).

7.3.1.4 Error handling in DeleteTransfer

[SWS_UCM_00283] DeleteTransfer error handling order

Upstream requirements: RS_UCM_00013, RS_UCM_00019, RS_UCM_00025

[DeleteTransfer method shall check the following error conditions and return the respective error code.

1. [SWS_UCM_00214]
2. [SWS_UCM_00215]

]

DeleteTransfer checks if the supplied parameter TransferId is valid.

[SWS_UCM_00214] DeleteTransfer InvalidTransferId

Upstream requirements: RS_UCM_00019

[DeleteTransfer shall raise the error ApplicationError kTransferIdInvalid in case an invalid TransferId is sent by the client.]

[SWS_UCM_00215] DeleteTransfer OperationNotPermitted

Upstream requirements: RS_UCM_00019

[Calling DeleteTransfer during processing or during the processing stream shall raise the error ApplicationError kOperationNotPermitted.]

7.3.2 Processing of Software Packages from a stream

It is also possible to process a Software Package while the transfer is still ongoing. The following requirements apply for this use case.

[SWS_UCM_00165] Processing from stream

Upstream requirements: RS_UCM_00001, RS_UCM_00003, RS_UCM_00004, RS_UCM_00025

[The UCM may support calling ProcessSwPackage directly from stream without waiting to receive the Software Package completely.]

[SWS_UCM_00166] Processing from stream states

Upstream requirements: RS_UCM_00024, RS_UCM_00025

[The UCM shall set the Transferring State to kTransferring and the Processing State to kProcessing if a Software Package is streamed.]

[SWS_UCM_00167] Cancelling streamed packages

Upstream requirements: RS_UCM_00020, RS_UCM_00025

[When Cancel is called, UCM shall remove all temporary data of a streamed Software Package.]

[SWS_UCM_00168] Transferring while processing from stream

Upstream requirements: [RS_UCM_00024](#), [RS_UCM_00025](#)

[The Processing State of a [Software Package](#) ([processingState](#)) state shall remain in [kProcessing](#) when [TransferData](#) is called.]

[SWS_UCM_00169] Finishing transfer while processing from stream

Upstream requirements: [RS_UCM_00024](#), [RS_UCM_00025](#)

[The streamed [Software Package TransferStateType](#) shall be set to [kTransferred](#) and the Processing State shall be set to [kProcessed](#) when [TransferExit](#) is called and the [Software Package](#) is completely processed.]

[SWS_UCM_00200] Failing authentication

Upstream requirements: [RS_VUCM_00039](#), [RS_UCM_00006](#)

[UCM shall delete the [Software Package](#) and its related data processed by [ProcessSwPackage](#) call if authentication is failing at [TransferExit](#) or [ProcessSwPackage](#) call.]

7.3.3 Processing Software Packages

In contrast to package transmission, only one [Software Package](#) can be processed at the same time to ensure consistency of the system. In the following, a software or package processing can involve any combination of an installation, update or removal of applications, configuration data, calibration data or manifests. It is up to the vendor-specific metadata inside a [Software Package](#) to describe the tasks UCM has to perform for its processing. For a removal, this might involve metadata describing which data needs to be deleted. Nevertheless, the communication sequence between the triggering application of the software modification and UCM is the same in any case. For an update of an existing application, the [Software Package](#) can contain only partial data, e.g. just an updated version of the execution manifest. Any UCM Client need to confirm that the UCM has its [CurrentStatus UpdateStateType](#) set to [kPreparing](#) state before starting an [update cycle](#).

[SWS_UCM_00391] Processing a Software Package

Upstream requirements: [RS_UCM_00001](#), [RS_UCM_00004](#)

[Calling [ProcessSwPackage](#) shall set the Processing State ([processingState](#)) to [kProcessing](#) until the processing is finished, which shall transition to [kProcessed](#) state.]

[SWS_UCM_00392] Failed processing a Software Package

Upstream requirements: [RS_UCM_00001](#), [RS_UCM_00004](#)

[If the processing of a [Software Package](#) failed, the Processing State ([processingState](#)) shall transition to [kProcessingFailed](#).]

At the invocation of method `ProcessSwPackage`, UCM processes transferred `Software Package` with `id` argument corresponding to this `Software Package`.

[SWS_UCM_00137] Processing several update `Software Packages`

Upstream requirements: `RS_UCM_00001`, `RS_UCM_00004`

[UCM shall support processing of several `Software Packages`, not in parallel, by calling method `ProcessSwPackage` several times in one `update cycle`.]

The UCM state machine provides some states where no progress (returned by `GetProgress`) is to be made by the UCM, namely `kActivated`, `kRolledBack`, `kRollingBackFailed`. In these states the progress information should be set to values which are indicating that no progress is to be expected (for instance 100%), but fundamentally the values are meaningless for those states. When processing a `Software Package`, the returned progress value and estimated duration correspond to currently active processing.

[SWS_UCM_00003] Cancelling the package processing

Upstream requirements: `RS_UCM_00020`

[On call of `Cancel` method, UCM shall abort the running package processing task, undo the changes to the `Software Cluster` for which processing started and free the reserved resources used for it. The Processing State of a `Software Package` (`processingState`) shall transition to `kProcessingFailed`.]

Cancelling the processing of a `Software Package` should be treated as an error, because an error is returned, and therefore set the `kProcessingFailed` Processing State (`processingState`), from where a new start of the processing is possible.

[SWS_UCM_00024] Revert all processed `Software Packages`

Upstream requirements: `RS_UCM_00008`

[UCM shall provide a method `RevertProcessedSwPackages` to revert all changes done with `ProcessSwPackage`.]

The main difference between a `RevertProcessedSwPackages` and a `Rollback` is that the former can only be performed before the successful activation of the targeted `Software Package(s)` while the latter can only be performed after such activation.

Depending on the capabilities of UCM and of the updated target, `RevertProcessedSwPackages` is used to revert all the changes that have been applied by `ProcessSwPackage`. `Cancel` is also used to revert the changes of the `Software Package` for which processing started by `ProcessSwPackage` method call and identified by `TransferId`. For example, if an application with large resource files is updated “in place” (i.e. in the same partition) then it might not be feasible to revert the update. In this case, to perform a rollback the triggering application could download a `Software Package` to restore a stable version of the application.

7.3.3.1 Error handling during Processing Software Packages

[SWS_UCM_00277] **ProcessSwPackage** error handling order

Upstream requirements: [RS_UCM_00026](#)

[**ProcessSwPackage** method shall check the following error conditions and return the respective error code.

1. [[SWS_UCM_00219](#)]
2. [[SWS_UCM_00017](#)]
3. [[SWS_UCM_00218](#)]
4. [[SWS_UCM_00098](#)]
5. [[SWS_UCM_00161](#)]
6. [[SWS_UCM_00029](#)]
7. [[SWS_UCM_00285](#)]
8. [[SWS_UCM_00231](#)]
9. [[SWS_UCM_00217](#)]
10. [[SWS_UCM_00267](#)]
11. [[SWS_UCM_00104](#)]
12. [[SWS_UCM_00245](#)]
13. [[SWS_UCM_00103](#)]
14. [[SWS_UCM_00150](#)]

]

[SWS_UCM_00219] **ProcessSwPackage** OperationNotPermitted

Upstream requirements: [RS_UCM_00025](#), [RS_UCM_00026](#)

[**ProcessSwPackage** shall raise the error `ApplicationError kOperationNotPermitted` in case the Update State (`updateState`) is not `kPreparing`.]

[SWS_UCM_00017] Sequential **Software Package** Processing

Upstream requirements: [RS_UCM_00001](#), [RS_UCM_00003](#), [RS_UCM_00026](#)

[Once method **ProcessSwPackage** has been called by a client, further calls to the same method shall be rejected with `ApplicationError kServiceBusy` as long as the processing is still ongoing.]

[SWS_UCM_00218] **ProcessSwPackage** InvalidTransferId

Upstream requirements: RS_UCM_00026

[**ProcessSwPackage** shall raise the error `ApplicationError kTransferIdInvalid` in case an invalid `TransferId` is sent by the client.]

ProcessSwPackage checks authentication in accordance to [SWS_UCM_00098] (`kAuthenticationFailed`)

[SWS_UCM_00161] **Check Software Package version compatibility against UCM version**

Upstream requirements: RS_UCM_00007

[At **ProcessSwPackage**, `TransferData` or `TransferExit` calls, UCM shall raise `ApplicationError kPackageVersionIncompatible` if the `MajorVersion` and `MinorVersion` of `minimumSupportedUcmVersion` attribute of the `Software Package` is less than the current `MajorVersion` and `MinorVersion` of UCM as available in `version` attribute. The Processing State of a `Software Package` (`processingState`) shall be transition back to `kProcessingFailed`.]

The `Software Package` is generated by a tooling including a packager which version could not match with the UCM version, leading to manifest interpretation issues for instance.

[SWS_UCM_00029] **Consistency Check of Manifest**

Upstream requirements: RS_UCM_00012

[UCM shall validate the content of the manifest against the schema defined for the meta-data(eg: for missing parameter or for value out of range of the parameter) and shall raise the `ApplicationError kPackageManifestInvalid` if it finds discrepancies there. The Processing State of a `Software Package` (`processingState`) shall transition to `kProcessingFailed`.]

[SWS_UCM_00285] **Removing or updating a Software Cluster not existing in the Machine**

Upstream requirements: RS_UCM_00015

[If a `Software Package`'s action is to remove or update a `Software Cluster` that is not at one of the states `kPresent`, `kRemoved`, `kUpdating` and `kAdded`, UCM shall raise `ApplicationError kSoftwareClusterMissing` when **ProcessSwPackage** is called. The Processing State of a `Software Package` (`processingState`) shall transition to `kProcessingFailed`.]

[SWS_UCM_00231] **ProcessSwPackage** IncompatibleDelta

Upstream requirements: RS_UCM_00007

[**ProcessSwPackage** shall raise the error `ApplicationError kDeltaIncompatible` if `deltaPackageApplicableVersion` is different from the currently installed `SoftwareClusterVersion` of the referenced `SoftwareCluster`.]

[SWS_UCM_00217] ProcessSwPackage InsufficientMemory

Upstream requirements: RS_UCM_00013, RS_UCM_00025

[ProcessSwPackage method shall raise the ApplicationError kMemoryInsufficient if the UCM buffer has not enough resources to process the corresponding Software Package. The Processing State of a Software Package (processingState) shall transition to kReady.]

[SWS_UCM_00267] Error when checksum is not recognised at processing time

Upstream requirements: RS_UCM_00012

[If checksum attribute of ArtifactChecksum or CryptoProvider are not recognised, UCM shall raise the ApplicationError kChecksumDescriptionInvalid. The Processing State of a Software Package (processingState) shall transition to kProcessingFailed.]

[SWS_UCM_00104] Integrity Check of processed Package

Upstream requirements: RS_UCM_00012

[UCM shall raise the ApplicationError kProcessedSoftwarePackageInconsistent if integrity check of the processed Software Packages fails. The Processing State of a Software Package (processingState) shall transition to kProcessingFailed.]

This operation is realized by the UCM to verify that it did not corrupt any files during the processing. This integrity check is vendor specific and may be realized by the UCM by checking the payload Checksum or by any other mechanisms.

ProcessSwPackage checks if the Software Cluster to be removed has the corresponding SoftwareClusterDesign attribute installationBehavior set to cannotBeRemoved. If this is the case, UCM shall not remove it in accordance to [SWS_UCM_00245].

ProcessSwPackage checks if the Software Cluster version being updated is older than currently present in Machine in accordance to [SWS_UCM_00103] (kOldVersion).

[SWS_UCM_00150] Cancellation of a Software Package processing

Upstream requirements: RS_UCM_00024

[ProcessSwPackage method shall raise the ApplicationError kProcessSwPackageCanceled if the Cancel method has been called during the processing of a Software Package. The Processing State of a Software Package (processingState) shall transition to kProcessingFailed.]

[SWS_UCM_00364] Update session failure is triggering production error

Upstream requirements: [RS_UCM_00026](#), [RS_UCM_00024](#)

[If `UpdateRequest` State Management `PrepareUpdate` method returns `ApplicationError kFailed`, UCM shall report `FAILED` to the [UCM_UPDATE_SESSION_FAILED](#). When the update session succeeds, a `PASSED` shall be reported alternatively.]

7.3.3.2 Error handling for Cancel**[SWS_UCM_00278] [Cancel](#) error handling order**

Upstream requirements: [RS_UCM_00020](#)

[[Cancel](#) method shall check the following error conditions and return the respective error code.

1. [\[SWS_UCM_00234\]](#)
2. [\[SWS_UCM_00235\]](#)

]

[SWS_UCM_00234] [Cancel](#) `OperationNotPermitted`

Upstream requirements: [RS_UCM_00020](#)

[[Cancel](#) shall raise the error `ApplicationError kOperationNotPermitted` in case the targeted `Software Package` processing has not yet started or has been already finished.]

[SWS_UCM_00235] [Cancel](#) `InvalidTransferId`

Upstream requirements: [RS_UCM_00020](#)

[[Cancel](#) shall raise the error `ApplicationError kTransferIdInvalid` in case an invalid `TransferId` is sent by the client.]

[SWS_UCM_00372] Cancel Failing is triggering production error

Upstream requirements: [RS_UCM_00020](#)

[When [Cancel](#) is failing, UCM shall report `FAILED` to the [UCM_CANCEL_FAILED](#) production error. When the [Cancel](#) succeeds, a `PASSED` shall be reported alternatively.]

7.3.3.3 Error handling for RevertProcessedSwPackages**[SWS_UCM_00279] [RevertProcessedSwPackages](#) error handling order**

Upstream requirements: [RS_UCM_00020](#)

[[RevertProcessedSwPackages](#) method shall check the following error conditions and return the respective error code.

1. [SWS_UCM_00237]
2. [SWS_UCM_00236]

]

[SWS_UCM_00237] **RevertProcessedSwPackages** OperationNotPermitted

Upstream requirements: RS_UCM_00020

[**RevertProcessedSwPackages** method call shall raise the error **ApplicationError kOperationNotPermitted** in case the processed **Software Packages** are successfully activated or it is called at other states than **kReady** (**Software Package(s)** are finished being processed) or **kProcessing** states.]

[SWS_UCM_00236] **RevertProcessedSwPackages** NotAbleToRevertPackages

Upstream requirements: RS_UCM_00020

[**RevertProcessedSwPackages** shall raise the error **ApplicationError kNotAbleToRevertPackages** in case reverting of processed **Software Packages** have failed.]

7.3.3.4 Error handling for GetSwProcessProgress

7.4 Activation Phase

The Activation Phase is the critical part of an **update cycle** in which the processed software is started and verified. If something does not work as expected the only way out is a rollback of all involved **Software Clusters** to previous versions and ending the **update cycle**.

UCM should notify the activation or rollback of **Software Packages** to other **Functional Clusters** of the AUTOSAR Adaptive Platform. Vendor specific solution dictates to which modules this information is available, in which form and if this is done directly when change is done or when change is executed.

7.4.1 Activation

The **SoftwareCluster** state **kPresent** does not express whether a **SoftwareCluster** is currently executed or not. You can refer to chapter 7.1 Software Cluster Lifecycle for more details about **kPresent** state and sequence diagram 11.4 for more details about activation.

An activation of **SoftwareClusters** is triggered by an **Activate** method call. At beginning of activation, UCM is asking **State Management** for an update session. Once granted, UCM is requesting **State Management** to stop running processes from the outdated **SoftwareClusters**. When processes stopped, UCM makes available to the AUTOSAR Adaptive Platform the updated or installed **SoftwareClusters**, the

core action step of the activation. A verification of the activated `SoftwareClusters` is then performed by requesting `State Management` changing the `SoftwareClusters Function Groups` modes to **Verify**. For an example of activation sequence, you can refer to chapter 11.4

[SWS_UCM_00293] `VerifyUpdate` method

Upstream requirements: [RS_UCM_00024](#)

[At `kVerifying` state and before triggering to `kActivated` state, `UCM` shall call the `State Management UpdateRequest Service Interface VerifyUpdate` method passing the list of `Function Groups` defined in `SoftwareCluster SoftwareClusterFunctionGroupRefCollection.ClaimedFunctionGroup-ForeignRef` attribute of the class.]

In the case a removal of a `Software Cluster`, the `ClaimedFunctionGroups` are removed in the `Machine` configuration. Therefore, `UCM` will call `VerifyUpdate` method passing an empty vector of `Function Groups`.

[SWS_UCM_00107] `Activated` state

Upstream requirements: [RS_UCM_00008](#), [RS_UCM_00030](#)

[`UCM` state `kActivated` shall be set after the new versions of updated `SoftwareClusters` have been verified.]

The state management [4] on the level of execution is handled by the `UCM`'s client controlling the update process.

`UCM` has to be able to update several `SoftwareClusters` for an update campaign. However, these `SoftwareClusters` could have dependencies not satisfied if updates are processed and activated one by one. Therefore, `UCM` splits the activation action from the general package processing.

Applicable version of a `SoftwareCluster` on which to apply delta is included into related `SoftwareCluster`'s `deltaPackageApplicableVersion` attribute

Every call to `ProcessSwPackage` makes necessary preparations of possible actions on the `Software Cluster (ActionType [SWS_UCM_00132])` : `kInstall`, `kRemove`, `kUpdate`, `kUpdateConfiguration`. The `Activate` call finalises the started actions during processing and then `UCM` applies changes at activation that were still pending from processing, like for instance updating the list of processes managed by `Execution Management`.

After `Activate`, the new set of `SoftwareClusters` can be started. Activation covers all the processed `Software Packages` for all the clients.

[SWS_UCM_00022] `Activation of Software Clusters`

Upstream requirements: [RS_UCM_00021](#)

[`UCM` shall activate all the `Software Clusters` extracted from the `Software Packages` when `Activate` is called.]

The activation method could lead to a full system reset. When `Software Package` updates underlying OS, `AUTOSAR Adaptive Platform` or any `Adaptive Application` which is configured to be part of `Function Group MachineFG`, the execution of updated software occurs through system reset by calling `State Management UpdateRequest Service Interface ResetMachine` method. Meta-data of `Software Package` defines the activation method.

[SWS_UCM_00371] UCM rollback after failed Machine restart

Upstream requirements: [RS_UCM_00008](#)

[After the Machine Reset, update sequence is continued by UCM if `UpdateRequest ResetMachineNotifier` field equal `kSuccessful` otherwise if the `ResetMachineNotifier` equals `kFailed`, update verification failed which triggers a rollback.]

In principle, it is possible to activate multiple versions of the same `SoftwareCluster` in one activation step. This could be useful for example with delta package updates but does not apply to firmware updates. The specification does not prohibit to create this kind of chained updates. The decision to use chained updates should be based on safety aspects and the applicability of the underlying update technology, if the update is for a classic or an adaptive platform, if a file system is involved or if the used platform even support it.

[SWS_UCM_00342] Update configuration only

Upstream requirements: [RS_UCM_00028](#), [RS_UCM_00029](#), [RS_UCM_00003](#)

[If `SoftwarePackage.actionType` is set to `updateConfiguration`, then UCM shall use empty vector with `UpdateRequest` interface.]

7.4.1.1 Error handling for `Activate`

[SWS_UCM_00281] `Activate` error handling order

Upstream requirements: [RS_UCM_00026](#)

[`Activate` method shall check the following error conditions and return the respective error code.

1. [\[SWS_UCM_00241\]](#)
2. [\[SWS_UCM_00329\]](#)
3. [\[SWS_UCM_00026\]](#)
4. [\[SWS_UCM_00258\]](#)
5. [\[SWS_UCM_00242\]](#)
6. [\[SWS_UCM_00280\]](#)

]

[SWS_UCM_00241] `Activate` OperationNotPermitted

Upstream requirements: [RS_UCM_00021](#)

[`Activate` shall raise the error `ApplicationError kOperationNotPermitted` in case the UCM Update State (`updateState`) of `CurrentStatus` is not `kPreparing` or there are no processed `Software Packages`.]

[SWS_UCM_00026] Dependency Check

Upstream requirements: [RS_UCM_00007](#)

[During the Update State (`updateState`) `kActivating`, UCM shall perform a `dependency check` to ensure that all the `Software Clusters` having dependencies are not missing any necessary `Software Cluster` as defined by `dependsOn` and do not conflict towards each other as defined by `conflictsTo`, otherwise return `ApplicationError kDependencyMissing`.]

[SWS_UCM_00363] Missing dependencies is triggering production error

Upstream requirements: [RS_UCM_00007](#)

[When at least one or several dependencies are missing, UCM shall report FAILED to the `UCM_MISSING_DEPENDENCIES` production error. When no missing dependencies, a PASSED shall be reported alternatively.]

If `Activate` method cannot establish an Update Session with `State Management`, it returns `kUpdateSessionRejected`, see [\[SWS_UCM_00258\]](#).

[SWS_UCM_00242] `Activate` PrepareUpdateFailed

Upstream requirements: [RS_SM_00001](#)

[`Activate` shall raise the error `ApplicationError kPrepareUpdateFailed` in case of activation state transition failure from `State Management` side.]

[SWS_UCM_00280] `Activate` VerificationFailed

Upstream requirements: [RS_UCM_00021](#)

[`Activate` shall raise the error `ApplicationError kVerificationFailed` in case of verification failure returned by `State Management`.]

7.4.2 Rollback

Rollback can be called in the case of A/B partitions or UCM uses some other solution to maintain backups of updated or removed `Software Packages`.

[SWS_UCM_00110] Rolling-back the software update

Upstream requirements: [RS_UCM_00008](#)

[At Update State `kRollingBack`, UCM shall disable the changes done by the software update by calling `State Management` UpdateRequest Service Interface `PrepareRollback` method for each `Function Group` of the processed `Software`

Cluster in the update cycle. Then UCM shall call State Management UpdateRequest Service Interface ResetMachine method if any Software Cluster requires a machine reboot to be rolled back.]

If a reset of the Machine is not necessary, an implementation specific way to inform Execution Management that a Software Cluster was updated can be performed.

[SWS_UCM_00299] Verify rolled back Software Clusters

Upstream requirements: RS_UCM_00008

[After a UCM successful Rollback using call State Management UpdateRequest Service Interface PrepareRollback method and optional Machine reset or manifest reparse, UCM shall call State Management UpdateRequest Service Interface VerifyUpdate method to confirm that all Software Clusters impacted by update are still safe to be launched.]

[SWS_UCM_00302] Rollback failing is triggering production error

Upstream requirements: RS_UCM_00045, RS_UCM_00008, RS_UCM_00027

[When a Rollback is failing, UCM shall report FAILED to the UCM_ROLLBACK_FAILED production error. When the Rollback succeeds, a PASSED shall be reported alternatively.]

[SWS_UCM_00368] UCM FAILED PREPAREROLLBACK is triggering production error

Upstream requirements: RS_UCM_00045

[If any call of the State Management UpdateRequest Service Interface PrepareRollback returns error kFailed too many times (maximumNumberOfRetries) or for too long (retryIntervalTime with role prepareUpdate), UCM shall report FAILED to the UCM_PREPAREROLLBACK_FAILED production error. When the Prepare Rollback succeeds, a PASSED shall be reported alternatively]

[SWS_UCM_00369] UCM REJECTED PREPAREROLLBACK is triggering production error

Upstream requirements: RS_UCM_00045

[If any call of the State Management UpdateRequest Service Interface PrepareRollback returns error kRejected too many times (maximumNumberOfRetries) or for too long (retryIntervalTime with role prepareUpdate), UCM shall report FAILED to the UCM_PREPAREROLLBACK_REJECTED production error. When the Prepare Rollback accepted, a PASSED shall be reported alternatively.]

7.4.2.1 Error handling for **Rollback**

[SWS_UCM_00282] **Rollback** error handling order

Upstream requirements: RS_UCM_00008

[**Rollback** method shall check the following error conditions and return the respective error code.

1. [SWS_UCM_00239]

]

[SWS_UCM_00239] **Rollback** OperationNotPermitted

Upstream requirements: RS_UCM_00020

[Rollback shall raise the error `ApplicationError kOperationNotPermitted` in case UCM current Update State (`updateState`) is not `kActivated`, `kVerifying` nor `kRollingBackFailed`.]

7.4.3 Boot options

During update process the executed software is switched from original software to updated software and in case of rollback, from updated software to original version. Which version of software is executed is dependent on the UCM state and this is managed by the UCM. In case of platform and OS update the switch between software versions occurs through system reset and depending on the system design the Execution Management [3] might be started before UCM. In this case there can't be direct interface between UCM and Execution Management [3] to define which versions of software would be executed. Instead this would be controlled through persistent controls which are referred as **Boot options** in this document.

[SWS_UCM_00094] Management of executable software

Upstream requirements: RS_UCM_00021

[UCM shall manage which version of software is available for the Execution Management [3] to launch.]

During the `kActivating` Update State (`updateState`), UCM modifies the **Boot options** so that in the next restart for the updated software the new versions will be executed. In the `kRollingBack` state, UCM modifies the **Boot options** so that in the next restart of the updated software the original versions will be executed.

7.5 Cleanup Phase

In the last step of an update cycle the UCM removes every artifact of the performed update which is not of use anymore and leaves the system in a state in which a new update cycle can be performed.

7.5.1 Cleanup

[SWS_UCM_00020] Finishing the packages activation

Upstream requirements: [RS_UCM_00015](#)

[UCM shall provide a method [Finish](#) to commit all the changes and clean up all temporary data of the processed Software Packages.]

UCM should also remove [Software Packages](#), logs or any older versions of changed software to save storage space. It is up to implementer to remove or not the [Software Packages](#).

[SWS_UCM_00259] Ending the update session

Upstream requirements: [RS_UCM_00021](#), [RS_UCM_00018](#)

[UCM shall call [State Management](#) `UpdateRequest` Service Interface `StopUpdateSession` method when UCM is exiting the [kCleaningUp](#) state.]

[SWS_UCM_00240] [Finish](#) `OperationNotPermitted`

Upstream requirements: [RS_UCM_00001](#), [RS_UCM_00026](#)

[Finish shall raise the error [ApplicationError](#) `kOperationNotPermitted` in case there are no activated nor rolled-back [Software Packages](#) pending finalization (i.e UCM Update State ([updateState](#)) is not [kActivated](#) nor [kRolledBack](#).)]

For UCM to be able to free all unneeded resources while processing the [Finish](#) request, it is up to the vendor and platform specific implementation to make sure that obsolete versions of changed [SoftwareClusters](#) are not executed anymore.

[SWS_UCM_00409] Cleanup failing is triggering production error

Upstream requirements: [RS_UCM_00015](#), [RS_UCM_00045](#)

[When a Cleanup is failing, UCM shall report FAILED to the UCM_CLEANUP_FAILED production error. When the Cleanup succeeds, a PASSED shall be reported alternatively.]

7.6 Status Reporting

Once [Software Packages](#) are transferred to UCM, they are ready to be processed to finally apply changes to the [AUTOSAR Adaptive Platform](#). In contrast to the

transmission, the processing and activation tasks have to happen in a strict sequential order.

To give an overview of the update sequence, the global state of **UCM** is described in this section. The details of the processing and activation phases and the methods are specified in the 7.3.3 and 7.4.1.

The global state of **UCM** can be queried using the field **CurrentStatus**. The field consists of a tuple of states, an Update State which indicates the state of the state machine, shown in Fig. 7.5, and the Running State which describes if the current Update state is running or suspended. This diagram does not include behaviour after a reset. Examples can be found of how **UCM** and its **CurrentStatus** field behave including reset management in chapter 11 Sequence Diagram.

[SWS_UCM_00019] Status Field of Package Management

Upstream requirements: [RS_UCM_00024](#)

[The global state of **UCM** shall be provided using the field **CurrentStatus**]

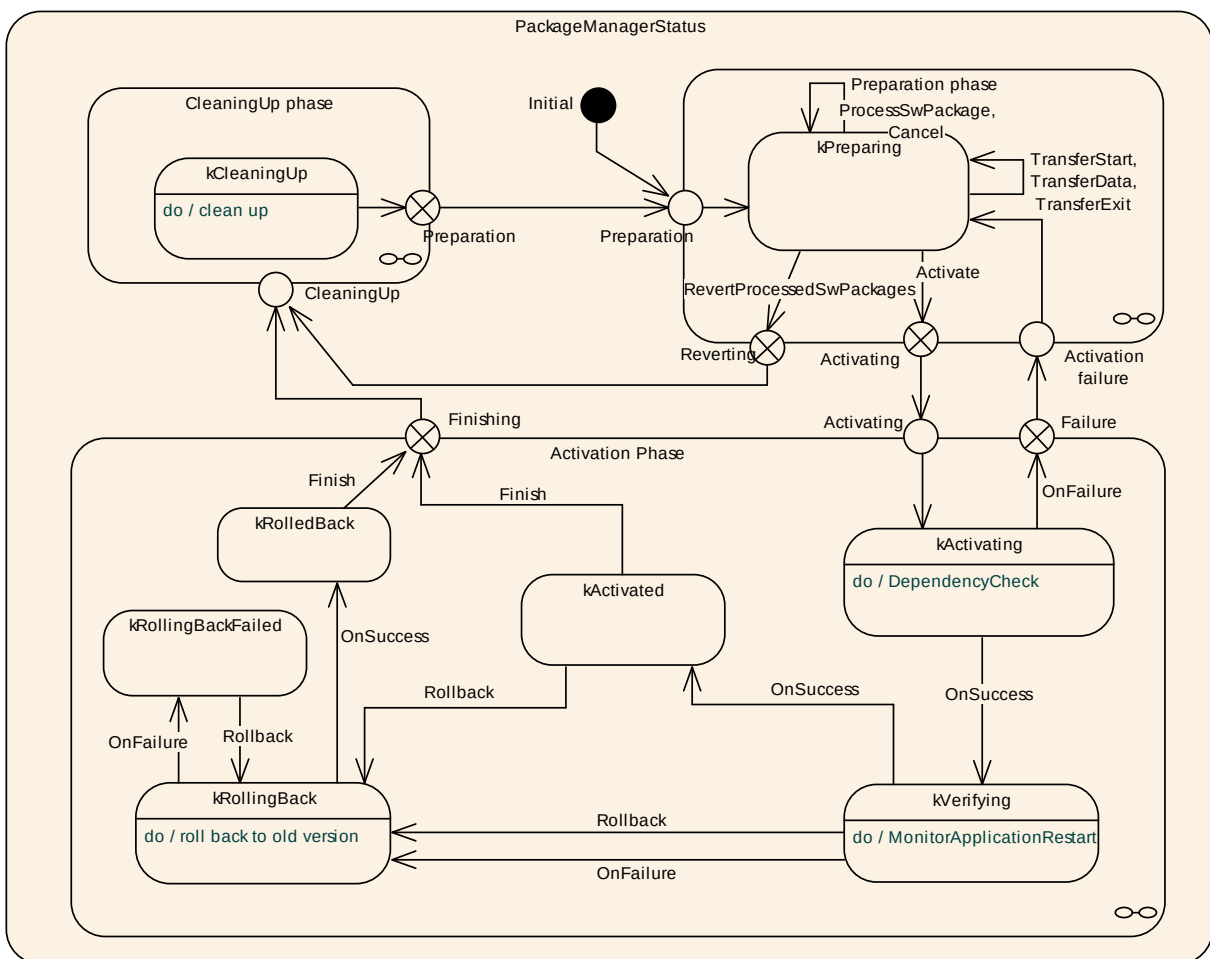


Figure 7.5: State Machine for an update cycle with the **PackageManagement service interface including the update phases.**

UCM supported method calls for each value of the Update State (`updateState`) of the field `CurrentStatus` are shown in Fig. 7.5.

7.6.1 Preparation phase of Package Management

[SWS_UCM_00080] Default state of Package Management

Upstream requirements: RS_UCM_00024

[`kPreparing` shall be the default state.]

[SWS_UCM_00149] Stay in Preparing state

Upstream requirements: RS_UCM_00024

[`kPreparing` shall be kept as `CurrentStatus` when `ProcessSwPackage` returns with error code `kProcessSwPackageCanceled`.]

[SWS_UCM_00081] Processing of Software Packages.

Upstream requirements: RS_UCM_00024

[Calling `ProcessSwPackage` shall preserve the Update State (`updateState`) `kPreparing`. Processing shall only be possible in the Update State (`updateState`) `kPreparing`.]

[SWS_UCM_00266] OperationNotPermitted error and UCM state

Upstream requirements: RS_UCM_00001, RS_UCM_00004

[UCM shall return `ApplicationError` `kOperationNotPermitted` if `ProcessSwPackage` is called by a client with UCM at Update State (`updateState`) of `CurrentStatus` different than `kPreparing`.]

[SWS_UCM_00152] Entering the Preparing state of Package Management after a missing dependency

Upstream requirements: RS_UCM_00024

[`kPreparing` state shall be set when `Activate` fails due to an `ApplicationError` `kDependencyMissing`.]

7.6.2 Activation phase of Package Management

[SWS_UCM_00084] Entering the `kActivating` Update State (`updateState`) of Package Management

Upstream requirements: RS_UCM_00024

[`kActivating` shall be set as Update State (`updateState`) when `Activate` is called. This triggers the dependency check and returns `ApplicationError` `kDependencyMissing` if this check fails.]

[SWS_UCM_00153] Action in kActivating Update State (`updateState`) of Package Management

Upstream requirements: [RS_UCM_00024](#)

[When `kActivating` is set as Update State (`updateState`) and after the `State Management` `UpdateRequest` `Service Interface` `RequestUpdateSession` method call by `UCM`, the `UCM` shall call the `State Management` `UpdateRequest` `Service Interface` `PrepareUpdate` method for the concerned `Software Cluster` including a list of all `Function Groups` belonging to that `Software Cluster`.]

In the case of installation of a new `Software Cluster`, the `ClaimedFunctionGroups` of this new `Software Cluster` are not yet configured in the `Machine`. Therefore, `UCM` will call `PrepareUpdate` method passing an empty vector of `Function Groups`.

[SWS_UCM_00260] PrepareUpdate, VerifyUpdate and PrepareRollback orders

Upstream requirements: [RS_UCM_00007](#), [RS_UCM_00021](#), [RS_UCM_00030](#)

[`UCM` shall compute the order of the `State Management` `UpdateRequest` `Service Interface` `PrepareUpdate`, `VerifyUpdate` and `PrepareRollback` method calls from the dependency model included in the `Software Cluster` manifests.]

[SWS_UCM_00261] PrepareUpdate, VerifyUpdate and PrepareRollback synchronous calls

Upstream requirements: [RS_UCM_00026](#)

[Calls to `State Management` `UpdateRequest` `Service Interface` `PrepareUpdate`, `VerifyUpdate` and `PrepareRollback` methods shall not be concurrent.]

[SWS_UCM_00373] Update preparation rejected

Upstream requirements: [RS_UCM_00026](#)

[If any call of the `State Management` `UpdateRequest` `Service Interface` `PrepareUpdate` method returns error `kRejected` too many times (`UcmRetryStrategy.MaximumNumberOfRetries`) or for too long (`UcmRetryStrategy.RetryIntervalTime` with role `prepareUpdate`), `UCM` shall transition from `kActivating` to `kPreparing` state and report `FAILED` to the `UCM_PREPAREUPDATE_REJECTED`. When the update preparation of the update accepted, a `PASSED` shall be reported alternatively.]

[SWS_UCM_00263] Update preparation failure

Upstream requirements: [RS_UCM_00026](#)

[If any one of the `State Management` `UpdateRequest` `Service Interface` `PrepareUpdate` method returns error `kFailed`, `UCM` shall transition from `kActivating` to `kPreparing` states and report `FAILED` to the `UCM_PREPAREUPDATE_FAILED`. When the update preparation of the update succeeds, a `PASSED` shall be reported alternatively.]

[SWS_UCM_00154] Entering the Verifying state of Package Management

Upstream requirements: [RS_UCM_00024](#)

[`kVerifying` shall be set as Update State (`updateState`) when the `dependency check` have been performed successfully (all dependencies are satisfied) and that the preparation of the `Software Clusters` by the `State Management` has been successfully performed.]

The machine could most likely be restarted in case a A/B partition is used. In case the A/B partition is not used, all affected `Function Groups` or the platform could be restarted. Immediately after the processed `Software Package` has been restarted, a system check has to be performed in order to make sure the machine is able to start up as expected. With this check it is verified that other safety relevant software like `Functional Cluster Platform Health Management` [5] is running and user can be protected from any issues caused by the update after the update has finished.

An update could most likely require to reparse the manifests after performing the atomic activation of the `Software Clusters` (switching A/B partition, changing sym-links, etc.) if a machine reset is not needed.

[SWS_UCM_00085] Entering the kActivated Update State (`updateState`) of Package Management

Upstream requirements: [RS_UCM_00024](#)

[`kActivated` Update State (`updateState`) shall be set when the `VerifyUpdate` method of `State Management` service interface `UpdateRequest` is returned successfully.]

By a successful return of `VerifyUpdate`, UCM assumes all impacted `Function Groups` (the ones related to the processed `Software Package`) have been successfully restarted and verified.

`kVerifying` state gives the client controlling the update process a chance to perform verification test by calling `State Management` `UpdateRequest` Service Interface [SWS_SM_91017] `VerifyUpdate` method, though functionality in verify state can be limited. Client can also coordinate the results over several `AUTOSAR Adaptive Platforms` and still perform a `Rollback` if verification indicates the need for it.

If the system check is successful, the client can decide either to `Rollback` the current active processing so that the previous processed working software gets started, or to perform `Finish` so that the changes of processed software become permanent. By calling `Finish` a clean-up is initiated and in case of A/B partition, a swap between the partitions happens and the newly inactive partition becomes a copy of the newly active partition. In case `Finish` succeeds (including the clean-up), the current `CurrentStatus` changes to `kPreparing`.

For `Rollback` the update software needs to be deactivated and possibly reactivated from original version, e.g. self-update of UCM. For this reason `Rollback` is also performed through two states, similarly as activation. Calling `Rollback` sets UCM into

`kRollingBack` state where original software version is made executable and where original software is activated by the `State Management`. This is started by calling `State Management` `UpdateRequest` Service Interface [SWS_SM_91017] `PrepareRollback` method for each `Software Cluster`. On success, UCM goes to `kRollingBack` state. In this state all the changes introduced during update process have been deactivated and can be cleaned by calling `Finish`.

[SWS_UCM_00126] Entering the `kRollingBack` state after a Rollback call

Upstream requirements: RS_UCM_00008, RS_UCM_00030

[The state `kRollingBack` shall be set when `Rollback` is called.]

[SWS_UCM_00155] Entering the `kRolling-Back` state after a failure in the `kVerify-ing` state

Upstream requirements: RS_UCM_00008, RS_UCM_00030

[The state `kRollingBack` shall be set if any of the `State Management` `UpdateRequest` Service Interface `VerifyUpdate` method calls returns the result `kFailed`.]

[SWS_UCM_00264] Update verification rejected

Upstream requirements: RS_UCM_00030, RS_UCM_00008

[If any call of the `State Management` `UpdateRequest` Service Interface `VerifyUpdate` returns error `kRejected` too many times (`UcmRetryStrategy.MaximumNumberOfRetries`) or for too long (`UcmRetryStrategy.RetryIntervalTime` with role `prepareUpdate`), UCM shall transition to `kRollingBack` state and report FAILED to the `UCM_UPDATE_VERIFICATION_REJECTED` production error. When the update verification accepted, a PASSED shall be reported alternatively.]

[SWS_UCM_00370] UCM FAILED Verification is triggering production error

Upstream requirements: RS_UCM_00030

[If any call of the `State Management` `UpdateRequest` Service Interface `VerifyUpdate` returns error `kFailed` too many times (`UcmRetryStrategy.MaximumNumberOfRetries`) or for too long (`UcmRetryStrategy.RetryIntervalTime` with role `prepareUpdate`), UCM shall transition to `kRollingBack` state and report FAILED to the `UCM_VERIFICATION_FAILED` production error. When the update verification accepted, a PASSED shall be reported alternatively.]

[SWS_UCM_00111] Entering the `kRollingBack` state

Upstream requirements: RS_UCM_00008, RS_UCM_00030

[The state `kRollingBack` shall be set when UCM calls `State Management` `UpdateRequest` Service Interface `PrepareRollback` method.]

[SWS_UCM_00300] Software Cluster failing to rollback

Upstream requirements: [RS_UCM_00024](#)

[If [Rollback](#) is failing, [UCM CurrentStatus](#) shall transition from [kRollingBack](#) to [kRollingBackFailed](#).]

[SWS_UCM_00301] Retry to Rollback again when UCM is in kRollingBackFailed state

Upstream requirements: [RS_UCM_00024](#)

[If [Rollback](#) method is called while being at [kRollingBackFailed](#), [UCM CurrentStatus](#) shall transition from [kRollingBackFailed](#) to [kRollingBack](#).]

7.6.3 Cleaning up phase of Package Management

[SWS_UCM_00146] Entering the Cleaning-up state after a Finish call

Upstream requirements: [RS_UCM_00008](#), [RS_UCM_00030](#)

[The state [kCleaningUp](#) shall be set when [Finish](#) is called and the [UCM](#) starts to perform cleanup actions.]

[SWS_UCM_00331] Delete Software Package at kCleaningUp

Upstream requirements: [RS_UCM_00015](#)

[When [UCM](#) is entering [kCleaningUp](#) from [kActivated](#) then [UCM](#) shall delete the activated [Software Packages](#).]

Due to downgrade protection of [\[SWS_UCM_00103\]](#), the successfully activated [Software Packages](#) cannot be used anymore by [UCM](#).

[SWS_UCM_00162] Entering the Cleaning-up state after a [RevertProcessedSwPackages](#) call

Upstream requirements: [RS_UCM_00008](#), [RS_UCM_00030](#)

[The state [kCleaningUp](#) shall be set when [RevertProcessedSwPackages](#) is called in [kPreparing Update State \(updateState\)](#) and the [UCM](#) starts to perform cleanup actions.]

[SWS_UCM_00163] Action in Cleaning-up state

Upstream requirements: [RS_UCM_00008](#), [RS_UCM_00030](#)

[When [kCleaningUp](#) state is set, the [UCM](#) shall clean up all data of the processed packages that are not needed anymore.]

[SWS_UCM_00164] Cleaning up of Software Packages

Upstream requirements: [RS_UCM_00008](#), [RS_UCM_00030](#)

[In [kCleaningUp](#) state, the [UCM](#) may remove (from the UCM buffer for instance) the "physical" Software Package (e.g. zip file) that was used to transport the the SoftwareCluster to the [UCM](#).]

[SWS_UCM_00127] Finishing update sequence

Upstream requirements: [RS_UCM_00008](#), [RS_UCM_00030](#)

[[kPreparing](#) shall be set when [Finish](#) is called and the clean-up has been successfully performed. This finishes the update sequence and next sequence can be started.]

[SWS_UCM_00147] Return to the Preparing state from Cleaning-up state

Upstream requirements: [RS_UCM_00024](#)

[[kPreparing](#) state shall be set when the Clean-up operation has been completed successfully.]

7.6.4 Suspend and resume

Some steps in the [UCM](#) state machine are able to take up valuable time and resources which have to be released in some scenarios, e.g. shutting down the ECU. To handle such situations in uncritical states, [UCM](#) provides the methods [Suspend](#) and [Resume](#) to be able to pause and continue the execution.

[SWS_UCM_00385] Suspend and resume support

Upstream requirements: [RS_UCM_00047](#)

[[UCM](#) may be able to pause and save any long lasting work currently done and resume it later.]

[SWS_UCM_00386] Suspend and resume not support

Upstream requirements: [RS_UCM_00047](#)

[If suspend and resume is not supported by [UCM](#) implementation the methods [Suspend](#) and [Resume](#) shall always return the error [kOperationNotPermitted](#).]

[SWS_UCM_00387] [Suspend](#) the execution of potentially long running Update States

Upstream requirements: [RS_UCM_00047](#)

[If suspend and resume is supported the Running State ([runningState](#)) of the [CurrentStatus](#) shall be set to [kSuspended](#) if [Suspend](#) has been called and any work shall be saved. This shall at least be supported for the Update States ([updateState](#)) [kPreparing](#) or [kCleaningUp](#)]

[SWS_UCM_00388] Resume the execution of potentially long running Update States

Upstream requirements: [RS_UCM_00047](#)

[The Running State of the [CurrentStatus](#) shall be set to [kRunning](#) if [Resume](#) has been called. Any work previously saved shall be resumed.]

[SWS_UCM_00389] Error behaviour for resume

Upstream requirements: [RS_UCM_00047](#)

[If [Resume](#) has been called and the Running State is not set to [kSuspended](#), UCM shall return the error [kOperationNotPermitted](#).]

Even while being suspended, the [UCM](#) still returns information about the currently available [Software Packages](#) and the [Software Clusters](#) which are handled in the current [update cycle](#), as well as the history and the progress of a possible processing of a [Software Package](#).

[SWS_UCM_00390] Error behaviour of service interface during suspension

Upstream requirements: [RS_UCM_00047](#)

[If the Running State is set to [kSuspended](#) all methods of the [PackageManagement](#) service interface shall return the error [kOperationNotPermitted](#) with exception of:

- [GetHistory](#)
- [GetId](#)
- [GetSwClusterChangeInfo](#)
- [GetSwClusterInfo](#)
- [GetSwClusterManifestInfo](#)
- [GetSwPackages](#)
- [GetProgress](#)

]

7.7 Robustness against reset

Failure during over-the-air updates could lead into corrupted or inconsistent software configuration and further updates might be blocked. For this reason [UCM](#) needs to be robust against interruptions like power downs.

[SWS_UCM_00157] Detection of reset

Upstream requirements: [RS_UCM_00027](#)

[At start up [UCM](#) shall identify if uncontrolled reset occurred.]

The way for UCM to detect uncontrolled reset is project specific. UCM could use hardware platform specific registers to detect Soft/Hard reset. Or it could access PHM Functional Cluster to detect uncontrolled reset. UCM could also check that the `CurrentStatus` persistent field does not contain the expected Update State (`updateState`) e.g. expecting the Update State to be `kVerifying` if the reset occurred during the critical phase of an `update cycle`.

[SWS_UCM_00158] Cleanup of interrupted actions

Upstream requirements: [RS_UCM_00027](#)

[After an uncontrolled reset, UCM shall check non volatile memory integrity, recover processed artifacts in case it is corrupted and resume interrupted actions in order to return the system into a state from where UCM can continue serving its Clients.]

After an uncontrolled reset, it can be possible as an example for UCM to confirm consistency of any processed artifacts based on `ArtifactChecksum` class associated to `SoftwareCluster`. If checksum value of an artifact does not match, it can be deleted and processed again.

[SWS_UCM_00270] UCM internal state persistency

Upstream requirements: [RS_UCM_00027](#)

[UCM shall persist `CurrentStatus` state field to be able to resume on-going update after an intended or unintended reboot.]

7.7.1 Boot monitoring

Activation failure during OS and Platform-self updates can lead to a state in which the system is not able to reach a point where UCM and the client are able to function as expected and thus not able to execute the rollback. For these cases the system should include component which is responsible to monitor that the OS and platform will start up correctly. In case of failure, the Boot monitoring component should trigger a reset or modify the boot options to trigger a rollback.

7.8 History

[SWS_UCM_00115] History

Upstream requirements: [RS_UCM_00032](#)

[`GetHistory` method shall retrieve all actions that have been performed by UCM within a specific time window input parameter.]

In the case the UCM Client requests a rollback after a successful activation, `CurrentStatus` field transitioning to `kActivated`, `GetHistory` method will later return

`HistoryType`, with subelement `resolution` of type `ResultType` equal to `kActivatedAndRolledBack`.

[SWS_UCM_00292] History elements ordering

Upstream requirements: RS_UCM_00032

[UCM shall return from `GetHistory` method a vector of `HistoryType` sorted in an increasing chronological order.]

[SWS_UCM_00160] Processing results records

Upstream requirements: RS_UCM_00032

[When UCM is entering `kVerifying`, UCM shall save activation time based on `time-BaseResource` and activation result of processed `Software Packages` in the history.]

[SWS_UCM_00271] Keeping history of failure error code

Upstream requirements: RS_UCM_00032

[UCM shall keep in `HistoryType` subelement `failureError` the last failure error code as described in [SWS_UCM_00136]. If no error occurred, the stored value shall be 0.]

[SWS_UCM_00303] failing to record history

Upstream requirements: RS_UCM_00045

[If UCM is failing to record a new entry in history, UCM shall report a production error: `UCM_HISTORY_RECORD_FAILED`. Any successful history update shall report a pass to this production error.]

7.9 Version Reporting

[SWS_UCM_00030] Report changes

Upstream requirements: RS_UCM_00011

[UCM shall provide a method `GetSwClusterChangeInfo` of the interface service `PackageManagement` to provide the identifiers and versions of the `SoftwareCluster` that are in state `kAdded`, `kUpdating` or `kRemoved`.]

[SWS_UCM_00311] Provide `SoftwareCluster` general information

Upstream requirements: RS_UCM_00002, RS_UCM_00011

[In case the `SoftwareCluster` referred by its name as input parameter to `GetSwClusterManifestInfo` is not in state `kPresent`, UCM shall raise `ApplicationError kSoftwareClusterMissing`.]

7.10 Securing Software Updates

UCM provides service interface using `ara::com`. There is no authentication of the client in UCM's update sequence.

For authentication of the `Software Package`, you can refer to 7.3.1

[SWS_UCM_00103] Update to `Software Cluster` version which is not newer than currently present and than previously removed

Upstream requirements: RS_UCM_00031

[If the `SoftwareClusterVersion` of a `SoftwareCluster` (returned by `GetSwPackages`) is less than or equal to the `SoftwareClusterVersion` of `SoftwareCluster` (currently present returned by `GetSwClusterInfo` or currently processed returned by `GetSwClusterChangeInfo`), the UCM methods `TransferExit`, `TransferData` or `ProcessSwPackage` shall raise the `ApplicationError kOldVersion`, report production error `UCM_OLD_VERSION_PACKAGE` and delete the rejected `Software Package`.]

[SWS_UCM_00190] Reinstallation of older `Software Cluster` version than previously removed

Upstream requirements: RS_UCM_00003, RS_UCM_00031

[New `Software Clusters` getting installed shall be compared with the history of all installed `Software Clusters` to prevent installation of a `Software Cluster` with a lower or equal version than previously installed.]

[SWS_UCM_00202] Trusted Platform compliance

Upstream requirements: RS_EM_00014

[UCM shall ensure that after processing updates, all the necessary changes to comply with the Trusted Platform are applied.]

The authentication tag of the Trusted Platform corresponding to the updated/removed/added executable files should also be updated/removed/added. See also Chapter "Trusted Platform" of the Execution Management [3] for details on the Trusted Platform.

7.11 Functional cluster lifecycle

7.11.1 Startup

[SWS_UCM_00274] UCM initialization

Upstream requirements: RS_UCM_00044

[UCM shall offer its services only after its internal initialization has been completed, and then report **Running** state to `Execution Management`.]

This requirement prevents calling [UCM](#) subordinate API while internal initialization is on-going. The concrete initialization tasks are implementation specific.

7.11.2 Shutdown

There are no requirements of shutdown behaviour from [UCM](#) functional cluster.

7.12 Reporting

7.12.1 Security Events

[SWS_UCM_00403] Security events for UCM

Status: DRAFT

Upstream requirements: [RS_Ids_00810](#)

[

Name	Description	ID
SEV_SW_UPDATE_FAILED	A SW update operation was requested, but it was not successful.	93
SEV_SW_UPDATE_SUCCESS	A SW update operation was executed successfully.	94

]

[SWS_UCM_00399] SEV SW UPDATE FAILED

Upstream requirements: [RS_Ids_00810](#)

[Upon failure to execute a SW update operation (ResultType kActivatedAndRolled-Back, or kVerificationFailed), UCM Shall raise [SEV_SW_UPDATE_FAILED](#).]

[SWS_UCM_00407] Mapping of context data elements for SEV SW UPDATE FAILED

Upstream requirements: [RS_Ids_00810](#)

[UCM shall construct the SEv context data using the following mappings:

- Action: ActionType
- Error Code: Error Code with the application error that was raised during operation, see [\[SWS_UCM_00136\]](#), failureError
- Resolution: ResultType
- SW Name: SwNameType
- SW Version: StrongRevisionLabelString

]

[SWS_UCM_00404] Security event context data definition: SEV_SW_UPDATE_FAILED*Status:* DRAFT*Upstream requirements:* [RS_Ids_00810](#)

[

SEV Name	SEV_SW_UPDATE_FAILED	
ID	93	
Description	A SW update operation was requested, but it was not successful.	
Context Data Version	1	
Context Data	Data Type	Allowed Values
Action	uint8	
ErrorCode	uint8	
Resolution	uint8	
SwName	uint16 [128, encoding UTF-8]	
ReceivedSwVersion	uint16 [32, encoding UTF-8]	

]

[SWS_UCM_00400] SEV SW UPDATE SUCCESS*Upstream requirements:* [RS_Ids_00810](#)

[Upon successful execution of a SW update operation (ResultType kActivated), UCM Shall raise [SEV_SW_UPDATE_SUCCESS](#).]

[SWS_UCM_00408] Mapping of context data elements for SEV SW UPDATE SUCCESS*Upstream requirements:* [RS_Ids_00810](#)

[UCM shall construct the SEv context data using the following mappings:

- Action: ActionType
- SW Name: SwNameType
- Received SW Version: StrongRevisionLabelString

]

[SWS_UCM_00405] Security event context data definition: SEV_SW_UPDATE_SUCCESS*Status:* DRAFT*Upstream requirements:* [RS_Ids_00810](#)

[

SEV Name	SEV_SW_UPDATE_SUCCESS	
ID	94	
Description	A SW update operation was executed successfully.	
Context Data Version	1	
Context Data	Data Type	Allowed Values





SEV Name	SEV_SW_UPDATE_SUCCESS	
Action	uint8	
SwName	uint16 [128, encoding UTF-8]	
ReceivedSwVersion	uint16 [32, encoding UTF-8]	

]

[SWS_UCM_00401] string in the context data is shorter than SecurityEventContextDataElement.maxLength

Upstream requirements: [RS_Ids_00810](#)

[If a string in the context data is shorter than SecurityEventContextDataElement.maxLength, UCM shall construct the context data so that the string is terminated with a termination character and the following context data element is directly appended without any padding.]

[SWS_UCM_00402] string in the context data is longer than SecurityEventContextDataElement.maxLength

Upstream requirements: [RS_Ids_00810](#)

[If a string in the context data is longer than SecurityEventContextDataElement.maxLength, UCM shall truncate the string to SecurityEventContextDataElement.maxLength.]

7.12.2 Log Messages

7.12.2.1 Standardized Logging

During the update process [UCM](#) interacts with [V-UCM](#) and other function clusters. There are multiple events based on the response during the interaction as part of the update process. Therefore, it is important to provide a way to trace update process events within the [UCM](#). The following trace points are introduced to be able to do analysis of important events during an update process.

[SWS_UCM_00332] [Software Package](#) transfer - Log successful [Software Package](#) transfer

Upstream requirements: [RS_UCM_00046](#)

[Whenever [UCM](#) successfully receives a [Software Package](#) (see [\[SWS_UCM_00010\]](#)), [UCM](#) shall log a DltMessage of type SoftwarePackageReceived.]

[SWS_UCM_00333] Software Package transfer - Log failure of Software Package transfer

Upstream requirements: RS_UCM_00046

[Whenever there is failure during `TransferData/TransferExit` in UCM (see [SWS_UCM_00275] and [SWS_UCM_00276]), UCM shall log a `DltMessage` of type `SoftwarePackageTransferFailed`.]

[SWS_UCM_00334] Software Package processing - Log successful Software Package processing

Upstream requirements: RS_UCM_00046

[Whenever UCM is able to process the `Software Package` successfully (see [SWS_UCM_00131]), UCM shall log a `DltMessage` of type `SoftwarePackageProcessed`.]

[SWS_UCM_00335] Software Package processing - Log failure of Software Package processing

Upstream requirements: RS_UCM_00046

[Whenever there is failure during `Software Package` processing (see [SWS_UCM_00277]), UCM shall log a `DltMessage` of type `SoftwarePackageProcessingFailed`.]

[SWS_UCM_00336] Software Cluster activation - Log installation of new Software Cluster

Upstream requirements: RS_UCM_00046

[Whenever UCM has successfully activated new `Software Cluster` (see [SWS_UCM_00022]), UCM shall log a `DltMessage` of type `SoftwareClusterInstalled`.]

[SWS_UCM_00337] Software Cluster activation - Log update of existing Software Cluster

Upstream requirements: RS_UCM_00046

[Whenever UCM has successfully updated `Software Cluster` to a newer version (see [SWS_UCM_00022]), UCM shall log a `DltMessage` of type `SoftwareClusterUpdated`.]

[SWS_UCM_00338] Software Cluster activation - Log removal of existing Software Cluster

Upstream requirements: RS_UCM_00046

[Whenever UCM has successfully removed a `Software Cluster` (see [SWS_UCM_00022]), UCM shall log a `DltMessage` of type `SoftwareClusterRemoved`.]

[SWS_UCM_00339] **Software Cluster** activation failure - Log failure of **Software Cluster** activation

Upstream requirements: [RS_UCM_00046](#)

[Whenever **UCM** fails to activate **Software Cluster** (see [SWS_UCM_00281]), **UCM** shall log a DltMessage of type SoftwareClusterActivationFailed.]

[SWS_UCM_00340] **Software Cluster** rollback - Log rollback of **Software Cluster**

Upstream requirements: [RS_UCM_00046](#)

[Whenever **UCM** performs rollback of **Software Cluster** (see [SWS_UCM_00110]), **UCM** shall log a DltMessage of type SoftwareClusterRolledback.]

[SWS_UCM_00376] LogMessage SoftwarePackageReceived

Status: DRAFT

Upstream requirements: [RS_UCM_00046](#)

Dlt-Message	SoftwarePackageReceived		
Description	Message that is sent by UCM after receiving software package		
MessageId	0x8000c000		
MessageType Info	DLT_LOG_INFO		
Dlt-Argument	ArgumentDescription	ArgumentType	ArgumentUnit
softwarePackageWithTransferId	Software package with transferId	predefined text	
transferId	TransferId	uint8 [16]	NoUnit
received	received	predefined text	

[SWS_UCM_00377] LogMessage SoftwarePackageTransferFailed

Status: DRAFT

Upstream requirements: [RS_UCM_00046](#)

Dlt-Message	SoftwarePackageTransferFailed		
Description	Message that is sent by UCM after failure in TransferData/TransferExit		
MessageId	0x8000c001		
MessageType Info	DLT_LOG_ERROR		
Dlt-Argument	ArgumentDescription	ArgumentType	ArgumentUnit
transferOfSoftwarePackageWithTransferId	Transfer of software package with transferId	predefined text	
transferId	TransferId	uint8 [16]	NoUnit
failedWithApplicationError Code	failed with application error code	predefined text	





errorCode	ErrorCode	uint8 [encoding UTF-8]	
-----------	-----------	------------------------	--

]

[SWS_UCM_00378] LogMessage SoftwarePackageProcessed

Status: DRAFT

Upstream requirements: [RS_UCM_00046](#)

[

Dlt-Message	SoftwarePackageProcessed		
Description	Message that is sent by UCM after processing software package		
MessageId	0x8000c002		
MessageType Info	DLT_LOG_INFO		
Dlt-Argument	ArgumentDescription	ArgumentType	ArgumentUnit
softwarePackageWithTransferId	Software package with transferid	predefined text	
transferId	TransferId	uint8 [16]	NoUnit
isProcessed	is processed	predefined text	

]

[SWS_UCM_00379] LogMessage SoftwarePackageProcessingFailed

Status: DRAFT

Upstream requirements: [RS_UCM_00046](#)

[

Dlt-Message	SoftwarePackageProcessingFailed		
Description	Message that is sent by UCM after failure in processing of software package		
MessageId	0x8000c003		
MessageType Info	DLT_LOG_ERROR		
Dlt-Argument	ArgumentDescription	ArgumentType	ArgumentUnit
processingOfSoftwarePackageWithTransferId	Processing of software package with transferid	predefined text	
transferId	TransferId	uint8 [16]	NoUnit
failedWithApplicationError Code	failed with application error code	predefined text	
errorCode	ErrorCode	uint8 [encoding UTF-8]	

]

[SWS_UCM_00380] LogMessage SoftwareClusterInstalled

Status: DRAFT

Upstream requirements: [RS_UCM_00046](#)

Dlt-Message	SoftwareClusterInstalled		
Description	Message that is sent by UCM after successful installation of new software cluster		
MessageId	0x8000c004		
MessageType Info	DLT_LOG_INFO		
Dlt-Argument	ArgumentDescription	ArgumentType	ArgumentUnit
softwareCluster ShortName	Software cluster shortname	uint8 [encoding UTF-8]	
version	Version	uint8 [encoding UTF-8]	
installed	Installed	predefined text	

[SWS_UCM_00381] LogMessage SoftwareClusterUpdated

Status: DRAFT

Upstream requirements: [RS_UCM_00046](#)

Dlt-Message	SoftwareClusterUpdated		
Description	Message that is sent by UCM after successful update of existing software cluster		
MessageId	0x8000c005		
MessageType Info	DLT_LOG_INFO		
Dlt-Argument	ArgumentDescription	ArgumentType	ArgumentUnit
softwareCluster ShortName	Software cluster shortname	uint8 [encoding UTF-8]	
version	Version	uint8 [encoding UTF-8]	
updated	updated	predefined text	

[SWS_UCM_00382] LogMessage SoftwareClusterRemoved

Status: DRAFT

Upstream requirements: [RS_UCM_00046](#)

Dlt-Message	SoftwareClusterRemoved		
Description	Message that is sent by UCM after successful removal of existing software cluster		
MessageId	0x8000c006		
MessageType Info	DLT_LOG_INFO		
Dlt-Argument	ArgumentDescription	ArgumentType	ArgumentUnit
softwareCluster ShortName	Software cluster shortname	uint8 [encoding UTF-8]	





version	Version	uint8 [encoding UTF-8]	
removed	removed	predefined text	

]

[SWS_UCM_00383] LogMessage SoftwareClusterActivationFailed

Status: DRAFT

Upstream requirements: [RS_UCM_00046](#)

[

Dlt-Message	SoftwareClusterActivationFailed		
Description	Message that is sent by UCM after failure during software cluster activation		
MessageId	0x8000c007		
MessageType Info	DLT_LOG_ERROR		
Dlt-Argument	ArgumentDescription	ArgumentType	ArgumentUnit
activationOf SoftwareCluster	Activation of software cluster	predefined text	
softwareCluster ShortName	Software cluster shortname	uint8 [encoding UTF-8]	
version	Version	uint8 [encoding UTF-8]	
failedWith ApplicationError Code	failed with application error code	predefined text	
errorCode	ErrorCode	uint8 [encoding UTF-8]	

]

[SWS_UCM_00384] LogMessage SoftwareClusterRollback

Status: DRAFT

Upstream requirements: [RS_UCM_00046](#)

[

Dlt-Message	SoftwareClusterRollback		
Description	Message that is sent by UCM after successful implicit/explicit rollback of software cluster		
MessageId	0x8000c008		
MessageType Info	DLT_LOG_INFO		
Dlt-Argument	ArgumentDescription	ArgumentType	ArgumentUnit
softwareCluster ShortName	Software cluster shortname	uint8 [encoding UTF-8]	
version	Version	uint8 [encoding UTF-8]	
rollback	rollback	predefined text	

]

7.12.3 Violation Messages

This functional cluster does not define any violation messages (i.e., DLT messages logged for Violations according to [SWS_CORE_00021]).

7.12.4 Production Errors

This chapter lists all production errors of the [UCM](#).

7.12.4.1 UCM ROLLBACK FAILED

[SWS_UCM_00323] Diagnostic Event: RollBack failed

Upstream requirements: [RS_UCM_00045](#)

[

Diagnostic Event (Error Name)	UCM_ROLLBACK_FAILED
Description	UCM failed to rollback one or several Software Clusters
Monitoring condition	This DTC is set during an Updating context
Failed condition	UCM fails to rollback one or several Software Clusters
Passed condition	UCM succeeds in retrying Rollback

]

7.12.4.2 HISTORY RECORD FAILED

[SWS_UCM_00320] Diagnostic Event: History recording failed

Upstream requirements: [RS_UCM_00045](#)

[

Diagnostic Event (Error Name)	UCM_HISTORY_RECORD_FAILED
Description	UCM failed to record history entry
Monitoring condition	This DTC is set during an Updating context
Failed condition	UCM fails to record history

]

7.12.4.3 CANCEL FAILED

[SWS_UCM_00325] Diagnostic Event: Campaign cancelling failed

Upstream requirements: [RS_UCM_00045](#)

[

Diagnostic Event (Error Name)	UCM_CANCEL_FAILED
Description	UCM failed to stop the ongoing Processing after a call of Cancel method was received
Monitoring condition	This DTC is set during an Updating context
Failed condition	UCM fails to stop the processing of a software package on call of Cancel





Passed condition	Update sequence is completed with Finish()
-------------------------	--

]

7.12.4.4 MISSING DEPENDENCIES

[SWS_UCM_00326] Diagnostic Event: Activation not possible because of missing dependencies

Upstream requirements: [RS_UCM_00045](#)

[

Diagnostic Event (Error Name)	UCM_MISSING_DEPENDENCIES
Description	Software Cluster dependencies are not fulfilled with the set of currently processed Software Clusters
Monitoring condition	This DTC is set during an Updating context
Failed condition	UCM detects an unmet dependency among the new Software Cluster set
Passed condition	UCM detects all dependencies are fulfilled

]

7.12.4.5 OLD VERSION PACKAGE

[SWS_UCM_00327] Diagnostic Event: Installing old software is not allowed

Upstream requirements: [RS_UCM_00045](#)

[

Diagnostic Event (Error Name)	UCM_OLD_VERSION_PACKAGE
Description	Attempt to update to older Software Cluster version than currently present and than previously removed
Failed condition	If there has been an attempt to update a Software Cluster to older version than currently present and than previously removed, UCM shall report this failure as diagnostic error
Passed condition	If UCM succeeded to update, UCM shall report the Prepassed status

]

7.12.4.6 PREPAREUPDATE FAILED

[SWS_UCM_00322] Diagnostic Event: PrepareUpdate call to SM failed

Upstream requirements: [RS_UCM_00045](#)

[

Diagnostic Event (Error Name)	UCM_PREPAREUPDATE_FAILED
Description	SM returned a negative result on call of PrepareUpdate()
Monitoring condition	This DTC is set during an Updating context
Failed condition	UCM fails to call PrepareUpdate
Passed condition	Subsequent call of PrepareUpdate succeeds

]

7.12.4.7 PREPAREUPDATE Rejected

[SWS_UCM_00262] Diagnostic Event: Update preparation rejected

Upstream requirements: [RS_UCM_00026](#)

[

Diagnostic Event (Error Name)	UCM_PREPAREUPDATE_REJECTED
Description	SM returned a negative result on call of PrepareUpdate()
Monitoring condition	This DTC is set during an Updating context
Failed condition	UCM fails to call PrepareUpdate
Passed condition	Subsequent call of PrepareUpdate succeeds

]

7.12.4.8 UPDATE SESSION FAILED

[SWS_UCM_00321] Diagnostic Event: Update session with SM rejected

Upstream requirements: [RS_UCM_00045](#)

[

Diagnostic Event (Error Name)	UCM_UPDATE_SESSION_FAILED
Description	UCM failed to start an update session
Monitoring condition	This DTC is set during an Updating context
Failed condition	SM returns kFailed on RequestUpdateSession
Passed condition	Subsequent call of RequestUpdateSession succeeds

]

7.12.4.9 UPDATE SESSION REJECTED

[SWS_UCM_00375] Diagnostic Event: Update session with SM rejected

Upstream requirements: [RS_UCM_00045](#)

[

Diagnostic Event (Error Name)	UCM_UPDATE_SESSION_REJECTED
Description	UCM failed to start an update session
Monitoring condition	This DTC is set during an Updating context
Failed condition	SM returns kRejected on RequestUpdateSession
Passed condition	Subsequent call of RequestUpdateSession succeeds

]

7.12.4.10 VERIFICATION FAILED

[SWS_UCM_00324] Diagnostic Event: Verification with SM at activation failed

Upstream requirements: [RS_UCM_00045](#)

[

Diagnostic Event (Error Name)	UCM_VERIFICATION_FAILED
Description	SM returned a negative result on the VerifyUpdate call
Monitoring condition	This DTC is set during an Updating context
Failed condition	UCM receives a negative result on calling VerifyUpdate
Passed condition	Update sequence is completed with Finish()

]

7.12.4.11 VERIFICATION REJECTED

[SWS_UCM_00374] Diagnostic Event: Update verification rejected

Upstream requirements: [RS_UCM_00030](#), [RS_UCM_00008](#)

[

Diagnostic Event (Error Name)	UCM_UPDATE_VERIFICATION_REJECTED
Description	SM returned a negative result on the VerifyUpdate call
Monitoring condition	This DTC is set during an Updating context
Failed condition	UCM receives a negative result on calling VerifyUpdate
Passed condition	Update sequence is completed with Finish()

]

7.12.4.12 PREPAREROLLBACK FAILED

[SWS_UCM_00366] Diagnostic Event: for UCM

Upstream requirements: [RS_UCM_00045](#)

[

Diagnostic Event (Error Name)	UCM_PREPAREROLLBACK_FAILED
Description	SM returned a negative result on the PrepareRollback call
Monitoring condition	This DTC is set during an Updating context
Failed condition	UCM receives a negative result on calling PrepareRollback
Passed condition	Prepare rollback sequence is completed with Finish()

]

7.12.4.13 PREPAREROLLBACK REJECTED

[SWS_UCM_00367] Diagnostic Event: for UCM

Upstream requirements: [RS_UCM_00045](#)

[

Diagnostic Event (Error Name)	UCM_PREPAREROLLBACK_REJECTED
Description	SM returned a negative result on the PrepareRollback call
Monitoring condition	This DTC is set during an Updating context
Failed condition	UCM receives a negative result on calling PrepareRollback
Passed condition	Prepare rollback sequence is completed with Finish()

]

7.12.4.14 CLEANUP FAILED

[SWS_UCM_00410] Diagnostic Event: Cleanup failing is triggering production error

Upstream requirements: [RS_UCM_00015](#), [RS_UCM_00045](#)

[

Diagnostic Event (Error Name)	UCM_CLEANUP_FAILED
Description	UCM Failed to clean up after an update cycle
Monitoring condition	This DTC is set during a clean-up context
Failed condition	UCM fails to clean up after an update cycle
Passed condition	UCM succeeds to perform clean up after update cycle

]

8 API specification

There are no APIs defined in this release.

9 Service Interfaces

9.1 Type definitions

This chapter lists all types provided by the UCM.

The following figure is informative and only meant to support reader having global view of UCM types and service interface.

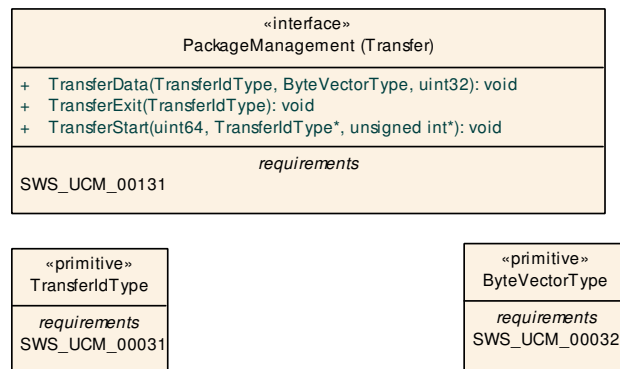


Figure 9.1: UCM composite structure - Transfer related

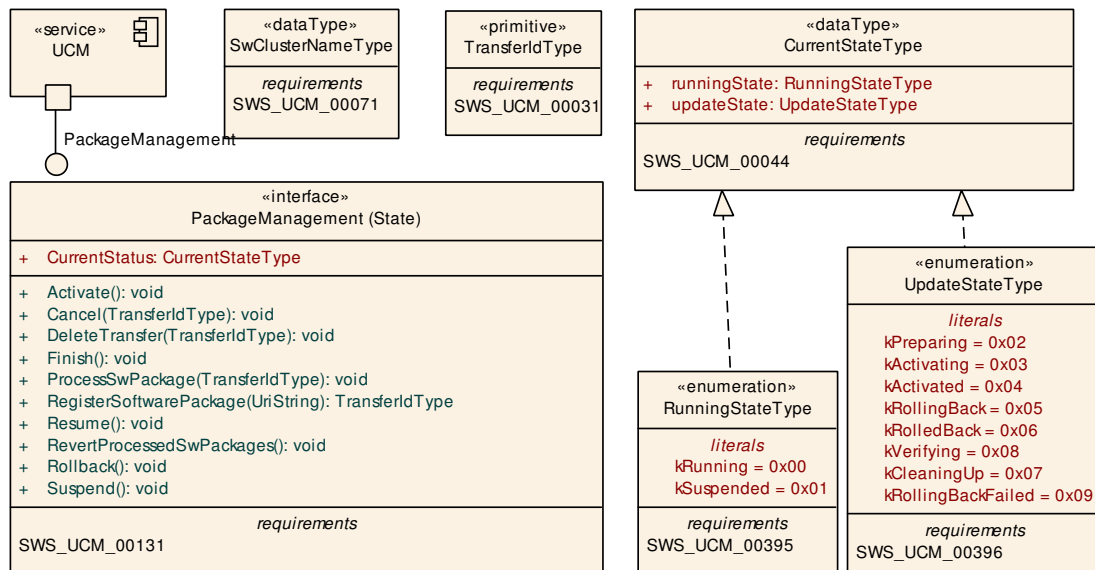


Figure 9.2: UCM composite structure - State related

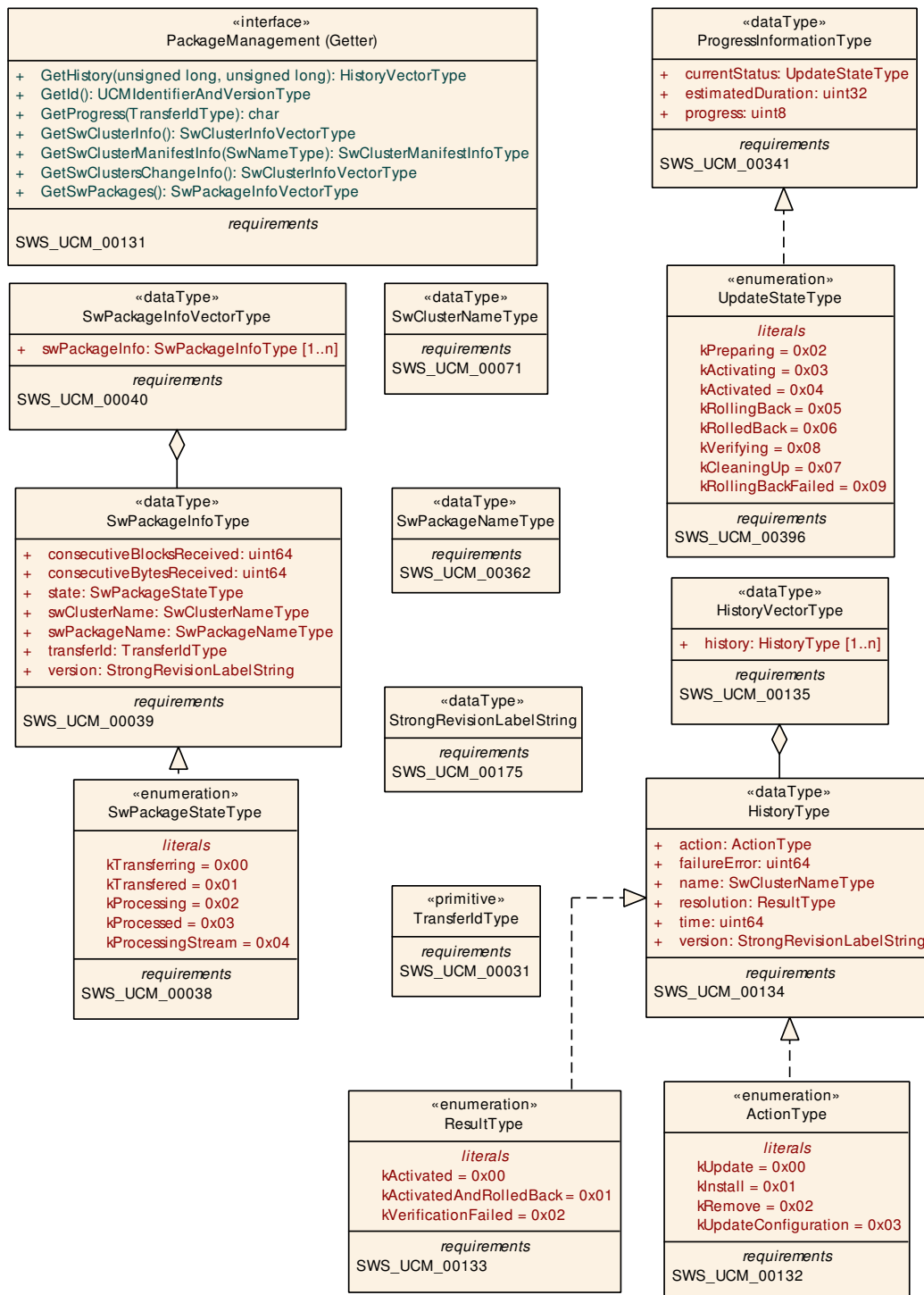


Figure 9.3: UCM composite structure - Information related (Part 1)

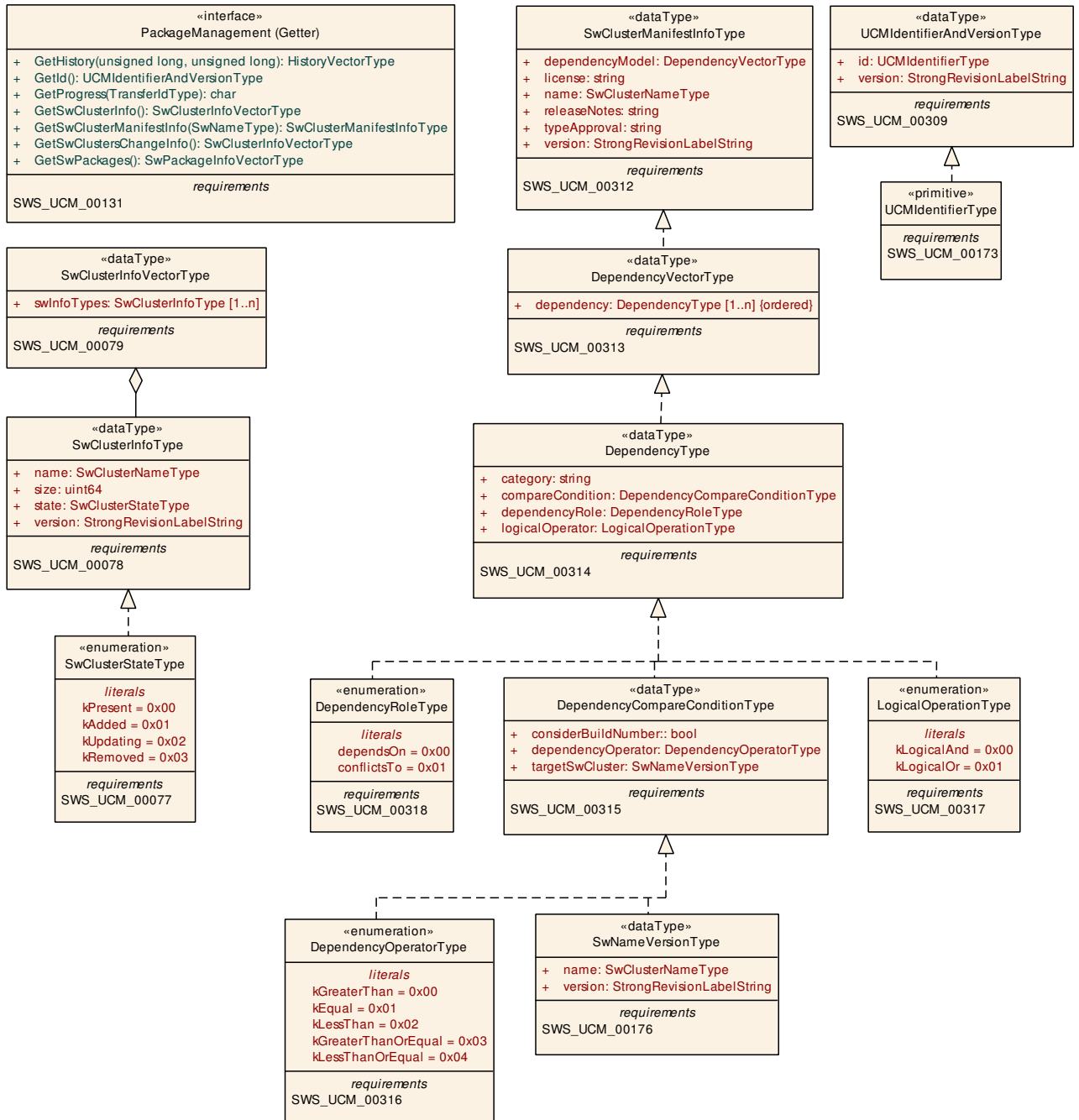


Figure 9.4: UCM composite structure - Information related (Part 2)

9.1.1 UCMLIdentifierType

[SWS_UCM_00173] Definition of ImplementationDataType UCMLIdentifierType

Upstream requirements: [RS_VUCM_00036](#)

[

Name	UCMLIdentifierType
Namespace	ara::ucm
Kind	STRING
Derived from	-
Description	UCM Module Instantiation Identifier.

]

9.1.2 UCMLIdentifierAndVersionType

[SWS_UCM_00309] Definition of ImplementationDataType UCMLIdentifierAndVersionType

Upstream requirements: [RS_VUCM_00035](#)

[

Name	UCMLIdentifierAndVersionType
Namespace	ara::ucm
Kind	STRUCTURE
Sub-elements	id UCMLIdentifierType version StrongRevisionLabelString
Derived from	-
Description	Represents UCM Module Instantion number and version of UCM.

]

9.1.3 TransferIdType

[SWS_UCM_00031] Definition of ImplementationDataType TransferIdType

Upstream requirements: [RS_UCM_00019](#), [RS_UCM_00025](#)

[

Name	TransferIdType
Namespace	ara::ucm
Kind	ARRAY <uint8_t>
Array size	16
Derived from	-
Description	Represents a handle identifier used to reference a particular transfer request.

]

9.1.4 SwPackageNameType

[SWS_UCM_00362] Definition of ImplementationDataType SwPackageNameType

Upstream requirements: [RS_UCM_00002](#)

[

Name	SwPackageNameType
Namespace	ara::ucm
Kind	STRING
Derived from	-
Description	SoftwarePackage shortName attribute inherited from referrable metaClass.

]

9.1.5 ProcessingStateType

[SWS_UCM_00394] Definition of ImplementationDataType ProcessingStateType

Upstream requirements: [RS_UCM_00006](#), [RS_UCM_00010](#), [RS_UCM_00011](#), [RS_UCM_00012](#)

[

Name	ProcessingStateType	
Namespace	ara::ucm	
Kind	TYPE_REFERENCE	
Derived from	uint8_t	
Description	Represents the processing state of a Software Package on the Platform.	
Range / Symbol	Limit	Description
kReady	0x00	Software package is being transferred.
kProcessing	0x02	Software package is currently being processed.
kProcessed	0x03	Software package processing finished.
kProcessingFailed	0x05	Processing of the software package failed.

]

9.1.6 TransferStateType

[SWS_UCM_00393] Definition of ImplementationDataType TransferStateType

Upstream requirements: [RS_UCM_00006](#), [RS_UCM_00010](#), [RS_UCM_00011](#), [RS_UCM_00012](#)

[

Name	TransferStateType
Namespace	ara::ucm
Kind	TYPE_REFERENCE
Derived from	uint8_t

▽



Description	Represents the transfer state of a Software Package on the Platform.	
Range / Symbol	Limit	Description
kTransferring	0x00	Software package is being transferred, i.e. not completely received.
kTransferred	0x01	Software package is completely transferred and ready to be processed.

]

9.1.7 SwClusterNameType

[SWS_UCM_00071] Definition of ImplementationDataType SwClusterNameType

Upstream requirements: [RS_UCM_00002](#)

[

Name	SwClusterNameType
Namespace	ara::ucm
Kind	STRING
Derived from	-
Description	SoftwareCluster shortName attribute inherited from referable metaClass.

]

9.1.8 StrongRevisionLabelString

[SWS_UCM_00175] Definition of ImplementationDataType StrongRevisionLabelString

Upstream requirements: [RS_UCM_00002](#)

[

Name	StrongRevisionLabelString
Namespace	ara::ucm
Kind	STRING
Derived from	-
Description	Primitive type representing SoftwareCluster (SoftwarePackage) version.

]

9.1.9 SwNameVersionType

[SWS_UCM_00176] Definition of ImplementationDataType SwNameVersionType

Upstream requirements: [RS_UCM_00002](#)

[

Name	SwNameVersionType
Namespace	ara::ucm
Kind	STRUCTURE
Sub-elements	swClusterName SwClusterNameType version StrongRevisionLabelString
Derived from	-
Description	Represents the information of a Software Package (Software Cluster) name and version.

]

9.1.10 ProgressInformationType

[SWS_UCM_00341] Definition of ImplementationDataType ProgressInformationType

Upstream requirements: [RS_UCM_00023](#), [RS_UCM_00024](#)

[

Name	ProgressInformationType
Namespace	ara::ucm
Kind	STRUCTURE
Sub-elements	currentStatus UpdateStateType progress uint8_t estimatedDuration uint32_t
Derived from	-
Description	Provides progress information of the work done in current Package Management global state. The progress will be set to a value representing the progress between 0% and 100% (0x00 ... 0x64). The estimatedDuration will be set in seconds, where 0 determines that no estimation is available if the progress is not equal to 100%. The currentStatus will be set to the current state of the Package Management state machine.

]

9.1.11 ByteVectorType

[SWS_UCM_00032] Definition of ImplementationDataType ByteVectorType

Upstream requirements: [RS_UCM_00025](#)

[

Name	ByteVectorType
Namespace	ara::ucm
Kind	VECTOR <uint8_t>
Derived from	-
Description	Byte vector representing raw data.

]

9.1.12 SwPackageStateType

[SWS_UCM_00038] Definition of ImplementationDataType SwPackageStateType

Upstream requirements: [RS_UCM_00024](#)

[

Name	SwPackageStateType
Namespace	ara::ucm
Kind	STRUCTURE
Sub-elements	transferState TransferStateType processingState ProcessingStateType
Derived from	-
Description	Represents the current status of a software package on the platform.

]

9.1.13 SwPackageInfoType

[SWS_UCM_00039] Definition of ImplementationDataType SwPackageInfoType

Upstream requirements: [RS_UCM_00002](#), [RS_UCM_00006](#), [RS_UCM_00010](#), [RS_UCM_00011](#),
[RS_UCM_00012](#)

[

Name	SwPackageInfoType
Namespace	ara::ucm
Kind	STRUCTURE





Sub-elements	swClusterName SwClusterNameType swPackageName SwPackageNameType version StrongRevisionLabelString transferId TransferIdType consecutiveBytesReceived uint64_t consecutiveBlocksReceived uint64_t state SwPackageStateType
Derived from	-
Description	Represents the information of a Software Package.

]

9.1.14 SwPackageInfoVectorType

[SWS_UCM_00040] Definition of ImplementationDataType SwPackageInfoVectorType

Upstream requirements: [RS_UCM_00002](#), [RS_UCM_00006](#), [RS_UCM_00010](#), [RS_UCM_00011](#),
[RS_UCM_00012](#)

[

Name	SwPackageInfoVectorType
Namespace	ara::ucm
Kind	VECTOR < SwPackageInfoType >
Derived from	-
Description	Represents a dynamic size array of Software Packages

]

9.1.15 SwClusterStateType

[SWS_UCM_00077] Definition of ImplementationDataType SwClusterStateType

Upstream requirements: [RS_UCM_00002](#), [RS_UCM_00006](#), [RS_UCM_00010](#), [RS_UCM_00011](#),
[RS_UCM_00012](#)

[

Name	SwClusterStateType	
Namespace	ara::ucm	
Kind	TYPE_REFERENCE	
Derived from	uint8_t	
Description	Represents the state of a SoftwareCluster on the adaptive platform.	
Range / Symbol	Limit	Description
kPresent	0x00	State of a SoftwareCluster that is installed on the adaptive platform and installation has finished.
kAdded	0x01	State of a SoftwareCluster that has been newly installed.
kUpdating	0x02	State of a SoftwareCluster that has been updated.





kRemoved	0x03	State of a SoftwareCluster that is being removed.
----------	------	---

]

9.1.16 SwClusterInfoType

[SWS_UCM_00078] Definition of ImplementationDataType SwClusterInfoType

Upstream requirements: [RS_UCM_00002](#), [RS_UCM_00011](#)

[

Name	SwClusterInfoType
Namespace	ara::ucm
Kind	STRUCTURE
Sub-elements	name SwClusterNameType version StrongRevisionLabelString state SwClusterStateType size uint64_t
Derived from	-
Description	Represents the information of a SoftwareCluster.

]

9.1.17 SwClusterInfoVectorType

[SWS_UCM_00079] Definition of ImplementationDataType SwClusterInfoVectorType

Upstream requirements: [RS_UCM_00002](#), [RS_UCM_00006](#), [RS_UCM_00010](#), [RS_UCM_00011](#),
[RS_UCM_00012](#)

[

Name	SwClusterInfoVectorType
Namespace	ara::ucm
Kind	VECTOR < SwClusterInfoType >
Derived from	-
Description	Represents a dynamic size array of SoftwareClusters

]

9.1.18 CurrentStatusType

[SWS_UCM_00044] Definition of ImplementationDataType CurrentStatusType

Upstream requirements: [RS_UCM_00024](#), [RS_UCM_00026](#)

Name	CurrentStatusType
Namespace	ara::ucm
Kind	STRUCTURE
Sub-elements	updateState UpdateStateType runningState RunningStateType
Derived from	-
Description	Represents the current status as a combination of the update state and the associated running state.

9.1.19 UpdateStateType

[SWS_UCM_00396] Definition of ImplementationDataType UpdateStateType

Upstream requirements: [RS_UCM_00024](#), [RS_UCM_00026](#)

Name	UpdateStateType	
Namespace	ara::ucm	
Kind	TYPE_REFERENCE	
Derived from	uint8_t	
Description	Represents the state of UCM.	
Range / Symbol	Limit	Description
kPreparing	0x02	UCM is ready to prepare an update cycle by getting software packages transferred or processing them.
kActivating	0x03	UCM is performing the dependency check and preparing the activation of the processed Software packages.
kActivated	0x04	Software changes introduced with processed Software Packages has been activated and executed.
kRollingBack	0x05	UCM is reverting changes introduced with processed packages.
kRolledBack	0x06	Software changes introduced with processed Software Packages has been deactivated and original software is executed.
kCleaningUp	0x07	Making sure that the system is in a clean state.
kVerifying	0x08	UCM (via State Management) is checking that the processed packages have been properly restarted.
kRollingBackFailed	0x09	UCM failed to revert changes introduced with processed packages.

9.1.20 RunningStateType

[SWS_UCM_00395] Definition of ImplementationDataType RunningStateType

Upstream requirements: [RS_UCM_00024](#)

[

Name	RunningStateType	
Namespace	ara::ucm	
Kind	TYPE_REFERENCE	
Derived from	uint8_t	
Description	Represents the running status of a update state.	
Range / Symbol	Limit	Description
kRunning	0x00	The current update state is running.
kSuspended	0x01	The current update state is suspended.

]

9.1.21 ActionType

[SWS_UCM_00132] Definition of ImplementationDataType ActionType

Upstream requirements: [RS_UCM_00032](#)

[

Name	ActionType	
Namespace	ara::ucm	
Kind	TYPE_REFERENCE	
Derived from	uint8_t	
Description	Represents the UCM action.	
Range / Symbol	Limit	Description
kUpdate	0x00	Update of a SoftwareCluster.
kInstall	0x01	Installation of a new SoftwareCluster.
kRemove	0x02	Removal of a SoftwareCluster.
kUpdateConfiguration	0x03	Update the configuration of a SoftwareCluster.

]

9.1.22 ResultType

[SWS_UCM_00133] Definition of ImplementationDataType ResultType

Upstream requirements: [RS_UCM_00032](#)

[

Name	ResultType	
Namespace	ara::ucm	
Kind	TYPE_REFERENCE	
Derived from	uint8_t	
Description	Represents the result of UCM action.	
Range / Symbol	Limit	Description
kActivated	0x00	Activation was successful.
kActivatedAndRolledBack	0x01	UCM was activated but rolled back by its Client.
kVerificationFailed	0x02	UCM's action failed.

]

9.1.23 HistoryType

[SWS_UCM_00134] Definition of ImplementationDataType HistoryType

Upstream requirements: [RS_UCM_00032](#)

[

Name	HistoryType	
Namespace	ara::ucm	
Kind	STRUCTURE	
Sub-elements	time uint64_t swClusterName SwClusterNameType version StrongRevisionLabelString action ActionType resolution ResultType failureError uint64_t	
Derived from	-	
Description	Time refers to the verification time of the software cluster (when UCM Subordinate enters kVerifying). UCM shall get time from Time Sync Functional Cluster via UcmToTimeBase ResourceMapping.timeBaseResource	

]

9.1.24 HistoryVectorType

[SWS_UCM_00135] Definition of ImplementationDataType HistoryVectorType

Upstream requirements: [RS_UCM_00032](#)

[

Name	HistoryVectorType
Namespace	ara::ucm
Kind	VECTOR < HistoryType >
Derived from	-
Description	Represents a list of UCM actions

]

9.1.25 SwClusterManifestInfoType

[SWS_UCM_00312] Definition of ImplementationDataType SwClusterManifestInfoType

Upstream requirements: [RS_UCM_00002](#), [RS_UCM_00011](#)

[

Name	SwClusterManifestInfoType
Namespace	ara::ucm
Kind	STRUCTURE
Sub-elements	swClusterName SwClusterNameType version StrongRevisionLabelString typeApproval StringType license StringType releaseNotes StringType dependencyModel DependencyVectorType
Derived from	-
Description	Represents the manifest information of a Software Cluster.

]

9.1.26 DependencyType

[SWS_UCM_00314] Definition of ImplementationDataType DependencyType

Upstream requirements: [RS_UCM_00007](#), [RS_VUCM_00037](#)

[

Name	DependencyType
Namespace	ara::ucm
Kind	STRUCTURE





Sub-elements	dependencyRole DependencyRoleType category StringType logicalOperator LogicalOperationType compareCondition DependencyCompareConditionType
Derived from	-
Description	Represents dependencies between Software Clusters

]

9.1.27 DependencyVectorType

[SWS_UCM_00313] Definition of ImplementationDataType DependencyVector Type

Upstream requirements: [RS_UCM_00007](#), [RS_VUCM_00037](#)

[

Name	DependencyVectorType
Namespace	ara::ucm
Kind	VECTOR < DependencyType >
Derived from	-
Description	Represents a vector of dependencies between Software Clusters

]

9.1.28 DependencyRoleType

[SWS_UCM_00318] Definition of ImplementationDataType DependencyRoleType

Upstream requirements: [RS_UCM_00007](#), [RS_VUCM_00037](#)

[

Name	DependencyRoleType	
Namespace	ara::ucm	
Kind	TYPE_REFERENCE	
Derived from	uint8_t	
Description	Represents the logical operation to be applied between dependencies of Software Clusters	
Range / Symbol	Limit	Description
kDependOn	0x00	depends of
kConflictsTo	0x01	conflicts to

]

9.1.29 LogicalOperationType

[SWS_UCM_00317] Definition of ImplementationDataType LogicalOperationType

Upstream requirements: [RS_UCM_00007](#), [RS_VUCM_00037](#)

[

Name	LogicalOperationType	
Namespace	ara::ucm	
Kind	TYPE_REFERENCE	
Derived from	uint8_t	
Description	Represents the logical operation to be applied between dependencies of Software Clusters	
Range / Symbol	Limit	Description
kLogicalAnd	0x00	Logical AND
kLogicalOr	0x01	Logical OR

]

9.1.30 DependencyCompareConditionType

[SWS_UCM_00315] Definition of ImplementationDataType DependencyCompareConditionType

Upstream requirements: [RS_UCM_00007](#), [RS_VUCM_00037](#)

[

Name	DependencyCompareConditionType	
Namespace	ara::ucm	
Kind	STRUCTURE	
Sub-elements	targetSwCluster SwNameVersionType dependencyOperator DependencyOperatorType considerBuildNumber bool	
Derived from	-	
Description	operator to be applied to target Software Cluster's version	

]

9.1.31 DependencyOperatorType

[SWS_UCM_00316] Definition of ImplementationDataType DependencyOperatorType

Upstream requirements: [RS_UCM_00007](#), [RS_VUCM_00037](#)

[

Name	DependencyOperatorType	
Namespace	ara::ucm	
Kind	TYPE_REFERENCE	
Derived from	uint8_t	
Description	Represents the dependency operator to be applied between versions of Software Cluster	
Range / Symbol	Limit	Description
kGreaterThan	0x00	Greater than
kEqual	0x01	Equal
kLessThan	0x02	Less than
kGreaterThanOrEqual	0x03	Greater than or equal
kLessThanOrEqual	0x04	Less than or equal

]

9.2 Provided Service Interfaces

9.2.1 Package Management

This chapter lists all provided service interfaces of the [UCM](#).

Port

[SWS_UCM_00073] Definition of Port PackageManagement provided by functional cluster UCM

Upstream requirements: [RS_UCM_00001](#), [RS_UCM_00003](#), [RS_UCM_00004](#)

[

Name	PackageManagement		
Kind	ProvidedPort	Interface	PackageManagement
Description	Provide services like receiving, processing and activating Software Packages. Providing history, update status and Software Package and Software Cluster information.		
Variation			

]

Service Interface

[SWS_UCM_00131] Definition of ServiceInterface PackageManagement

Upstream requirements: [RS_UCM_00001](#), [RS_UCM_00002](#), [RS_UCM_00008](#), [RS_UCM_00010](#),
[RS_UCM_00011](#), [RS_UCM_00015](#), [RS_UCM_00018](#), [RS_UCM_00021](#),
[RS_UCM_00023](#), [RS_UCM_00024](#), [RS_UCM_00025](#), [RS_UCM_00032](#)

Name	PackageManagement
Namespace	ara::ucm
Version	1.0
Fields	CurrentStatus
Methods	• GetId • RegisterSoftwarePackage • TransferStart • TransferData • TransferExit • DeleteTransfer • ProcessSwPackage • RevertProcessedSwPackages • Cancel • Activate • Rollback • Finish • GetHistory • GetSwClusterChangeInfo • GetSwClusterInfo • GetSwClusterManifestInfo • GetSwPackages • GetProgress • Suspend • Resume

[SWS_UCM_00361] Definition of Field PackageManagement.CurrentStatus

Upstream requirements: [RS_UCM_00018](#), [RS_UCM_00024](#)

Field	CurrentStatus
Description	The current status of UCM.
Version	1.0
Type	CurrentStatusType
HasGetter	true
HasNotifier	true
HasSetter	false
Enclosing Service Interface	PackageManagement

[SWS_UCM_00343] Definition of Method PackageManagement.GetId

Upstream requirements: [RS_UCM_00001](#), [RS_UCM_00002](#), [RS_UCM_00004](#), [RS_UCM_00010](#)

[

Method	GetId	
Description	Get the UCM Instance Identifier.	
Version	1.0	
FireAndForget	false	
Parameter	id	
	Description	UCM Module Instantiation Identifier.
	Type	UCMIdentifierType
	Variation	
	Direction	OUT
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00344] Definition of Method PackageManagement.RegisterSoftwarePackage

Upstream requirements: [RS_UCM_00019](#), [RS_UCM_00025](#)

[

Method	RegisterSoftwarePackage	
Description	registers a package that could already be in Machine's file system or to be downloaded.	
Version	1.0	
FireAndForget	false	
Parameter	uri	
	Description	uri pointing to software package.
	Type	UriString
	Variation	
	Direction	IN
Parameter	transferId	
	Description	transfer identifier.
	Type	TransferIdType
	Variation	
	Direction	OUT
Application Errors	kInvalidUri	Provided URI is not valid.
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00345] Definition of Method PackageManagement.TransferStartUpstream requirements: [RS_UCM_00019](#), [RS_UCM_00025](#)

Method	TransferStart	
Description	Start the transfer of a Software Package after having received a Vehicle Package. The size of the Software Package to be transferred to UCM must be provided. UCM will generate a Transfer ID for subsequent calls to TransferData, TransferExit, ProcessSwPackage, DeleteTransfer.	
Version	1.0	
FireAndForget	false	
Parameter	size	
	Description	Size (in bytes) of the Software Package to be transferred.
	Type	uint64_t
	Variation	
	Direction	IN
Parameter	id	
	Description	Return TransferId.
	Type	TransferIdType
	Variation	
	Direction	OUT
Parameter	blockSize	
	Description	Size of the blocks to be received with TransferData method.
	Type	uint32_t
	Variation	
	Direction	OUT
Application Errors	kMemoryInsufficient	Insufficient memory to perform operation.
Enclosing Service Interface	PackageManagement	

[SWS_UCM_00346] Definition of Method PackageManagement.TransferDataUpstream requirements: [RS_UCM_00019](#), [RS_UCM_00025](#)

Method	TransferData	
Description	Block-wise transfer of a Software Package to UCM.	
Version	1.0	
FireAndForget	false	
Parameter	id	
	Description	Transfer ID.
	Type	TransferIdType
	Variation	
	Direction	IN
Parameter	data	
	Description	Data block of the Software Package.
	Type	ByteVectorType





	Variation	
	Direction	IN
Parameter	blockCounter	
	Description	Block counter value of the current block.
	Type	uint64_t
	Variation	
	Direction	IN
Application Errors	kOperationNotPermitted	The operation is not supported in the current context.
Application Errors	kTransferIdInvalid	The Transfer ID is invalid.
Application Errors	kBlockIncorrect	The same block number is received twice.
Application Errors	kBlockSizeIncorrect	The size of the block exceeds the provided block size from TransferStart or TransferVehiclePackage.
Application Errors	kSizeIncorrect	The size of the Software or Vehicle Package exceeds the provided size in TransferStart.
Application Errors	kMemoryInsufficient	Insufficient memory to perform operation.
Application Errors	kTransferFailed	UCM cannot persist transferred block.
Application Errors	kBlockInconsistent	Consistency check for transferred block failed.
Application Errors	kPackageFormatUnsupported	The Vehicle Package or Software Package archiving format is not supported.
Application Errors	kAuthenticationFailed	Package authentication failed.
Application Errors	kPackageManifestInvalid	Package manifest could not be read.
Application Errors	kPackageVersionIncompatible	The version of the Software or Vehicle Package to be processed is not compatible with the current version of UCM or V-UCM.
Application Errors	kPackageInconsistent	Package integrity check failed.
Application Errors	kSwcRemovalDenied	Attempt to remove PLATFORM_CORE Software Cluster.
Application Errors	kOldVersion	Software Package version is too old.
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00347] Definition of Method PackageManagement.TransferExit

Upstream requirements: [RS_UCM_00019](#), [RS_UCM_00025](#)

[

Method	TransferExit
Description	Finish the transfer of a Software Package to UCM.
Version	1.0





FireAndForget	false	
Parameter	id	
	Description	Transfer ID of the currently running request.
	Type	TransferIdType
	Variation	
	Direction	IN
Application Errors	kOperationNotPermitted	The operation is not supported in the current context.
Application Errors	kTransferIdInvalid	The Transfer ID is invalid.
Application Errors	kDataInsufficient	TransferExit has been called but total transferred data size does not match expected data size provided with TransferStart call.
Application Errors	kAuthenticationFailed	Package authentication failed.
Application Errors	kPackageFormatUnsupported	The Vehicle Package or Software Package archiving format is not supported.
Application Errors	kPackageInconsistent	Package integrity check failed.
Application Errors	kPackageVersionIncompatible	The version of the Software or Vehicle Package to be processed is not compatible with the current version of UCM or V-UCM.
Application Errors	kPackageManifestInvalid	Package manifest could not be read.
Application Errors	kSwcRemovalDenied	Attempt to remove PLATFORM_CORE Software Cluster.
Application Errors	kOldVersion	Software Package version is too old.
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00348] Definition of Method [PackageManagement.DeleteTransfer](#)

Upstream requirements: [RS_UCM_00019](#), [RS_UCM_00025](#)

[

Method	DeleteTransfer	
Description	Delete a transferred Software Package.	
Version	1.0	
FireAndForget	false	
Parameter	id	
	Description	Transfer ID of the currently running request.
	Type	TransferIdType
	Variation	
	Direction	IN
Application Errors	kTransferIdInvalid	The Transfer ID is invalid.





Application Errors	kOperationNotPermitted	The operation is not supported in the current context.
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00349] Definition of Method [PackageManagement.ProcessSwPackage](#)

Upstream requirements: [RS_UCM_00005](#), [RS_UCM_00015](#), [RS_UCM_00026](#)

[

Method	ProcessSwPackage	
Description	Process a previously transferred Software Package or a partly transferred Software Package from a stream.	
Version	1.0	
FireAndForget	false	
Parameter	id	
	Description	The Transfer ID of the Software Package which should be processed.
	Type	TransferIdType
	Variation	
	Direction	IN
Application Errors	kOperationNotPermitted	The operation is not supported in the current context.
Application Errors	kServiceBusy	Another processing is already ongoing and therefore the current processing request has to be rejected.
Application Errors	kTransferIdInvalid	The Transfer ID is invalid.
Application Errors	kAuthenticationFailed	Package authentication failed.
Application Errors	kPackageVersionIncompatible	The version of the Software or Vehicle Package to be processed is not compatible with the current version of UCM or V-UCM.
Application Errors	kPackageManifestInvalid	Package manifest could not be read.
Application Errors	kSoftwareClusterMissing	The Software Cluster is not present in the Machine.
Application Errors	kDeltaIncompatible	Delta package dependency check failed.
Application Errors	kMemoryInsufficient	Insufficient memory to perform operation.
Application Errors	kChecksumDescriptionInvalid	Checksum attribute not recognised.
Application Errors	kProcessedSoftwarePackageInconsistent	The processed Software Package integrity check has failed.





Application Errors	kSwc1Re-movalDenied	Attempt to remove PLATFORM_CORE Software Cluster.
Application Errors	kOldVersion	Software Package version is too old.
Application Errors	kProcessSw-PackageCan-celed	The processing operation has been interrupted by a Cancel() call.
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00350] Definition of Method [PackageManagement.RevertProcessedSwPackages](#)

Upstream requirements: [RS_UCM_00026](#)

[

Method	RevertProcessedSwPackages	
Description	Revert the changes done by processing (ProcessSwPackage) of one or several software packages.	
Version	1.0	
FireAndForget	false	
Application Errors	kOpera-tionNotPer-mitted	The operation is not supported in the current context.
Application Errors	kNotAble-ToRevert-Packages	RevertProcessedSwPackages failed.
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00351] Definition of Method [PackageManagement.Cancel](#)

Upstream requirements: [RS_UCM_00020](#)

[

Method	Cancel	
Description	This method aborts an ongoing processing of a Software Package.	
Version	1.0	
FireAndForget	false	
Parameter	id	
	Description	The Transfer ID.
	Type	TransferIdType
	Variation	
	Direction	IN
Application Errors	kOpera-tionNotPer-mitted	The operation is not supported in the current context.





Application Errors	kTransferId-Invalid	The Transfer ID is invalid.
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00352] Definition of Method PackageManagement.Activate

Upstream requirements: [RS_UCM_00021](#), [RS_UCM_00030](#)

[

Method	Activate	
Description	This method activates the Software Clusters extracted from the processed Software Packages.	
Version	1.0	
FireAndForget	false	
Application Errors	kOperationNotPermitted	The operation is not supported in the current context.
Application Errors	kDependencyMissing	Activation is not allowed because dependencies are missing.
Application Errors	kPersistenceAllocationFailed	UCM failed to allocate persistent data.
Application Errors	kUpdateSessionRejected	Start of an update session was rejected by State Management
Application Errors	kPrepareUpdateFailed	Error during update preparation step.
Application Errors	kVerificationFailed	State Management returned verification failure
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00353] Definition of Method PackageManagement.Rollback

Upstream requirements: [RS_UCM_00008](#)

[

Method	Rollback	
Description	Rollback the system to the state before the packages were processed.	
Version	1.0	
FireAndForget	false	
Application Errors	kOperationNotPermitted	The operation is not supported in the current context.
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00354] Definition of Method PackageManagement.Finish

Upstream requirements: [RS_UCM_00015](#)

[

Method	Finish	
Description	This method finishes the processing for the current set of processed Software Packages. It does a cleanup of all data of the processing including the sources of the Software Packages.	
Version	1.0	
FireAndForget	false	
Application Errors	kOperationNotPermitted	The operation is not supported in the current context.
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00355] Definition of Method PackageManagement.GetHistory

Upstream requirements: [RS_UCM_00032](#)

[

Method	GetHistory	
Description	Getter method to retrieve all actions that have been performed by UCM.	
Version	1.0	
FireAndForget	false	
Parameter	timestampGE	
	Description	Earliest timestamp (inclusive)
	Type	<code>uint64_t</code>
	Variation	
	Direction	IN
Parameter	timestampLT	
	Description	Latest timestamp (exclusive)
	Type	<code>uint64_t</code>
	Variation	
	Direction	IN
Parameter	history	
	Description	The history of all actions that have been performed by UCM.
	Type	HistoryVectorType
	Variation	
	Direction	OUT
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00356] Definition of Method PackageManagement.GetSwClusterChangeInfo

Upstream requirements: [RS_UCM_00011](#), [RS_UCM_00018](#)

[

Method	GetSwClusterChangeInfo	
Description	This method returns a list pending changes to the set of SoftwareClusters on the adaptive platform. The returned list includes all SoftwareClusters that are to be added, updated or removed. The list of changes is extended in the course of processing Software Packages.	
Version	1.0	
FireAndForget	false	
Parameter	swInfo	
	Description	List of SoftwareClusters that are in state kAdded,kUpdating or kRemoved.
	Type	SwClusterInfoVectorType
	Variation	
	Direction	OUT
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00357] Definition of Method PackageManagement.GetSwClusterInfo

Upstream requirements: [RS_UCM_00002](#)

[

Method	GetSwClusterInfo	
Description	This method returns a list with information of all SoftwareClusters that are in state kPresent.	
Version	1.0	
FireAndForget	false	
Parameter	swInfo	
	Description	List of installed SoftwareClusters that are in state kPresent.
	Type	SwClusterInfoVectorType
	Variation	
	Direction	OUT
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00358] Definition of Method PackageManagement.GetSwClusterManifestInfo

Upstream requirements: [RS_UCM_00002](#)

[

Method	GetSwClusterManifestInfo	
Description	This method returns the general information of a Software Cluster that are in state kPresent.	
Version	1.0	





FireAndForget	false	
Parameter	swClusterName	
	Description	Name of the Software Cluster that is in state kPresent.
	Type	SwClusterNameType
	Variation	
	Direction	IN
Parameter	swInfo	
	Description	Manifest information of Software Cluster at state kPresent.
	Type	SwClusterManifestInfoType
	Variation	
	Direction	OUT
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00359] Definition of Method PackageManagement.GetSwPackages

Upstream requirements: [RS_UCM_00011](#), [RS_UCM_00018](#)

[

Method	GetSwPackages	
Description	This method returns the Software Packages that available in UCM.	
Version	1.0	
FireAndForget	false	
Parameter	packages	
	Description	List of Software Packages.
	Type	SwPackageInfoVectorType
	Variation	
	Direction	OUT
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00360] Definition of Method PackageManagement.GetProgress

Upstream requirements: [RS_UCM_00011](#), [RS_UCM_00018](#)

[

Method	GetProgress	
Description	Get the progress information of the currently active state in the Package Management state machine.	
Version	1.0	
FireAndForget	false	
Parameter	progressInformation	
	Description	The progress information of the current active state in the Package Management state machine.
	Type	ProgressInformationType



△

	Variation	
	Direction	OUT
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00397] Definition of Method PackageManagement.Suspend

Upstream requirements: [RS_UCM_00047](#)

[

Method	Suspend	
Description	This method suspends ongoing time consuming actions or actions related to specific states.	
Version	1.0	
FireAndForget	false	
Application Errors	kOpera- tionNotPer- mitted	The operation is not supported in the current context.
Enclosing Service Interface	PackageManagement	

]

[SWS_UCM_00398] Definition of Method PackageManagement.Resume

Upstream requirements: [RS_UCM_00047](#)

[

Method	Resume	
Description	This method resumes ongoing time consuming actions or actions related to specific states.	
Version	1.0	
FireAndForget	false	
Application Errors	kOpera- tionNotPer- mitted	The operation is not supported in the current context.
Enclosing Service Interface	PackageManagement	

]

9.3 Required Interface

9.3.1 State Management Update Request

UCM requires the UpdateRequest Service Interface [SWS_SM_91017] provided by [State Management](#)

Port

[SWS_UCM_00288] Definition of Port UpdateRequest required by functional cluster UCM*Upstream requirements:* [RS_UCM_00026](#)

[

Name	UpdateRequest		
Kind	RequiredPort	Interface	UpdateRequest
Description	The UpdateRequest interface is intended to be used by UCM to interact with StateManagement to perform updates, installation and removal of SoftwareClusters.		
Variation			

]

9.4 Application Errors

9.4.1 Application Error Domain

9.4.1.1 UCLErrorDomain

This section lists all application errors of the [UCM](#).

[SWS_UCM_00136] Definition of Application Error Domain of functional cluster UCM*Upstream requirements:* [RS_UCM_00006](#), [RS_UCM_00007](#), [RS_UCM_00012](#), [RS_UCM_00013](#), [RS_UCM_00014](#)

[

Name	Code	Description
kAuthenticationFailed	8	Package authentication failed.
kBlockInconsistent	25	Consistency check for transferred block failed.
kBlockIncorrect	2	The same block number is received twice.
kBlockSizeIncorrect	30	The size of the block exceeds the provided block size from TransferStart or TransferVehiclePackage.
kChecksumDescriptionInvalid	35	Checksum attribute not recognised.
kDataInsufficient	6	TransferExit has been called but total transferred data size does not match expected data size provided with TransferStart call.
kDeltaIncompatible	29	Delta package dependency check failed.
kDependencyMissing	21	Activation is not allowed because dependencies are missing.
kInvalidUri	43	Provided URI is not valid.
kMemoryInsufficient	1	Insufficient memory to perform operation.
kNotAbleToRevertPackages	15	RevertProcessedSwPackages failed.
kOldVersion	9	Software Package version is too old.
kOperationNotPermitted	5	The operation is not supported in the current context.
kPackageFormatUnsupported	40	The Vehicle Package or Software Package archiving format is not supported.
kPackageInconsistent	7	Package integrity check failed.
kPackageManifestInvalid	13	Package manifest could not be read.





kPackageUnexpected	32	The Software Package name does not correspond to the RequestedPackage field value.
kPackageVersionIncompatible	24	The version of the Software or Vehicle Package to be processed is not compatible with the current version of UCM or V-UCM.
kPersistencyAllocationFailed	41	UCM failed to allocate persistent data.
kPrepareUpdateFailed	19	Error during update preparation step.
kProcessSwPackageCanceled	22	The processing operation has been interrupted by a Cancel() call.
kProcessedSoftwarePackageInconsistent	23	The processed Software Package integrity check has failed.
kServiceBusy	12	Another processing is already ongoing and therefore the current processing request has to be rejected.
kSizeIncorrect	3	The size of the Software or Vehicle Package exceeds the provided size in TransferStart.
kSoftwareClusterMissing	37	The Software Cluster is not present in the Machine.
kSwclRemovalDenied	39	Attempt to remove PLATFORM_CORE Software Cluster.
kTransferFailed	38	UCM cannot persist transferred block.
kTransferIdInvalid	4	The Transfer ID is invalid.
kUpdateSessionRejected	33	Start of an update session was rejected by State Management
kVerificationFailed	36	State Management returned verification failure

」

10 Configuration

This chapter defines the configuration structure, default values, and semantic constraints for this configuration model.

10.1 Configuration Specification

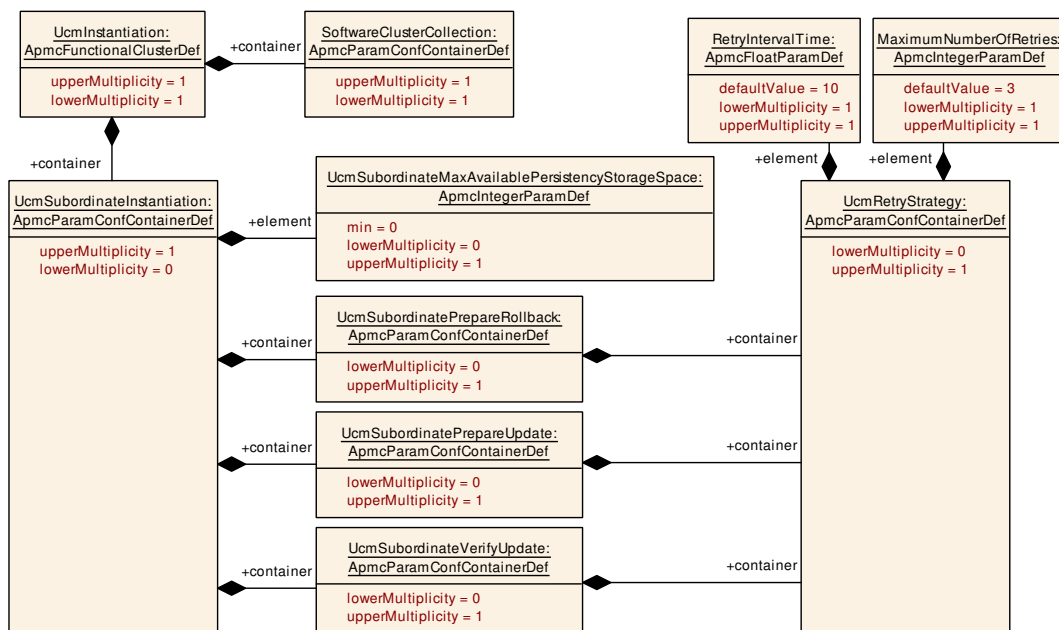


Figure 10.1: Top-level Update Management Configuration

[APMC_UCM_00001] Definition of ApmcFunctionalClusterDef UcmInstantiation

Functional Cluster Name	UcmInstantiation	
Description	This functional cluster definition represents the ability to define the target-configuration of a UCM instantiation.	

Included Containers		
Container Name	Multiplicity	Dependency
SoftwareClusterCollection	1	This container represents the collection of software clusters in the context of the enclosing functional cluster definition.
UcmInstantiationGeneral	1	This container provides some parameters that are applicable on the UCM instantiation level, independently of whether it assumes a master role or a subordinate role.
UcmSubordinateInstantiation	0..1	This container provides the configuration of a UCM Subordinate.

[APMC_UCM_00003] Definition of ApmcParamConfContainerDef SoftwareClusterCollection

Container Name	SoftwareClusterCollection
Parent Container	UcmInstantiation
Description	This container represents the collection of software clusters in the context of the enclosing functional cluster definition.
Multiplicity	1
Configuration Parameters	

No Included Parameters

Included Containers		
Container Name	Multiplicity	Dependency
SoftwareCluster	0..*	This container represents the configuration of a Software Cluster.

]

10.1.1 Ucm Subordinate Configuration

[APMC_UCM_00002] Definition of ApmcParamConfContainerDef UcmSubordinateInstantiation [

Container Name	UcmSubordinateInstantiation
Parent Container	UcmInstantiation
Description	This container provides the configuration of a UCM Subordinate.
Multiplicity	0..1
Configuration Parameters	

Included Parameters		
Parameter Name	Multiplicity	Apmc ID
UcmSubordinateMaxAvailablePersistencyStorageSpace	0..1	[APMC_UCM_00019]

Included Containers		
Container Name	Multiplicity	Dependency
UcmSubordinatePrepareRollback	0..1	This container provides the configuration of prepare rollback retries initiated by the UCM Subordinate.
UcmSubordinatePrepareUpdate	0..1	This container provides the configuration of prepare update retries initiated by the UCM Subordinate.
UcmSubordinateVerifyUpdate	0..1	This container provides the configuration of verify update retries initiated by the UCM Subordinate.

]

[APMC_UCM_00019] Definition of ApmcIntegerParamDef UcmSubordinateMaxAvailablePersistencyStorageSpace [

Parameter Name	UcmSubordinateMaxAvailablePersistencyStorageSpace	
Parent Container	UcmSubordinateInstantiation	
Description	This container names the maximum amount of space available for persistent data handled by the Persistency of installed packages. The UCM needs to figure out from traversing the minimum storage requirement from existing Persistency whether specific packages can be installed from the perspective of available storage space. Note that the minimum storage requirement of Persistency needs to include space for the handling of the storage, which shall be calculated by the tooling that creates the manifest information inside the package.	
Multiplicity	0..1	
Type	ApmcIntegerParamDef	
Range	0 .. 18446744073709551615	
Default value	–	

]

[APMC_UCM_00020] Definition of ApmcParamConfContainerDef UcmSubordinatePrepareRollback [

Container Name	UcmSubordinatePrepareRollback	
Parent Container	UcmSubordinateInstantiation	
Description	This container provides the configuration of prepare rollback retries initiated by the UCM Subordinate.	
Multiplicity	0..1	
Configuration Parameters		

No Included Parameters

Included Containers		
Container Name	Multiplicity	Dependency
UcmRetryStrategy	0..1	This container represents the configuration of a retry strategy for update management.

]

[APMC_UCM_00021] Definition of ApmcParamConfContainerDef UcmSubordinatePrepareUpdate [

Container Name	UcmSubordinatePrepareUpdate	
Parent Container	UcmSubordinateInstantiation	
Description	This container provides the configuration of prepare update retries initiated by the UCM Subordinate.	
Multiplicity	0..1	
Configuration Parameters		

No Included Parameters

Included Containers		
Container Name	Multiplicity	Dependency
UcmRetryStrategy	0..1	This container represents the configuration of a retry strategy for update management.

]

[APMC_UCM_00022] Definition of ApmcParamConfContainerDef UcmSubordinateVerifyUpdate [

Container Name	UcmSubordinateVerifyUpdate
Parent Container	UcmSubordinateInstantiation
Description	This container provides the configuration of verify update retries initiated by the UCM Subordinate.
Multiplicity	0..1
Configuration Parameters	

No Included Parameters

Included Containers		
Container Name	Multiplicity	Dependency
UcmRetryStrategy	0..1	This container represents the configuration of a retry strategy for update management.

]

[APMC_UCM_00023] Definition of ApmcParamConfContainerDef UcmRetryStrategy [

Container Name	UcmRetryStrategy
Parent Container	UcmSubordinatePrepareRollback , UcmSubordinatePrepareUpdate , UcmSubordinateVerifyUpdate
Description	This container represents the configuration of a retry strategy for update management.
Multiplicity	0..1
Configuration Parameters	

Included Parameters		
Parameter Name	Multiplicity	Apmc ID
MaximumNumberOfRetries	1	[APMC_UCM_00024]
RetryIntervalTime	1	[APMC_UCM_00025]

No Included Containers

]

[APMC_UCM_00024] Definition of ApmcIntegerParamDef MaximumNumberOfRetries [

Parameter Name	MaximumNumberOfRetries
Parent Container	UcmRetryStrategy
Description	This parameter defines the maximum number of time the UCM module instantiation shall attempt a retry.
Multiplicity	1
Type	ApmcIntegerParamDef



△

Range	0 .. 18446744073709551615	
Default value	3	

└

[APMC_UCM_00025] Definition of ApmcFloatParamDef RetryIntervalTime

Parameter Name	RetryIntervalTime	
Parent Container	UcmRetryStrategy	
Description	This parameter defines the time (in seconds) between two retry attempts.	
Multiplicity	1	
Type	ApmcFloatParamDef	
Range	-INF .. INF	
Default value	10	

└

10.1.2 General Configuration

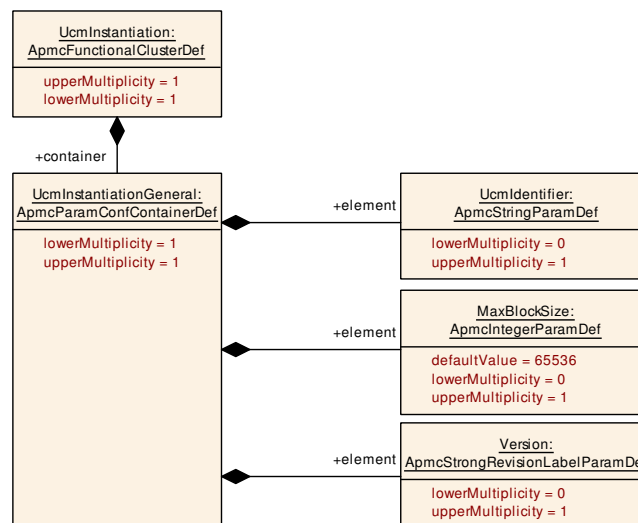


Figure 10.2: Update Management General Configuration

[APMC_UCM_00026] Definition of ApmcParamConfContainerDef UcmInstantiationGeneral

Container Name	UcmInstantiationGeneral
Parent Container	UcmInstantiation
Description	This container provides some parameters that are applicable on the UCM instantiation level, independently of whether it assumes a master role or a subordinate role.
Multiplicity	1
Configuration Parameters	

Included Parameters		
Parameter Name	Multiplicity	Apmc ID
MaxBlockSize	0..1	[APMC_UCM_00028]
UcmIdentifier	0..1	[APMC_UCM_00027]
Version	0..1	[APMC_UCM_00029]

No Included Containers

]

[APMC_UCM_00028] Definition of ApmcIntegerParamDef MaxBlockSize [

Parameter Name	MaxBlockSize	
Parent Container	UcmInstantiationGeneral	
Description	This attribute denotes the maximum block size (unit: bytes) used in the UCM implementation.	
Multiplicity	0..1	
Type	ApmcIntegerParamDef	
Range	0 .. 18446744073709551615	
Default value	65536	

]

[APMC_UCM_00027] Definition of ApmcStringParamDef UcmIdentifier [

Parameter Name	UcmIdentifier	
Parent Container	UcmInstantiationGeneral	
Description	This represents the identification of a UCM.	
Multiplicity	0..1	
Type	ApmcStringParamDef	
Default value	–	
Verbatim String	–	

]

[APMC_UCM_00029] Definition of ApmcStrongRevisionLabelParamDef Version [

Parameter Name	Version	
Parent Container	UcmInstantiationGeneral	
Description	This attribute defines the software version of the UCM on this platform. Note that the definition of the version is required if the ability of the Software Package to require a minimum version of the UCM is utilized.	
Multiplicity	0..1	
Type	ApmcStrongRevisionLabelParamDef	
Default value	–	

]

10.1.3 SoftwareCluster

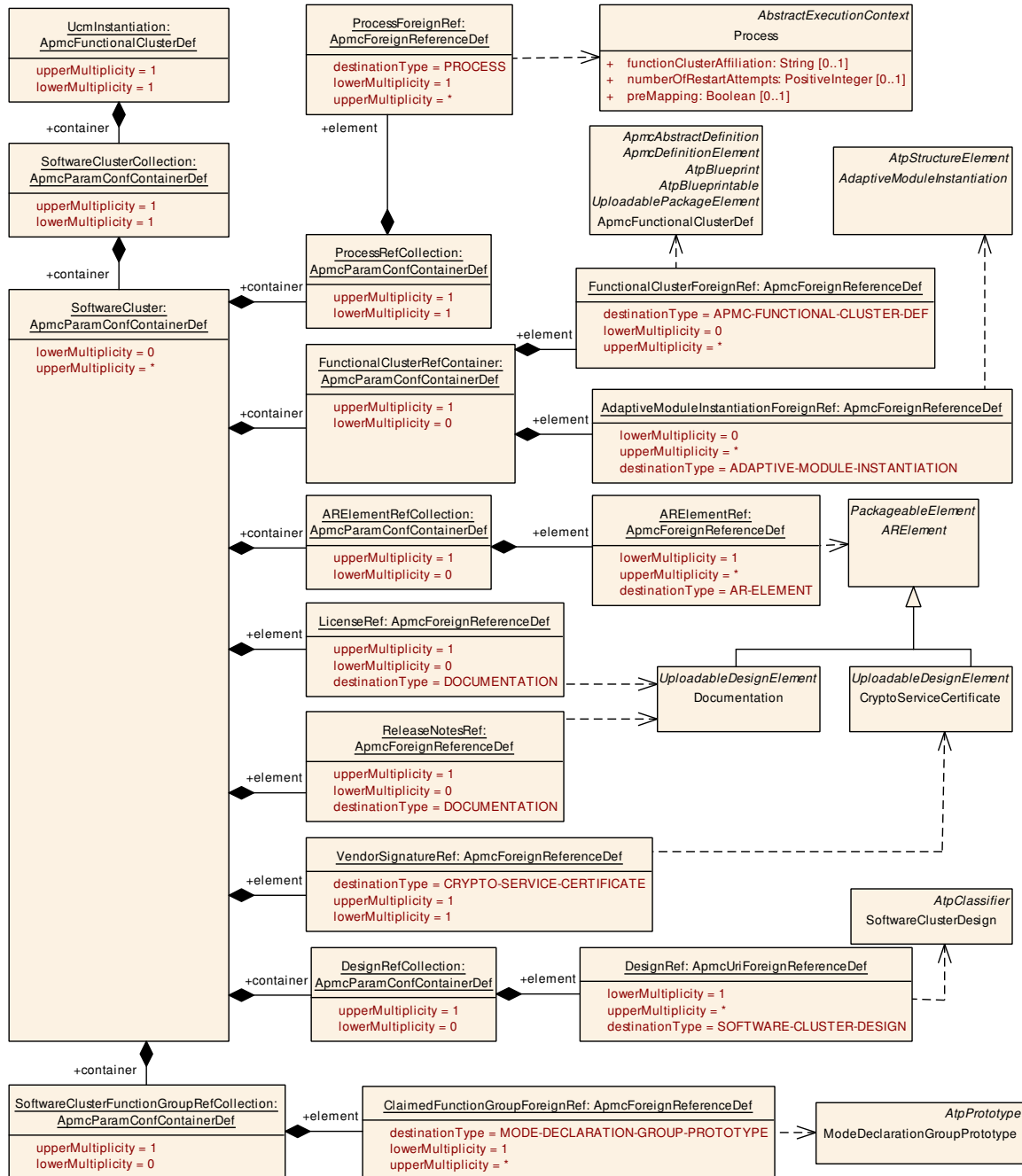


Figure 10.3: Update Management Software Cluster Configuration

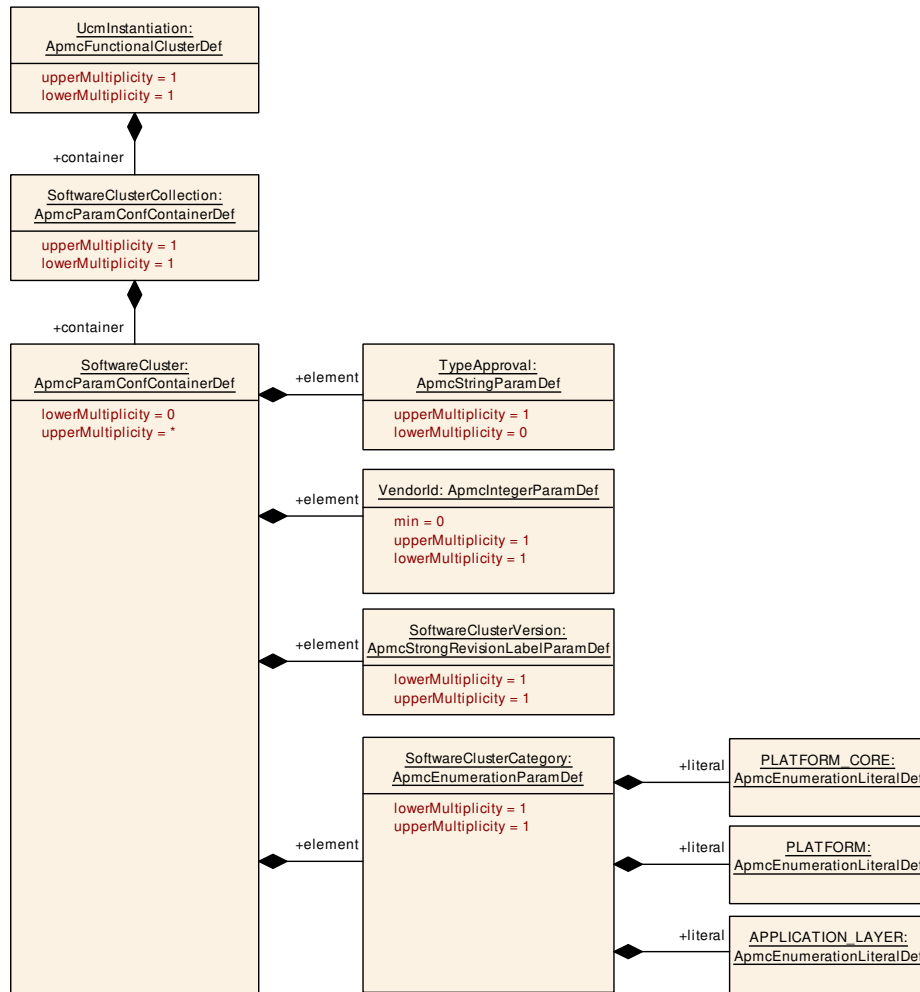


Figure 10.4: Update Management SoftwareCluster primitive attributes

[APMC_UCM_00004] Definition of ApmcParamConfContainerDef SoftwareCluster

Container Name	SoftwareCluster
Parent Container	SoftwareClusterCollection
Description	This container represents the configuration of a Software Cluster.
Multiplicity	0..*
Configuration Parameters	

Included Parameters		
Parameter Name	Multiplicity	Apmc ID
SoftwareClusterCategory	1	[APMC_UCM_00035]
SoftwareClusterVersion	1	[APMC_UCM_00034]
TypeApproval	0..1	[APMC_UCM_00032]
VendorId	1	[APMC_UCM_00033]
DiagnosticPropsForeignRef	0..1	[APMC_UCM_00030]
LicenseRef	0..1	[APMC_UCM_00010]





Included Parameters		
Parameter Name	Multiplicity	Apmc ID
ReleaseNotesRef	0..1	[APMC_UCM_00011]
VendorSignatureRef	1	[APMC_UCM_00012]

Included Containers		
Container Name	Multiplicity	Dependency
ARElementRefCollection	0..1	This container collects all references to ARElements that are relevant in the context of the enclosing software cluster.
ArtifactChecksumCollection	0..1	This container represents the collection of Artifact Checksums in the context pf the enclosing Software Clusters
ArtifactLocatorCollection	0..1	This container represents the collection of Artifact Locators in the context pf the enclosing Software Clusters
DesignRefCollection	0..1	This container collects all references to SoftwareClusterDesigns that are relevant in the context of the enclosing software cluster.
FunctionalClusterRefContainer	0..1	This container collects references to functional clusters that are owned by the enclosing Software Cluster.
ProcessRefCollection	1	This container represents the collection of references to Process within the enclosing Software Cluster.
SoftwareClusterFunctionGroupRefCollection	0..1	This container contains the collection of function groups used by the enclosing Software Cluster.

└

[APMC_UCM_00035] Definition of ApmcEnumerationParamDef SoftwareCluster Category

Parameter Name	SoftwareClusterCategory	
Parent Container	SoftwareCluster	
Description	This parameter defines the category of the enclosing Software Cluster.	
Multiplicity	1	
Type	ApmcEnumerationParamDef	
Range	APPLICATION_LAYER	a Software Cluster of this category represents a driving-relevant function on application level, e.g. a lane keeping assistant, window lift controller, seat positioning. Such a Software Cluster can be installed, removed, and updated.
	PLATFORM	a Software Cluster of this category represents the parts of the platform software (e.g. configuration of functional clusters) that could be installed, removed, and updated.
	PLATFORM_CORE	A Software Cluster of this category typically represents any kind of platform software, e.g. bootloader, hypervisor, OS, adaptive platform module. Such a Software Cluster cannot be removed by a UCM, but updates are possible. Please note that it is not forbidden that such a Software Cluster contains application level software.

└

[APMC_UCM_00034] Definition of ApmcStrongRevisionLabelParamDef Software ClusterVersion [

Parameter Name	SoftwareClusterVersion
Parent Container	SoftwareCluster
Description	This container is used to configure the version of the enclosing Software Cluster.
Multiplicity	1
Type	ApmcStrongRevisionLabelParamDef
Default value	–

]

[APMC_UCM_00032] Definition of ApmcStringParamDef TypeApproval [

Parameter Name	TypeApproval
Parent Container	SoftwareCluster
Description	This parameter contains the type approval of the enclosing Software Cluster.
Multiplicity	0..1
Type	ApmcStringParamDef
Default value	–
Verbatim String	–

]

[APMC_UCM_00033] Definition of ApmcIntegerParamDef VendorId [

Parameter Name	VendorId
Parent Container	SoftwareCluster
Description	This parameter defines the vendor ID.
Multiplicity	1
Type	ApmcIntegerParamDef
Range	0 .. 18446744073709551615
Default value	–

]

[APMC_UCM_00030] Definition of ApmcForeignReferenceDef DiagnosticProps ForeignRef [

Parameter Name	DiagnosticPropsForeignRef
Parent Container	SoftwareCluster
Description	This reference identifies the applicable diagnostic properties.
Multiplicity	0..1
Type	Foreign reference to SOFTWARE-CLUSTER-DIAGNOSTIC-DEPLOYMENT-PROPS

]

[APMC_UCM_00010] Definition of ApmcForeignReferenceDef LicenseRef [

Parameter Name	LicenseRef
Parent Container	SoftwareCluster
Description	This foreign reference identifies the definition of the license of the Software Cluster.
Multiplicity	0..1
Type	Foreign reference to DOCUMENTATION

]

[APMC_UCM_00011] Definition of ApmcForeignReferenceDef ReleaseNotesRef [

Parameter Name	ReleaseNotesRef
Parent Container	SoftwareCluster
Description	This foreign reference identifies the definition of the release notes of the Software Cluster.
Multiplicity	0..1
Type	Foreign reference to DOCUMENTATION

]

[APMC_UCM_00012] Definition of ApmcForeignReferenceDef VendorSignatureRef [

Parameter Name	VendorSignatureRef
Parent Container	SoftwareCluster
Description	This foreign reference identifies the definition of the vendor signature of the Software Cluster.
Multiplicity	1
Type	Foreign reference to CRYPTO-SERVICE-CERTIFICATE

]

[APMC_UCM_00005] Definition of ApmcParamConfContainerDef ProcessRefCollection [

Container Name	ProcessRefCollection
Parent Container	SoftwareCluster
Description	This container represents the collection of references to Process within the enclosing Software Cluster.
Multiplicity	1
Configuration Parameters	

Included Parameters		
Parameter Name	Multiplicity	Apmc ID
ProcessForeignRef	1..*	[APMC_UCM_00041]

No Included Containers

]

[APMC_UCM_00041] Definition of ApmcForeignReferenceDef ProcessForeignRef

Parameter Name	ProcessForeignRef
Parent Container	ProcessRefCollection
Description	This reference identifies a process that is owned by the enclosing Software Cluster.
Multiplicity	1..*
Type	Foreign reference to PROCESS

]

[APMC_UCM_00006] Definition of ApmcParamConfContainerDef FunctionalClusterRefContainer

Container Name	FunctionalClusterRefContainer
Parent Container	SoftwareCluster
Description	This container collects references to functional clusters that are owned by the enclosing Software Cluster.
Multiplicity	0..1
Configuration Parameters	

Included Parameters		
Parameter Name	Multiplicity	Apmc ID
AdaptiveModuleInstantiationForeignRef	0..*	[APMC_UCM_00007]
FunctionalClusterForeignRef	0..*	[APMC_UCM_00040]

No Included Containers

]

[APMC_UCM_00007] Definition of ApmcForeignReferenceDef AdaptiveModuleInstantiationForeignRef

Parameter Name	AdaptiveModuleInstantiationForeignRef
Parent Container	FunctionalClusterRefContainer
Description	This reference identifies a particular adaptive module instantiation that is owned by the enclosing Software Cluster.
Multiplicity	0..*
Type	Foreign reference to ADAPTIVE-MODULE-INSTANTIATION

]

[APMC_UCM_00040] Definition of ApmcForeignReferenceDef FunctionalClusterForeignRef

Parameter Name	FunctionalClusterForeignRef
Parent Container	FunctionalClusterRefContainer
Description	This reference identifies a particular functional cluster configuration that is owned by the enclosing Software Cluster.
Multiplicity	0..*





Type	Foreign reference to APMC-FUNCTIONAL-CLUSTER-DEF
------	--

]

[APMC_UCM_00008] Definition of ApmcParamConfContainerDef ARElementRef Collection [

Container Name	ARElementRefCollection
Parent Container	SoftwareCluster
Description	This container collects all references to ARElements that are relevant in the context of the enclosing software cluster.
Multiplicity	0..1
Configuration Parameters	

Included Parameters		
Parameter Name	Multiplicity	Apmc ID
ARElementRef	1..*	[APMC_UCM_00009]

No Included Containers

]

[APMC_UCM_00009] Definition of ApmcForeignReferenceDef ARElementRef [

Parameter Name	ARElementRef
Parent Container	ARElementRefCollection
Description	This reference identifies an ARElement that is relevant in the context of the enclosing Software Cluster.
Multiplicity	1..*
Type	Foreign reference to AR-ELEMENT

]

[APMC_UCM_00017] Definition of ApmcParamConfContainerDef SoftwareClusterFunctionGroupRefCollection [

Container Name	SoftwareClusterFunctionGroupRefCollection
Parent Container	SoftwareCluster
Description	This container contains the collection of function groups used by the enclosing Software Cluster.
Multiplicity	0..1
Configuration Parameters	

Included Parameters		
Parameter Name	Multiplicity	Apmc ID
ClaimedFunctionGroupForeignRef	1..*	[APMC_UCM_00018]

No Included Containers

]

[APMC_UCM_00018] Definition of ApmcForeignReferenceDef ClaimedFunctionGroupForeignRef

Parameter Name	ClaimedFunctionGroupForeignRef
Parent Container	SoftwareClusterFunctionGroupRefCollection
Description	This function group is used in the context of the enclosing Software Cluster.
Multiplicity	1..*
Type	Foreign reference to MODE-DECLARATION-GROUP-PROTOTYPE

]

[APMC_UCM_00043] Definition of ApmcParamConfContainerDef ArtifactChecksumCollection

Container Name	ArtifactChecksumCollection
Parent Container	SoftwareCluster
Description	This container represents the collection of Artifact Checksums in the context pf the enclosing Software Clusters
Multiplicity	0..1
Configuration Parameters	

No Included Parameters

Included Containers		
Container Name	Multiplicity	Dependency
ArtifactChecksum	0..*	This container represents the configuration of one Artifact Checksum.

]

[APMC_UCM_00036] Definition of ApmcParamConfContainerDef ArtifactLocatorCollection

Container Name	ArtifactLocatorCollection
Parent Container	SoftwareCluster
Description	This container represents the collection of Artifact Locators in the context pf the enclosing Software Clusters
Multiplicity	0..1
Configuration Parameters	

No Included Parameters

Included Containers		
Container Name	Multiplicity	Dependency
ArtifactLocator	0..*	This container represents the configuration of one Artifact Locator.

]

[APMC_UCM_00015] Definition of ApmcParamConfContainerDef DesignRefCollection

Container Name	DesignRefCollection
Parent Container	SoftwareCluster
Description	This container collects all references to SoftwareClusterDesigns that are relevant in the context of the enclosing software cluster.
Multiplicity	0..1
Configuration Parameters	

Included Parameters		
Parameter Name	Multiplicity	Apmc ID
DesignRef	1..*	[APMC_UCM_00016]

No Included Containers

]

[APMC_UCM_00016] Definition of ApmcUriForeignReferenceDef DesignRef [

Parameter Name	DesignRef
Parent Container	DesignRefCollection
Description	This reference identifies the applicable Software Cluster Design.
Multiplicity	1..*
Type	Foreign reference to SOFTWARE-CLUSTER-DESIGN

]

10.1.4 Relation to Diagnostics

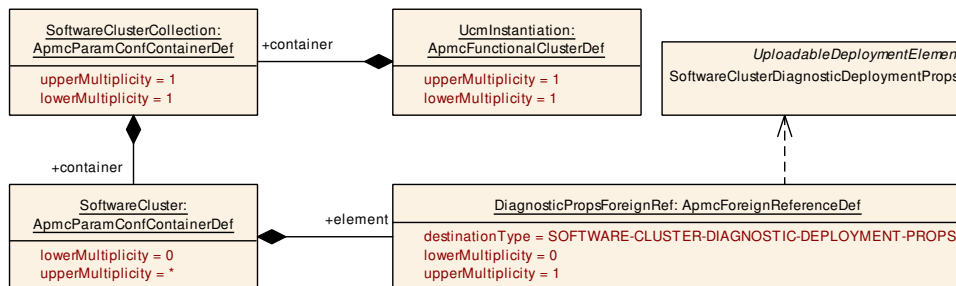


Figure 10.5: Diagnostic-related Configuration

10.1.5 ArtifactLocator

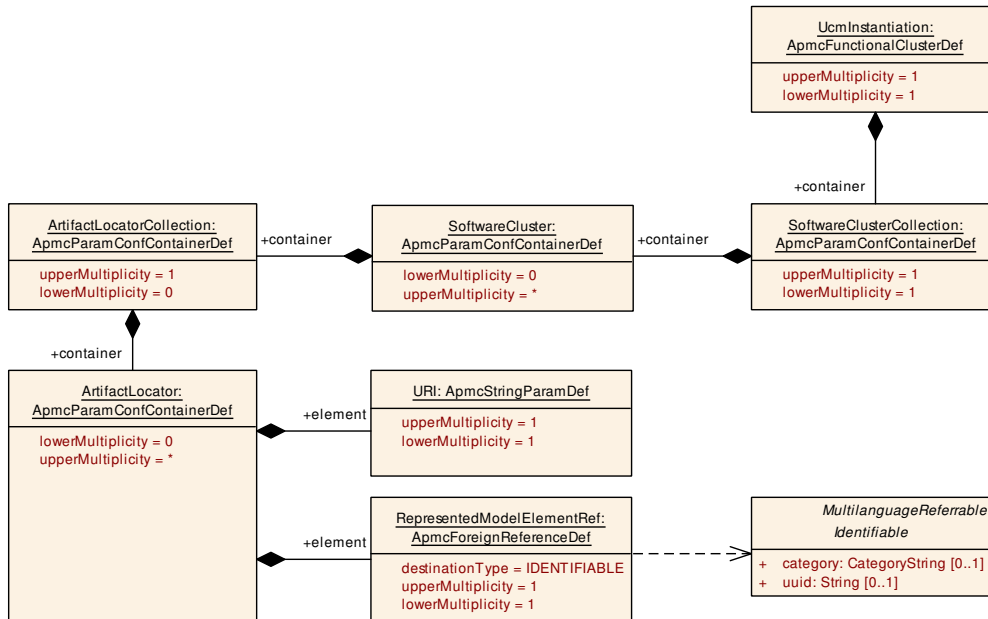


Figure 10.6: Artifact Locator Configuration

[APMC_UCM_00037] Definition of ApmcParamConfContainerDef ArtifactLocator

Container Name	ArtifactLocator		
Parent Container	ArtifactLocatorCollection		
Description	This container represents the configuration of one Artifact Locator.		
Multiplicity	0..*		
Configuration Parameters			
Included Parameters			
Parameter Name	Multiplicity	Apmc ID	
URI	1	[APMC_UCM_00038]	
RepresentedModelElementRef	1	[APMC_UCM_00039]	
No Included Containers			

For parameter table [APMC_UCM_00038] [URI](#), see definition below container [Artifact Checksum](#).

[APMC_UCM_00039] Definition of ApmcForeignReferenceDef RepresentedModelElementRef

Parameter Name	RepresentedModelElementRef
Parent Container	ArtifactLocator
Description	This parameter identifies the model element that corresponds to the artifact.
Multiplicity	1
Type	Foreign reference to IDENTIFIABLE

10.1.6 ArtifactChecksum

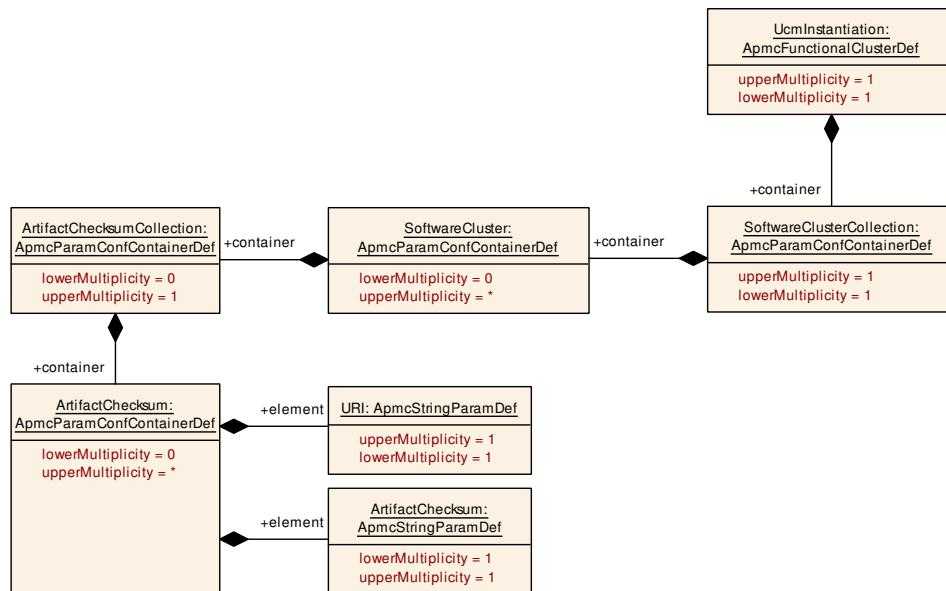


Figure 10.7: Artifact Checksum Configuration

[APMC_UCM_00044] Definition of ApmcParamConfContainerDef ArtifactChecksum

Container Name	ArtifactChecksum
Parent Container	ArtifactChecksumCollection
Description	This container represents the configuration of one Artifact Checksum.
Multiplicity	0..*
Configuration Parameters	

Included Parameters		
Parameter Name	Multiplicity	Apmc ID
ArtifactChecksum	1	[APMC_UCM_00042]
URI	1	[APMC_UCM_00038]

No Included Containers

]

[APMC_UCM_00042] Definition of ApmcStringParamDef ArtifactChecksum [

Parameter Name	ArtifactChecksum
Parent Container	ArtifactChecksum
Description	This parameter carries the serialized checksum of the corresponding artifact.
Multiplicity	1
Type	ApmcStringParamDef
Default value	–
Verbatim String	–

]

[APMC_UCM_00038] Definition of ApmcStringParamDef URI [

Parameter Name	URI
Parent Container	ArtifactChecksum , ArtifactLocator
Description	This parameter identifies the location of the artifact
Multiplicity	1
Type	ApmcStringParamDef
Default value	–
Verbatim String	–

]

10.1.7 Correspondence Table

The following table lists the corresponding model elements for the APMC-based Target Configuration and for the legacy M2-based Target Configuration. In [Chapter 7](#), only the APMC-based configuration elements are used.

Readers interested to read [Chapter 7](#) may use the table to understand how the M2-based legacy approach corresponds to the APMC-based configuration elements.

M1 Model Element	M2 Model Element
UcmInstantiation	UcmModuleInstantiation
SoftwareCluster	SoftwareCluster
SoftwareCluster.DiagnosticPropsForeignRef	SoftwareCluster.diagnosticDeploymentProps
SoftwareCluster.LicenseRef	SoftwareCluster.license
SoftwareCluster.ReleaseNotesRef	SoftwareCluster.releaseNotes
SoftwareCluster.SoftwareClusterCategory	Identifiable.category
SoftwareCluster.SoftwareClusterVersion	SoftwareCluster.version
SoftwareCluster.TypeApproval	SoftwareCluster.typeApproval
SoftwareCluster.VendorId	SoftwareCluster.vendorId
SoftwareCluster.VendorSignatureRef	SoftwareCluster.vendorSignature
ARElementRefCollection.ARElementRef	SoftwareCluster.containedARElement
ArtifactChecksum	ArtifactChecksum
ArtifactChecksum.ArtifactChecksum	ArtifactChecksum.checksumValue





M1 Model Element	M2 Model Element
ArtifactChecksum.URI	ArtifactChecksum.uri
ArtifactLocator	ArtifactLocator
ArtifactLocator.RepresentedModelElementRef	ArtifactLocator.representedModelElement
ArtifactLocator.URI	ArtifactChecksum.uri
DesignRefCollection.DesignRef	SoftwareCluster.design
FunctionalClusterRefContainer	SoftwareCluster.moduleInstantiation
ProcessRefCollection	SoftwareCluster.containedProcess
SoftwareClusterFunctionGroupRefCollection.ClaimedFunctionGroupForeignRef	SoftwareCluster.claimedFunctionGroup
UcmInstantiationGeneral.MaxBlockSize	UcmModuleInstantiation.maxBlockSize
UcmInstantiationGeneral.UcmIdentifier	UcmModuleInstantiation.identifier
UcmInstantiationGeneral.Version	UcmModuleInstantiation.version
UcmSubordinateInstantiation	UcmSubordinateModuleInstantiation
UcmSubordinateInstantiation.UcmSubordinateMaxAvailablePersistencyStorageSpace	UcmSubordinateModuleInstantiation.maxAvailablePersistencyStorageSpace
UcmSubordinatePrepareRollback	UcmSubordinateModuleInstantiation.prepareRollback
UcmSubordinatePrepareUpdate	UcmSubordinateModuleInstantiation.prepareUpdate
UcmSubordinateVerifyUpdate	UcmSubordinateModuleInstantiation.verifyUpdate

Table 10.1: Correspondence between APMC (M1) elements and MetaModel (M2) elements for FunctionalCluster UcmInstantiation

10.2 Default Values

This section defines the default values for attributes specified in [7] that are part of the configuration model of this functional cluster.

Qualified attribute name	Default Value
UcmModuleInstantiation.maxBlockSize	65536
UcmSubordinateModuleInstantiation.prepareRollback.maximumNumberOfRetries	3
UcmSubordinateModuleInstantiation.prepareRollback.retryIntervalTime	10
UcmSubordinateModuleInstantiation.prepareUpdate.maximumNumberOfRetries	3
UcmSubordinateModuleInstantiation.prepareUpdate.retryIntervalTime	10
UcmSubordinateModuleInstantiation.verifyUpdate.maximumNumberOfRetries	3
UcmSubordinateModuleInstantiation.verifyUpdate.retryIntervalTime	10

10.3 Semantic Constraints

This functional cluster does not define any semantic constraints for elements specified in [\[7\]](#).

11 Sequence diagrams

The following sequence charts are simplified examples and have no normative meaning. The relevant definitions are in chapter 7 only.

11.1 Update process

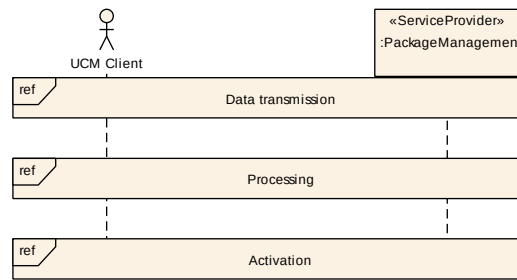


Figure 11.1: Sequence diagram showing the update process

11.2 Data transmission

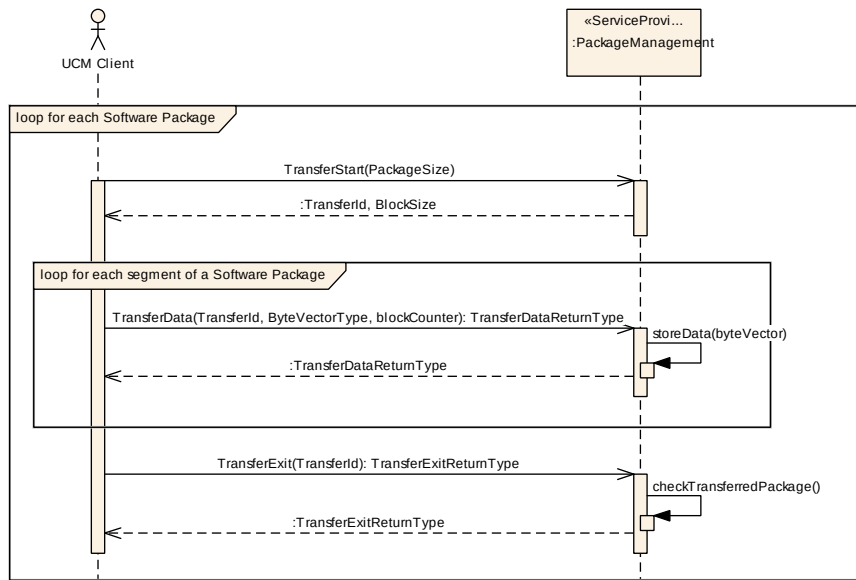


Figure 11.2: Sequence diagram showing the data transmission

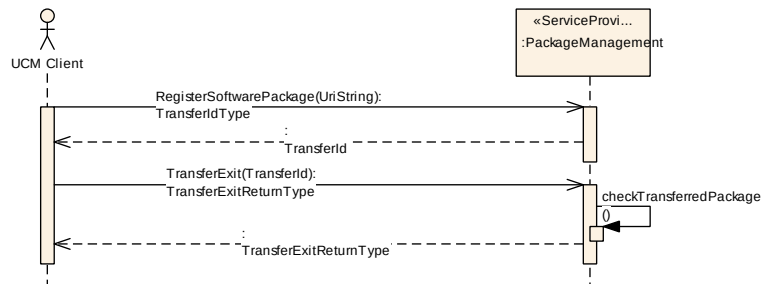


Figure 11.3: Sequence diagram showing Software Package registration

11.3 Package processing

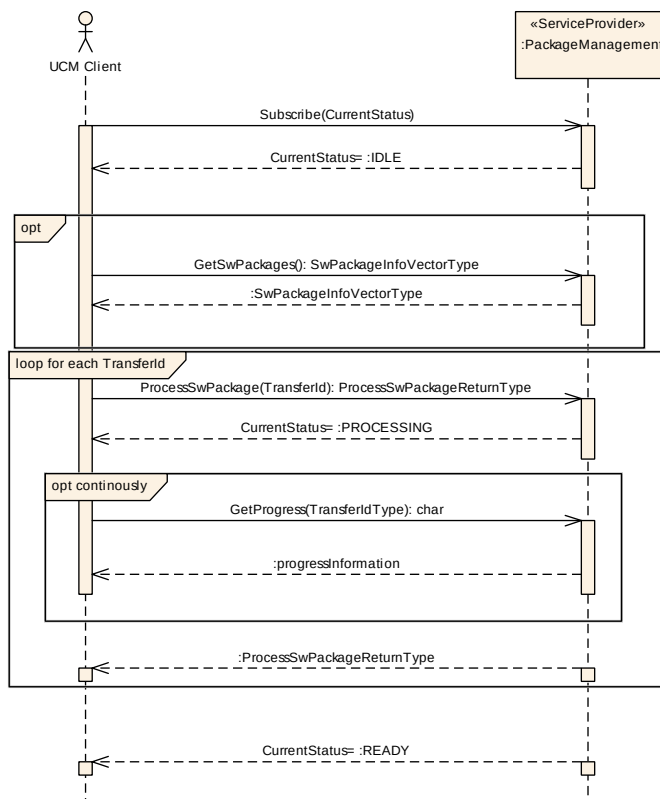
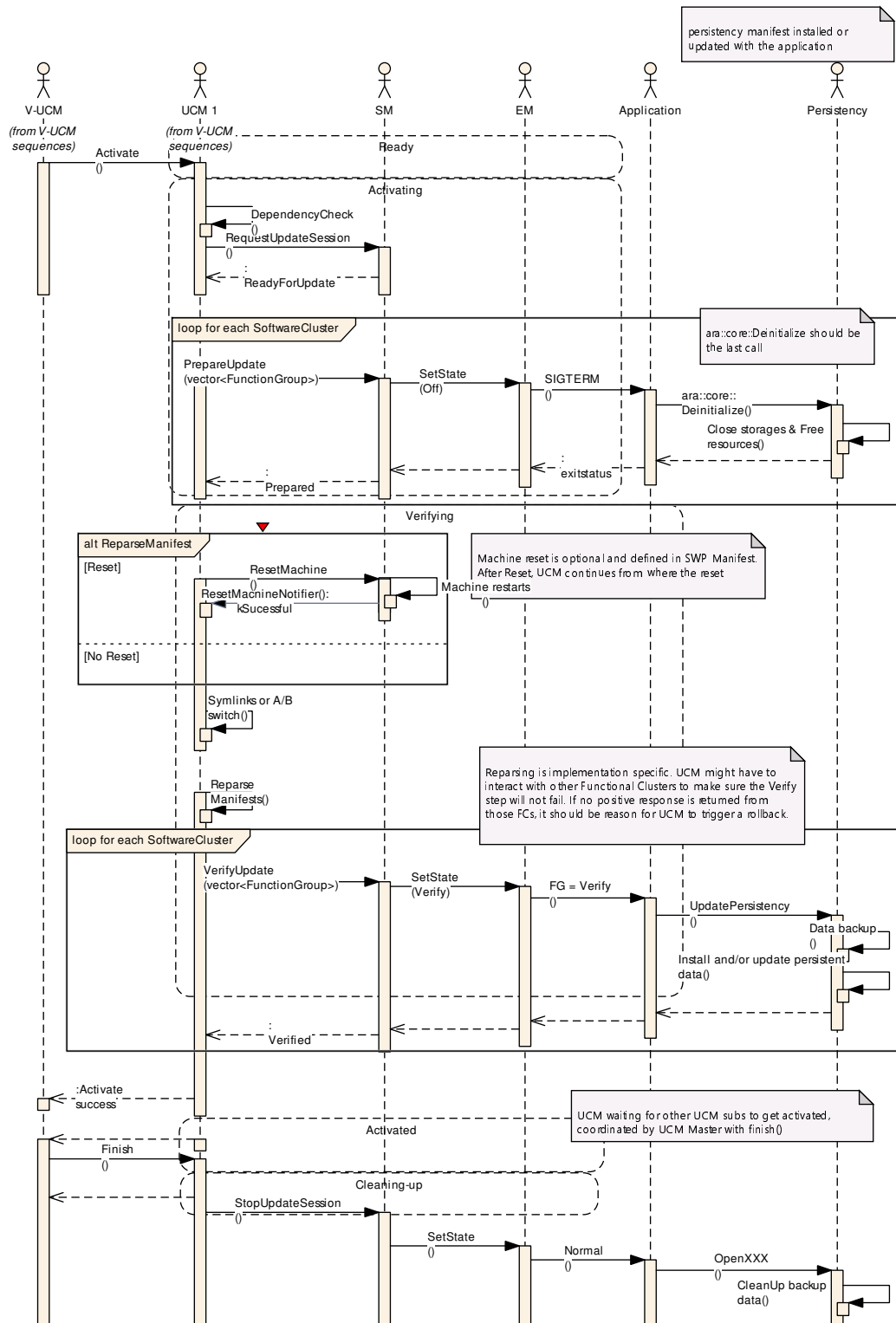


Figure 11.4: Sequence diagram showing the package processing

11.4 Activation



11.5 Failing activation

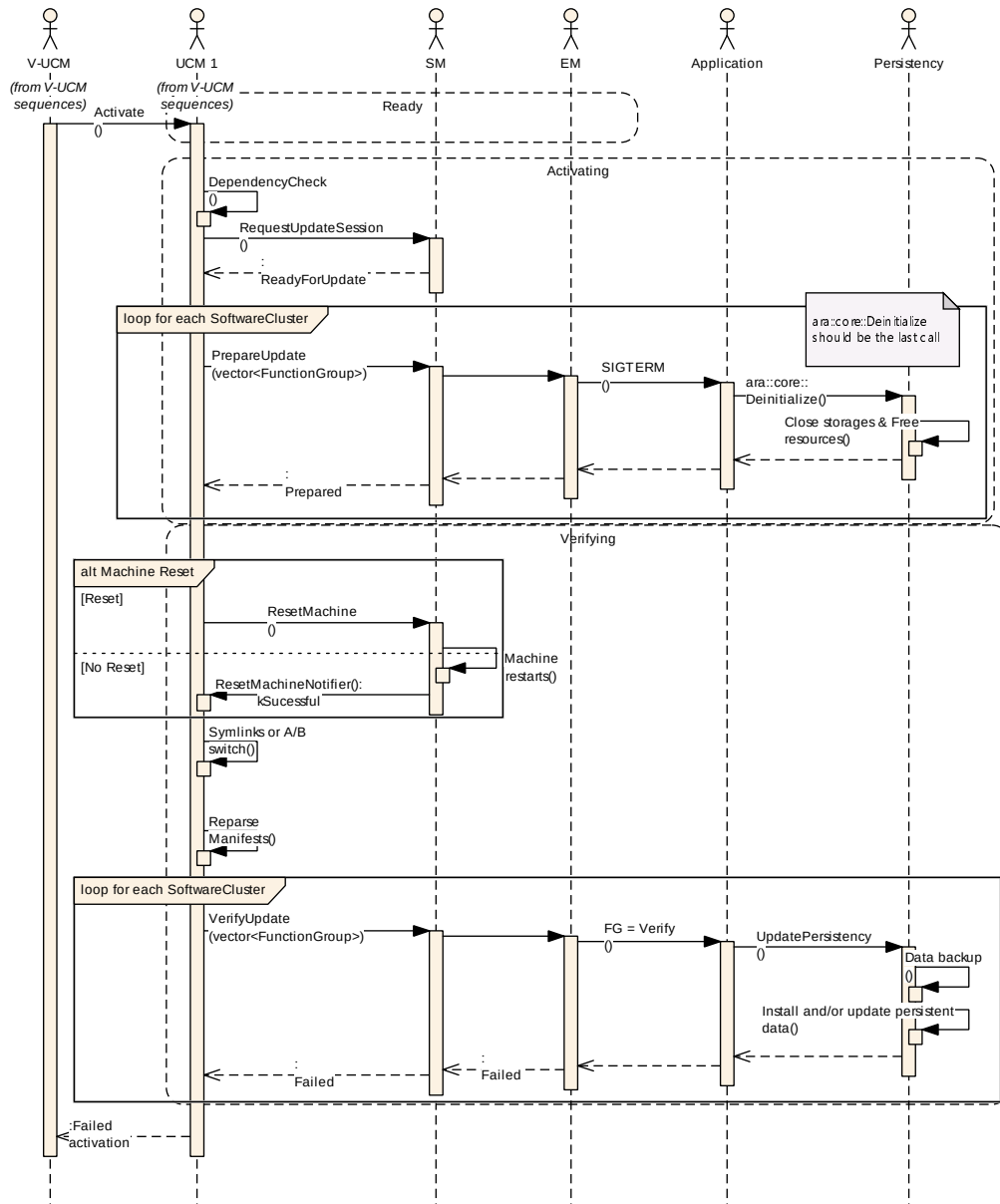


Figure 11.6: Sequence diagram showing an activation failing

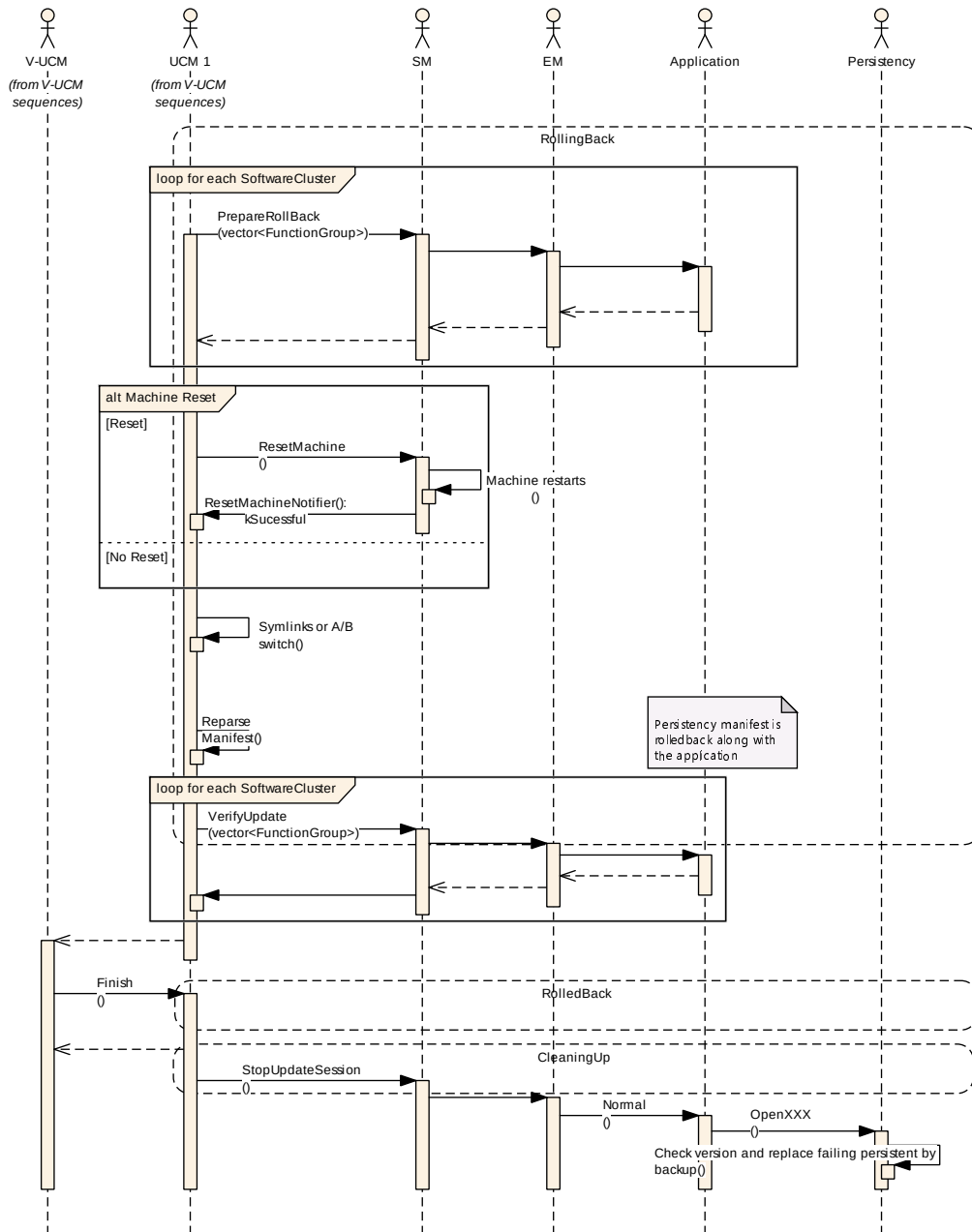


Figure 11.7: Sequence diagram showing an activation failing

11.6 Failing rollback

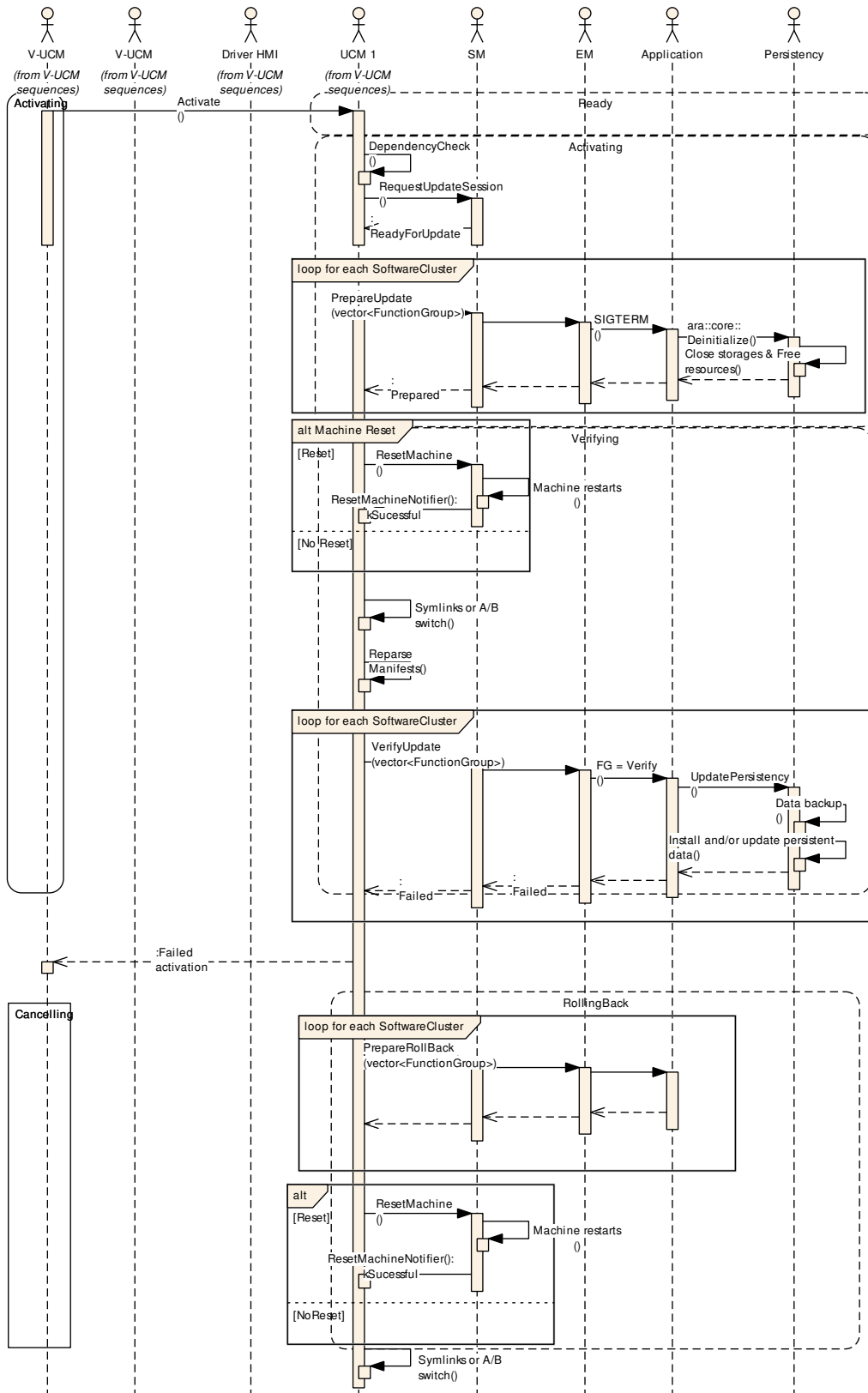


Figure 11.8: Sequence diagram showing a rollback failing

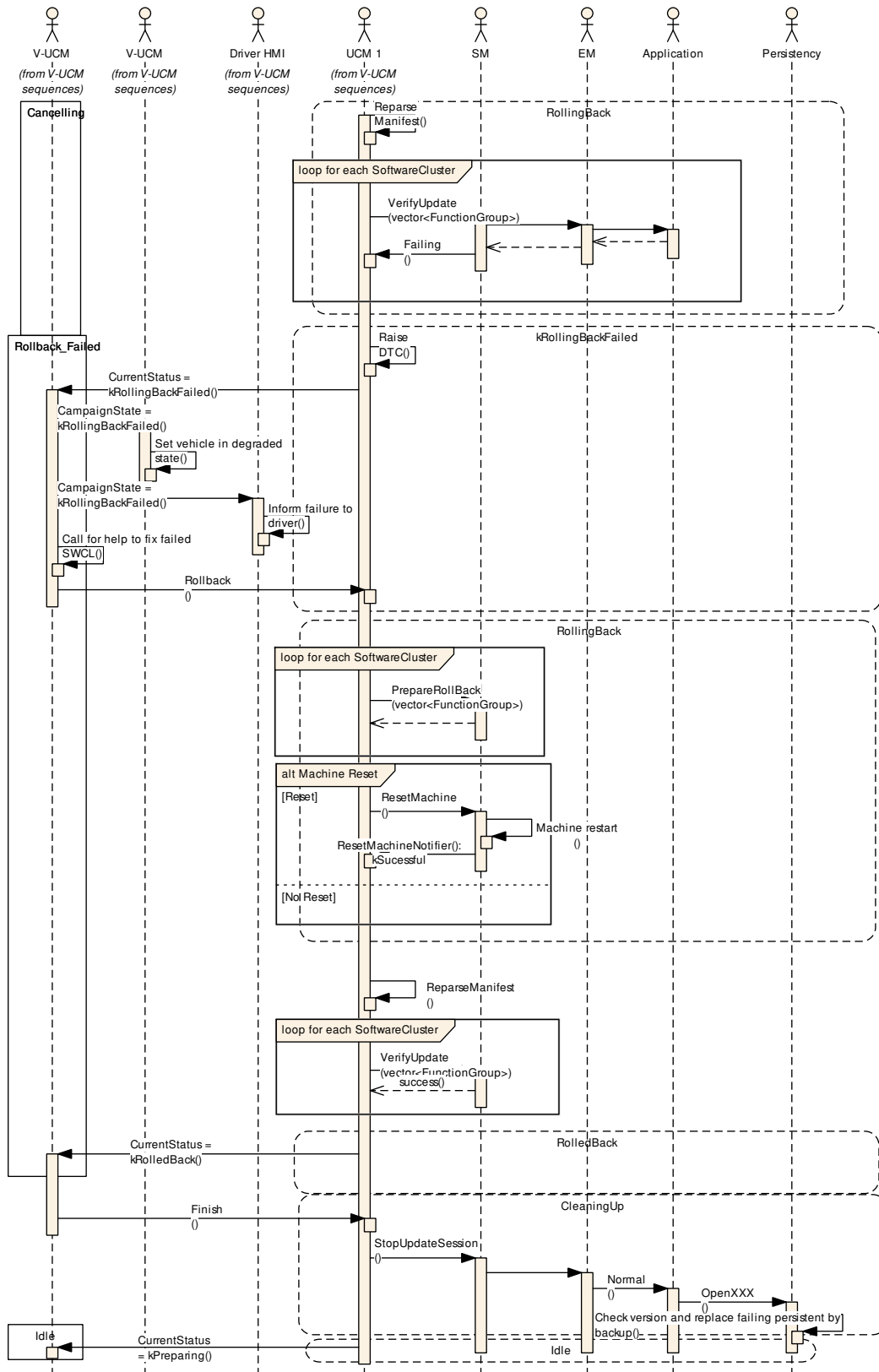


Figure 11.9: Sequence diagram showing a rollback failing

11.7 V-UCM simplified vehicle update

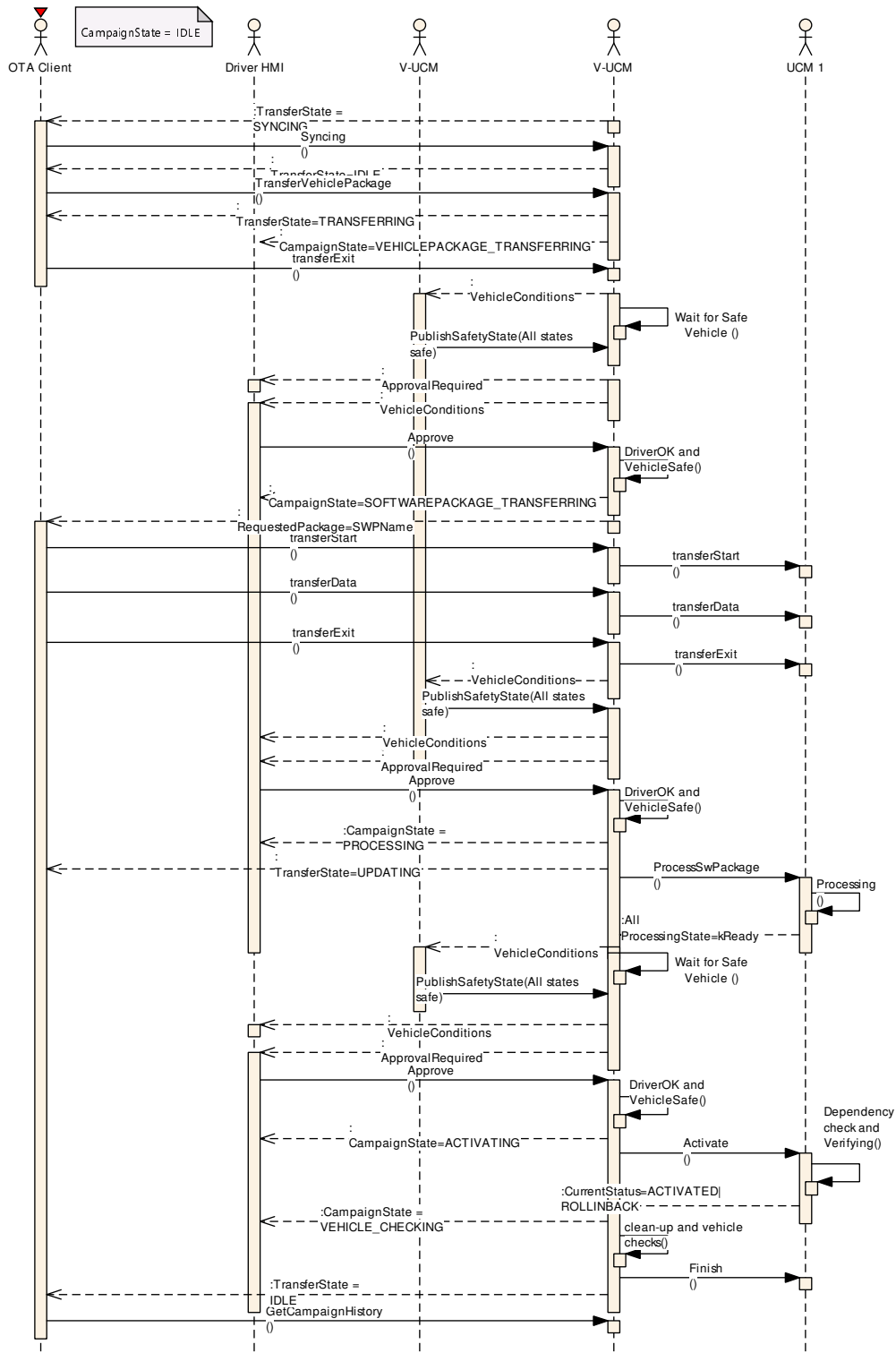


Figure 11.10: Sequence diagram showing vehicle update

A Mentioned Manifest Elements

For the sake of completeness, this chapter contains a set of class tables representing meta-classes mentioned in the context of this document but which are not contained directly in the scope of describing specific meta-model semantics.

Chapter is generated.

Class	CryptoServiceCertificate			
Note	This meta-class represents the ability to model a cryptographic certificate. Tags: atp.recommendedPackage=CryptoServiceCertificates			
Base	ARElement, ARObject, CollectableElement, Identifiable, MultilanguageReferrable, PackageableElement, Referrable, UploadableDesignElement, UploadablePackageElement			
Aggregated by	ARPackage.element			
Attribute	Type	Mult.	Kind	Note
algorithmFamily	CryptoCertificateAlgorithmFamilyEnum	0..1	attr	This attribute represents a description of the family of crypto algorithm used to generate public key and signature of the cryptographic certificate.
format	CryptoCertificateFormatEnum	0..1	attr	This attribute can be used to provide information about the format used to create the certificate
maximumLength	PositiveInteger	0..1	attr	This attribute represents the ability to define the maximum length of the certificate in bytes.
nextHigherCertificate	CryptoServiceCertificate	0..1	ref	The reference identifies the next higher certificate in the certificate chain.
serverNameIdentification	String	0..1	attr	Server Name Indication (SNI) is needed if the IP address hosts multiple servers (on the same port), each of them using a different certificate. If the client sends the SNI to the Server in the client hello, the server looks the SNI up in its certificate list and uses the certificate identified by the SNI.

Table A.1: CryptoServiceCertificate

Class	PersistencyDeployment (abstract)			
Note	This abstract meta-class serves as a base class for concrete classes representing different aspects of persistency. This Class is only used by the AUTOSAR Adaptive Platform.			
Base	ARElement, ARObject, CollectableElement, Identifiable, MultilanguageReferrable, PackageableElement, Referrable, UploadableDeploymentElement, UploadableExclusivePackageElement, UploadablePackageElement			
Subclasses	PersistencyFileStorage, PersistencyKeyValueStorage			
Aggregated by	ARPackage.element			
Attribute	Type	Mult.	Kind	Note
deploymentUri (ordered)	PersistencyDeploymentUri	*	aggr	This aggregation represents the collection of URIs relevant for the enclosing PersistencyDeployment.
maximumAllowedSize	PositiveUnlimitedInteger	0..1	attr	The value of this attribute represents the maximum size (unit: bytes) allowed at target-configuration time for the enclosing PersistencyDeployment.
minimumSustainedSize	PositiveInteger	0..1	attr	The value of this attribute represents the minimum size (unit: bytes) guaranteed at deployment time for the enclosing PersistencyDeployment.





Class	PersistencyDeployment (abstract)			
redundancyHandling	PersistencyRedundancyHandling	*	aggr	This aggregation represents the chosen approaches to handle redundancy. Stereotypes: atpSplittable Tags: atp.Splitkey=redundancyHandling
updateStrategy	PersistencyCollectionLevelUpdateStrategyEnum	0..1	attr	This attribute shall be used to specify the update strategy of the respective PersistencyDeployment as a whole.
version	StrongRevisionLabelString	0..1	attr	The attribute represents the version of the <code>PersistencyFileStorage</code> or <code>PersistencyKeyValueStorage</code> .

Table A.2: PersistencyDeployment

Class	PersistencyDeploymentUri			
Note	This meta-class represents the ability to contain URIs relevant for the persistency deployment. This Class is only used by the AUTOSAR Adaptive Platform.			
Base	<code>ARObject</code>			
Aggregated by	PersistencyDeployment.deploymentUri			
Attribute	Type	Mult.	Kind	Note
uri	UriString	0..1	attr	This attribute holds the storage location for the concrete subclass of PersistencyDeployment, e.g. file on the file system.

Table A.3: PersistencyDeploymentUri

Class	ProcessToMachineMapping			
Note	This meta-class has the ability to associate a Process with a Machine. This relation involves the definition of further properties, e.g. timeouts. This Class is only used by the AUTOSAR Adaptive Platform.			
Base	<code>ARObject</code> , <code>Identifiable</code> , <code>MultilanguageReferrable</code> , Referrable			
Aggregated by	<code>ProcessToMachineMappingSet.processToMachineMapping</code>			
Attribute	Type	Mult.	Kind	Note
design	<code>ProcessDesignToMachineDesignMapping</code>	0..1	ref	This reference represents the identification of the design-time representation for the ProcessToMachineMapping that owns the reference.
machine	<code>Machine</code>	0..1	ref	This reference identifies the Machine in the context of the ProcessToMachineMapping.
nonOsModuleInstantiation	<code>NonOsModuleInstantiation</code>	0..1	ref	This supports the optional case that the process represents a platform module.
persistencyCentralStorageURI	UriString	0..1	attr	This attribute identifies a central place for the mapped Process to store the list of available storages and version information.
process	<code>Process</code>	0..1	ref	This reference identifies the Process in the context of the ProcessToMachineMapping.
shallNotRunOn	<code>ProcessorCore</code>	*	ref	This reference indicates a collection of cores onto which the mapped process shall not be executing.
shallRunOn	<code>ProcessorCore</code>	*	ref	This reference indicates a collection of cores onto which the mapped process shall be executing.

Table A.4: ProcessToMachineMapping

Class	Referrable (abstract)			
Note	Instances of this class can be referred to by their identifier (while adhering to namespace borders).			
Base	<i>ARObject</i>			
Subclasses	<i>AtpDefinition</i> , <i>BswDistinguishedPartition</i> , <i>BswModuleCallPoint</i> , <i>BswModuleClientServerEntry</i> , <i>BswVariableAccess</i> , <i>CouplingPortTrafficClassAssignment</i> , <i>CppImplementationDataTypeContextTarget</i> , <i>DiagnosticEnvModeElement</i> , <i>EthernetPriorityRegeneration</i> , <i>ExclusiveAreaNestingOrder</i> , <i>HwDescriptionEntity</i> , <i>ImplementationProps</i> , <i>ModeTransition</i> , <i>MultilanguageReferrable</i> , <i>NmNetworkHandle</i> , <i>PncMappingIdent</i> , <i>SingleLanguageReferrable</i> , <i>SoConIPdulIdentifier</i> , <i>SomeipRequiredEventGroup</i> , <i>TpConnectionIdent</i>			
Attribute	Type	Mult.	Kind	Note
shortName	Identifier	1	attr	This specifies an identifying shortName for the object. It needs to be unique within its context and is intended for humans but even more for technical reference. Stereotypes: atpIdentityContributor Tags: xml.enforceMinMultiplicity=true xml.sequenceOffset=-100
shortName Fragment	ShortNameFragment	*	aggr	This specifies how the Referrable.shortName is composed of several shortNameFragments. Tags: xml.sequenceOffset=-90

Table A.5: Referrable

Class	SoftwareCluster			
Note	This meta-class represents the ability to define an uploadable software-package, i.e. the SoftwareCluster shall contain all software and configuration for a given purpose. Tags: atp.recommendedPackage=SoftwareClusters This Class is only used by the AUTOSAR Adaptive Platform.			
Base	<i>ARElement</i> , <i>ARObject</i> , <i>CollectableElement</i> , <i>Identifiable</i> , <i>MultilanguageReferrable</i> , <i>PackageableElement</i> , <i>Referrable</i> , <i>UploadableDeploymentElement</i> , <i>UploadablePackageElement</i>			
Aggregated by	ARPackage.element			
Attribute	Type	Mult.	Kind	Note
artifactChecksum	ArtifactChecksum	*	aggr	This aggregation carries the checksums for artifacts contained in the enclosing SoftwareCluster. Please note that the value of these checksums is only applicable at the time of configuration. Stereotypes: atpSplitable Tags: atp.Splitkey=artifactChecksum.shortName
artifactLocator	ArtifactLocator	*	aggr	This aggregation represents the artifact locations that are relevant in the context of the enclosing SoftwareCluster
claimed FunctionGroup	ModeDeclarationGroup Prototype	*	ref	Each SoftwareCluster can reserve the usage of a given functionGroup such that no other SoftwareCluster is allowed to use it
contained ARElement	ARElement	*	ref	This reference represents the collection of model elements that cannot derive from UploadablePackageElement and that contribute to the completeness of the definition of the SoftwareCluster. Stereotypes: atpSplitable Tags: atp.Splitkey=containedARElement
contained Package Element	UploadablePackageElement	*	ref	This reference identifies model elements that are required to complete the manifest content. Stereotypes: atpSplitable Tags: atp.Splitkey=containedPackageElement
contained Process	Process	*	ref	This reference represent the processes contained in the enclosing SoftwareCluster.





Class	SoftwareCluster			
design	SoftwareClusterDesign	*	ref	This reference represents the identification of all SoftwareClusterDesigns applicable for the enclosing SoftwareCluster. Stereotypes: atpUriDef
diagnostic Deployment Props	SoftwareCluster DiagnosticDeployment Props	0..1	ref	This reference identifies the applicable SoftwareClusterDiagnosticDeploymentProps that are applicable for the referencing SoftwareCluster.
license	Documentation	*	ref	This attribute allows for the inclusion of the full text of a license of the enclosing SoftwareCluster. In many cases open source licenses require the inclusion of the full license text to any software that is released under the respective license.
module Instantiation	AdaptiveModule Instantiation	*	ref	This reference identifies AdaptiveModuleInstantiations that need to be included with the SoftwareCluster in order to establish infrastructure required for the installation of the SoftwareCluster. Stereotypes: atpSplitable Tags: atp.Splitkey=moduleInstantiation
releaseNotes	Documentation	0..1	ref	This attribute allows for the explanations of changes since the previous version. The list of changes might require the creation of multiple paragraphs of test.
typeApproval	String	0..1	attr	This attribute carries the homologation information that may be specific for a given country.
vendorId	PositiveInteger	0..1	attr	Vendor ID of this Implementation according to the AUTOSAR vendor list.
vendor Signature	CryptoService Certificate	0..1	ref	This reference identifies the certificate that represents the vendor's signature.
version	StrongRevisionLabel String	0..1	attr	This attribute can be used to describe a version information for the enclosing SoftwareCluster.

Table A.6: SoftwareCluster

Class	SoftwareClusterDesign			
Note	This meta-class represents the ability for the OEM to design the grouping of software uploadable to a specific target Machine. Tags: atp.recommendedPackage=SoftwareClusterDesigns This Class is only used by the AUTOSAR Adaptive Platform.			
Base	<i>ARElement, ARObject, AtpClassifier, CollectableElement, Identifiable, MultilanguageReferrable, PackageableElement, Referrable</i>			
Aggregated by	ARPackage.element			
Attribute	Type	Mult.	Kind	Note
conflictsTo	SoftwareCluster DependencyFormula	0..1	aggr	This aggregation handles conflicts. If it yields true then the SoftwareClusterDesign shall not be installed. Stereotypes: atpSplitable Tags: atp.Splitkey=conflictsTo
contained Process	ProcessDesign	*	ref	This reference represent the ProcessDesigns contained in the enclosing SoftwareCluster. Stereotypes: atpSplitable Tags: atp.Splitkey=containedProcess
dependsOn	SoftwareCluster DependencyFormula	0..1	aggr	This aggregation can be taken to identify a dependency for the enclosing SoftwareClusterDesign. Stereotypes: atpSplitable Tags: atp.Splitkey=dependsOn





Class	SoftwareClusterDesign			
diagnostic Contribution	DiagnosticContributionSet	*	ref	This reference identifies the corresponding collection of DiagnosticContributionSet. Stereotypes: atpSplitable Tags: atp.Splitkey=diagnosticContribution
dolpLogical Address	DolpLogicalAddress	0..1	ref	This reference identifies the Dolp logical address that shall be used to contact the diagnostic management in the SoftwareCluster derived from the enclosing SoftwareClusterDesign. Stereotypes: atpSplitable Tags: atp.Splitkey=dolpLogicalAddress
installation Behavior	SoftwareClusterInstallationBehaviorEnum	0..1	attr	This attribute controls the behavior of the SoftwareCluster in terms of installation.
intendedTarget Machine	MachineDesign	*	ref	This reference can be taken to identify the Machine Design for which the final SoftwareCluster shall be developed. Stereotypes: atpUriDef
required ARElement	ARElement	*	ref	This reference represents the collection of ARElements that are required for the completeness of the definition of the SoftwareCluster. Stereotypes: atpSplitable Tags: atp.Splitkey=requiredARElement
requiredDesign Element	UploadableDesignElement	*	ref	This reference points to uploadable design elements that have been identified as relevant in the context of the enclosing SoftwareClusterDesign. Stereotypes: atpSplitable Tags: atp.Splitkey=requiredDesignElement
root Composition	RootSwClusterDesignComponentPrototype	0..1	aggr	This aggregation represents the design of the software inside the SwClusterDesign terms of the communication endpoints.
version	StrongRevisionLabelString	0..1	attr	This attribute can be used to describe a version information for the enclosing SoftwareClusterDesign.

Table A.7: SoftwareClusterDesign

Enumeration	SoftwareClusterInstallationBehaviorEnum
Note	This enumeration defines possible approaches for the installation behavior of a SoftwareCluster. This Enumeration is only used by the AUTOSAR Adaptive Platform.
Aggregated by	SoftwareClusterDesign.installationBehavior
Literal	Description
canBeRemoved	The enclosing SoftwareCluster can be removed from the target Machine or updated with a newer version. Tags: atp.EnumerationLiteralIndex=0
cannotBeRemoved	The enclosing SoftwareCluster cannot be removed from the target Machine. It can only be updated with a newer version. Tags: atp.EnumerationLiteralIndex=1

Table A.8: SoftwareClusterInstallationBehaviorEnum

Class	SoftwarePackage
Note	This meta-class represents the ability to formalize the content of a software package. Tags: atp.recommendedPackage=SoftwarePackages This Class is only used by the AUTOSAR Adaptive Platform.
Base	<i>ARElement, ARObject, CollectableElement, Identifiable, MultilanguageReferrable, PackageableElement, Referrable, UploadableDeploymentElement, UploadablePackageElement</i>





Class	SoftwarePackage			
Aggregated by	ARPackage.element			
Attribute	Type	Mult.	Kind	Note
actionType	SoftwarePackageActionTypeEnum	0..1	attr	This attribute defines the action to be taken in the step of processing the enclosing SoftwarePackage.
activationAction	SoftwarePackageActivationActionEnum	0..1	attr	This attribute governs the action to be taken after the installation of the SoftwareCluster completed.
artifactLocator	ArtifactLocator	0..1	aggr	This attribute identifies the software package at configuration time, out of the context of an AUTOSAR model.
compressed Software PackageSize	PositiveInteger	0..1	attr	This size represents the size of the compressed Software Package.
deltaPackage Applicable Version	StrongRevisionLabelString	0..1	attr	This attribute identifies the version of the included SoftwareCluster for which the enclosing SoftwarePackage can be used as a delta update
estimated DurationOf Operation	TimeValue	0..1	attr	This attribute provides an estimation about how long the operation of the SoftwarePackage is going to take for its transfer, processing and activation when updated standalone (not within an update campaign)
minimum SupportedUcm Version	RevisionLabelString	0..1	attr	This attribute identifies the minimum supported version of the UCM for this SoftwarePackage.
packagerId	PositiveInteger	0..1	attr	This attribute identifies Id of the organization that provides the packager generating the SoftwarePackage.
packager Signature	CryptoServiceCertificate	0..1	ref	This reference identifies the certificate that represents the packager's signature.
primary DownloadUri	UriDescription	0..1	aggr	This attribute represents the description of the primary download location. Stereotypes: atpSplitable Tags: atp.Splitkey=primaryDownloadUri.shortName
purposeOf Update	Documentation	0..1	ref	The referenced Documentation is supposed to provide a description of the purpose of the update.
secondary DownloadUri	UriDescription	*	aggr	This attribute represents the description of the secondary download locations. Stereotypes: atpSplitable Tags: atp.Splitkey=secondaryDownloadUri.shortName
softwareCluster	SoftwareCluster	0..1	ref	This reference identifies the SoftwareCluster that belongs to the SoftwarePackage. The nature of this relation is actually more like an aggregation than a reference. But the relation is still modelled as a reference because two ARElements cannot aggregate each other.
uncompressed SoftwareCluster Size	PositiveInteger	0..1	attr	This attribute gives an indication about the storage that has to be available on the target.

Table A.9: SoftwarePackage

Primitive	StrongRevisionLabelString
Note	This primitive represents a revision label which identifies an object under version control. It represents a pattern which requires three integer numbers separated by a dot, representing from left to right Major Version, MinorVersion, PatchVersion and additional labels for pre-release version and build metadata. Legal patterns are for example: 1.0.0-alpha+001 1.0.0+20130313144700 1.0.0-beta+exp.sha.5114f85 Tags: xml.xsd.customType=STRONG-REVISION-LABEL-STRING xml.xsd.pattern=(0 [1-9]\d*)\.(0 [1-9]\d*)\.(0 [1-9]\d*)(-((0 [1-9]\d*)\d*[a-zA-Z-][0-9a-zA-Z-]*)\.(0 [1-9]\d*)\d*[a-zA-Z-][0-9a-zA-Z-]*)*)?(\+([0-9a-zA-Z-]+\.[0-9a-zA-Z-]+)*)? xml.xsd.type=string

Table A.10: StrongRevisionLabelString

Class	UcmModuleInstantiation (abstract)			
Note	This meta-class represents the ability to define the target-configuration of a UCM instantiation. This Class is only used by the AUTOSAR Adaptive Platform.			
Base	<i>ARObject</i> , <i>AdaptiveModuleInstantiation</i> , <i>AtpClassifier</i> , <i>AtpFeature</i> , <i>AtpStructureElement</i> , <i>Identifiable</i> , <i>MultilanguageReferrable</i> , <i>NonOsModuleInstantiation</i> , Referrable			
Subclasses	UcmMasterModuleInstantiation, UcmSubordinateModuleInstantiation			
Aggregated by	<i>AtpClassifier.atpFeature</i> , <i>Machine.moduleInstantiation</i>			
Attribute	Type	Mult.	Kind	Note
identifier	String	0..1	attr	This represents the identification of a UCM.
maxBlockSize	PositiveInteger	0..1	attr	This attribute denotes the maximum block size (unit: bytes) used in the UCM implementation.
version	StrongRevisionLabel String	0..1	attr	This attribute defines the software version of the UCM on this platform. Note that the definition of the version is required if the ability of the SoftwarePackage to require a minimum version of the UCM is utilized.

Table A.11: UcmModuleInstantiation

Class	UcmSubordinateModuleInstantiation			
Note	This meta-class represents the ability to define the target-configuration of a UCM Subordinate instantiation. This Class is only used by the AUTOSAR Adaptive Platform.			
Base	<i>ARObject</i> , <i>AdaptiveModuleInstantiation</i> , <i>AtpClassifier</i> , <i>AtpFeature</i> , <i>AtpStructureElement</i> , <i>Identifiable</i> , <i>MultilanguageReferrable</i> , <i>NonOsModuleInstantiation</i> , Referrable , UcmModuleInstantiation			
Aggregated by	<i>AtpClassifier.atpFeature</i> , <i>Machine.moduleInstantiation</i>			
Attribute	Type	Mult.	Kind	Note
maxAvailable Persistency StorageSpace	PositiveInteger	0..1	attr	This attribute names the maximum amount of space available for persistent data handled by the Persistency of installed packages. The UCM needs to figure out from traversing the minimum storage requirement from existing PersistencyDeployments whether specific packages can be installed from the perspective of available storage space. Note that the minimum storage requirement of PersistencyDeployment needs to include space for the handling of the storage, which shall be calculated by the tooling that creates the manifest information inside the package.
prepareRollback	UcmRetryStrategy	0..1	aggr	This attribute identifies the configuration of prepare rollback retries initiated by the Ucm Subordinate.
prepareUpdate	UcmRetryStrategy	0..1	aggr	This attribute identifies the configuration of prepare update retries initiated by the Ucm Subordinate.
verifyUpdate	UcmRetryStrategy	0..1	aggr	This attribute identifies the configuration of verify update retries initiated by the Ucm Subordinate.

Table A.12: UcmSubordinateModuleInstantiation

Class	UcmToTimeBaseResourceMapping			
Note	This meta-class maps the UCM Module Instantiation to the TimeSync Module Instantiation. Tags: atp.recommendedPackage=FCInteractions This Class is only used by the AUTOSAR Adaptive Platform.			
Base	<i>ARElement</i> , <i>ARObject</i> , <i>CollectableElement</i> , <i>FunctionalClusterInteractsWithFunctionalClusterMapping</i> , <i>Identifiable</i> , <i>MultilanguageReferrable</i> , <i>PackageableElement</i> , Referrable , <i>UploadableDeployment Element</i> , <i>UploadablePackageElement</i>			





<i>Class</i>	UcmToTimeBaseResourceMapping			
Aggregated by	ARPackage.element			
<i>Attribute</i>	<i>Type</i>	<i>Mult.</i>	<i>Kind</i>	<i>Note</i>
timeBaseResource	TimeBaseResource	0..1	ref	This reference identifies the relevant TimeBaseResource.
ucm	UcmModuleInstantiation	0..1	ref	This reference identifies the relevant UcmModule Instantiation.

Table A.13: UcmToTimeBaseResourceMapping

B Mentioned APMC Elements from other Functional Clusters

For the sake of completeness, this chapter contains a set of APMC configuration element tables mentioned in the context of this document but which are not contained directly in the scope of describing specific target configuration semantics.

Chapter is generated.

C Demands and constraints on Base Software

[SWS_UCM_CONSTR_00002] UCM confidential information handling

Upstream requirements: [RS_UCM_00002](#), [RS_UCM_00010](#), [RS_UCM_00011](#)

[The [PackageManagement](#) interface shall only be mapped via `ara::com` to a secure endpoint using secure communication channel providing confidentiality protection.]

The [GetSwClusterInfo](#), [GetSwClusterChangeInfo](#), [GetHistory](#) and [GetSw-Packages](#) methods are using data that could identify vehicle user and therefore should be protected for confidentiality.

D Interfaces to other Functional Clusters (informative)

D.1 Overview

AUTOSAR decided not to standardize interfaces which are exclusively used between Functional Clusters (on platform-level only), to allow efficient implementations, which might depend e.g. on the used Operating System.

This chapter provides informative guidelines how the interaction between Functional Clusters looks like, by clustering the relevant requirements of this document. In addition, the standardized public interfaces which are accessible by user space applications (see chapter 8) can also be used for interaction between Functional Clusters.

The goal is to provide a clear understanding of Functional Cluster boundaries and interaction, without specifying syntactical details. This ensures compatibility between documents specifying different Functional Clusters and supports parallel implementation of different Functional Clusters. Details of the interfaces are up to the platform provider.

D.2 Interfaces Tables

D.2.1 UCM update notification

UCM shall provide the notification to other Functional Clusters that changes have been done to the software. This enables other functional clusters to check if updated manifests have changes relevant for the concerned Functional Cluster. This can be done through the field `CurrentStatus` provided by the UCM service.

E Security Analysis of Installation and Update

This chapter presents a summary for the security analysis of the UCM. Some of the threats could not be addressed by specifying AUTOSAR requirements. The main reason for not specifying the countermeasures is to allow vendors to flexibly decide on the solution that fits their setup. Here we aim to raise awareness and provide advice on the selected topics:

E.1 Securing Software Package

UCM is responsible for applying changes of the platform and applications contained in the Software Packages it receives. Therefore, integrity and authenticity of Software Packages are critical to protect system integrity. It shall be ensured that the Software Packages are neither illegitimately altered nor issued by unauthorized parties. This can be achieved by applying cryptographic techniques such as digital signatures. The period that Software Package resides in UCM before being activated shall not be neglected. It provides a window of opportunity for an attacker to tamper with the Software Package after the authentication is done at TransferExit.

Information disclosure is another security threat category that might be applicable to Software Packages. Packages that contain sensitive information, such as intellectual properties or cryptographic keys, require confidentiality protection in addition to integrity and authenticity when being persisted or transmitted over a communication channel.

Another aspect of protecting Software Update Packages is their freshness. An attacker may try to manipulate the system by downgrading the software via replaying an authentic but older Software Update Package. In this regard, the platform shall ensure that only newer packages (i.e. packages that contain newer version of installed SWCL) can be installed.

E.2 Securing Calls to UCM

UCM provides a very critical functionality in the platform that allows modifying applications and platform components. In that sense, it is critical to prevent unauthorized access to UCM, meaning only legitimate callers should be allowed to reach the UCM service interface. This is primarily enforced in the communication layer supported by the Identity and Access Management. Additionally, the calls to the UCM interface shall be protected against altering, e.g. changing API arguments. When the service and client reside on the same machine, the security relies on the integrity of the operating system and the platform. In case, the service and the client are running on different machines, a secure communication, assuring authenticity and integrity of communication, is additionally required.

Moreover, some API methods of the UCM interface returns sensitive information about the platform. This subset (GetSwClusterInfo, GetSwClusterChangeInfo, GetHistory, GetSwPackages) shall be protected against information disclosure and should only be reachable over a channel that provides confidentiality.

E.3 Suppressing Call to UCM

Multiple scenarios can be envisioned where an attacker targets suppressing the calls to UCM. The attack could block the calls to or the response from UCM. In both cases the caller of the service may assume that UCM is not responding and retries its request. This would lead to undesired overhead on the system. For such scenarios, it is recommended that both UCM and the UCM Client consider reporting security events when same calls repeatedly received at UCM or calls repeatedly fail at the caller side. This information could potentially be picked up by Intrusion Detection Systems or Anomaly Detection Systems.

E.4 Resource Starvation

According to the current specification, the available resources for transferring a Software Package is only checked when TransferStart is called but not reserved. This means, while the transfer is ongoing, the system storage can be exhausted by other processes using the same storage media. A similar case is possible for processing of Software Package, as the resources are only checked at the beginning but not reserved. In this regard, a solution could be to reserve the necessary resources for the Software Package transfer or processing from the beginning to prevent attacks aiming at such scenarios.

At the same time, reserving the resources might provide opportunity to the attacker in other scenarios. The specification allows transferring multiple Software Packages in parallel. Consequently, a misbehaving or compromised client can open unlimited number of transfer sessions causing UCM to run out of resources. To cope with this scenario, a threshold for the number of parallel transfer sessions can be defined.

E.5 Zombie Sessions

The AUTOSAR specification does not enforce any expiry time for the established transfer sessions. As a result, the resources that are hold by an ongoing session will not be released no matter how long time it takes. At the same time, in certain cases it may take a long time for larger software packages to be transferred to UCM, especially when they are received from external sources with weak connectivity on-the-fly. However, a timeout may be considered for such a transfer to prevent attackers from mounting denial of service attacks by long term allocation of resources.

F History of Constraints and Specification Items

Please note that the lists in this chapter also include constraints and specification items that have been removed from the specification in a later version. These constraints and specification items do not appear as hyperlinks in the document.

F.1 Constraint and Specification Item History of this document according to AUTOSAR Release R19-11.

F.1.1 Added Specification Items in R19-11

Number	Heading
[SWS_UCM_00009]	UCM exposing its identifier
[SWS_UCM_00105]	UCM confidential information handling
[SWS_UCM_00161]	Check Software Package version compatibility against UCM version
[SWS_UCM_00162]	Entering the Cleaning-up state after a RevertProcessedSwPackages call
[SWS_UCM_00163]	Action in Cleaning-up state
[SWS_UCM_00164]	Cleaning up of Software Packages
[SWS_UCM_00165]	Processing from stream
[SWS_UCM_00166]	Processing from stream state
[SWS_UCM_00167]	Cancelling streamed packages
[SWS_UCM_00168]	Transferring while processing from stream
[SWS_UCM_00169]	Finishing transfer while processing from stream
[SWS_UCM_00170]	Log message retrieving
[SWS_UCM_00171]	Log level changing
[SWS_UCM_00172]	Log messages removing
[SWS_UCM_00173]	UCMIdentifierType table
[SWS_UCM_00174]	SwNameVectorType table
[SWS_UCM_00175]	StrongRevisionLabelString table
[SWS_UCM_00176]	SwNameVersionType table
[SWS_UCM_00177]	SwNameVersionVectorType table
[SWS_UCM_00178]	ProvidedPort VehiclePackageManagement
[SWS_UCM_00179]	RequiredPort VehicleStateManager
[SWS_UCM_00180]	RequiredPort VehicleDriverApplication
[SWS_UCM_00181]	ProvidedInterface VehiclePackageManagement
[SWS_UCM_00182]	RequiredInterface VehicleDriverApplication
[SWS_UCM_00183]	RequiredInterface VehicleStateManager
[SWS_UCM_00210]	Transferring of software packages on kProcessApproving or kProcessing state





Number	Heading
[SWS_UCM_01001]	UCM Master processes Vehicle Package
[SWS_UCM_01002]	UCM Master shall provide UCM services
[SWS_UCM_01003]	UCM Master checks states of UCM subordinates
[SWS_UCM_01004]	Only one UCM Master shall be active per network domain
[SWS_UCM_01005]	UCM Master is discovering UCMs in vehicle
[SWS_UCM_01006]	Vehicle Package transfer to UCM Master
[SWS_UCM_01007]	Start transfer of a Vehicle Package or Software Package to UCM Master
[SWS_UCM_01008]	Transfer data of a Vehicle Package to UCM Master
[SWS_UCM_01009]	Exit the transfer of a Vehicle Package to UCM Master
[SWS_UCM_01010]	Delete a Vehicle Package transferred to UCM Master
[SWS_UCM_01101]	Provide information of installed Software Clusters in vehicle
[SWS_UCM_01102]	Get information of available Software Clusters in Backend
[SWS_UCM_01103]	Inform Backend of needed Software Clusters for an update
[SWS_UCM_01105]	Interaction of UCM Master with Vehicle Driver
[SWS_UCM_01106]	Exclusive use of Vehicle Driver Interface
[SWS_UCM_01107]	UCM Master provides progress information to Vehicle Driver
[SWS_UCM_01108]	Unsupported safety policy by Vehicle driver interface
[SWS_UCM_01109]	Vehicle State Manager shall provide to UCM Master a safety state
[SWS_UCM_01110]	UCM Master shall be able to set the safety policy to be computed by Vehicle State Manager
[SWS_UCM_01111]	Exclusive use of Vehicle State Manager
[SWS_UCM_01112]	Unsupported safety policy by Vehicle State Manager
[SWS_UCM_01113]	Switching vehicle into update mode
[SWS_UCM_01114]	SafetyPolicyType table
[SWS_UCM_01115]	VehicleStateManagerErrorDomain
[SWS_UCM_01116]	VehicleDriverApplicationErrorDomain
[SWS_UCM_01177]	CampaignStateType table
[SWS_UCM_01201]	Sequential orchestration of campaigns
[SWS_UCM_01203]	CampaignState field
[SWS_UCM_01204]	Initial state
[SWS_UCM_01205]	UCM Master internal state persistency
[SWS_UCM_01206]	Trigger on kTransferApproving state
[SWS_UCM_01207]	Trigger on kTransferring state
[SWS_UCM_01208]	Trigger on kProcessApproving state
[SWS_UCM_01209]	Trigger on kProcessing state
[SWS_UCM_01211]	Trigger on kActivateApproving state
[SWS_UCM_01212]	Trigger on kActivating state





Number	Heading
[SWS_UCM_01213]	Trigger on <code>kVehicleChecking</code> state
[SWS_UCM_01214]	Final action on <code>kVehicleChecking</code> state
[SWS_UCM_01215]	Trigger on <code>kRollingBack</code> state
[SWS_UCM_01216]	Final action on <code>kRollingBack</code> state
[SWS_UCM_01217]	Monitoring of UCM subordinates
[SWS_UCM_01218]	Transition from <code>kIdle</code> state to <code>kSyncing</code> state
[SWS_UCM_01219]	Transition from <code>kSyncing</code> state to <code>kIdle</code> state
[SWS_UCM_01220]	Transition from <code>kIdle</code> state to <code>kVehiclePackageTransferring</code> state
[SWS_UCM_01221]	Transition from <code>kVehiclePackageTransferring</code> state to <code>kIdle</code> state
[SWS_UCM_01222]	Transition from <code>kVehiclePackageTransferring</code> state to <code>kTransferring</code> state
[SWS_UCM_01223]	Transition from <code>kVehiclePackageTransferring</code> state to <code>kTransferApproving</code> state
[SWS_UCM_01224]	Transition from <code>kTransferApproving</code> state to <code>kTransferring</code> state
[SWS_UCM_01225]	Transition from <code>kTransferApproving</code> state to <code>kIdle</code> state
[SWS_UCM_01226]	Transition from <code>kTransferring</code> state to <code>kTransferApproving</code> state
[SWS_UCM_01227]	Transition from <code>kTransferring</code> state to <code>kIdle</code> state
[SWS_UCM_01228]	Transition from <code>kTransferring</code> state to <code>kProcessing</code> state
[SWS_UCM_01229]	SafetyPolicy while processing stream
[SWS_UCM_01230]	Transition from <code>kTransferring</code> state to <code>kProcessApproving</code> state
[SWS_UCM_01231]	Transition from <code>kProcessApproving</code> state to <code>kProcessing</code> state
[SWS_UCM_01232]	Transition from <code>kProcessApproving</code> state to <code>kIdle</code> state
[SWS_UCM_01233]	Transition from <code>kProcessing</code> state to <code>kProcessApproving</code> state
[SWS_UCM_01234]	Transition from <code>kProcessing</code> state to <code>kActivating</code> state
[SWS_UCM_01235]	Transition from <code>kProcessing</code> state to <code>kActivateApproving</code> state
[SWS_UCM_01236]	Transition from <code>kProcessing</code> state to <code>kIdle</code> state
[SWS_UCM_01237]	Transition from <code>kActivateApproving</code> state to <code>kActivating</code> state
[SWS_UCM_01238]	Transition from <code>kActivateApproving</code> state to <code>kIdle</code> state
[SWS_UCM_01239]	Transition from <code>kActivating</code> state to <code>kRollingBack</code> state
[SWS_UCM_01240]	Transition from <code>kActivating</code> state to <code>kVehicleChecking</code> state
[SWS_UCM_01241]	Transition from <code>kVehicleChecking</code> state to <code>kRollingBack</code> state
[SWS_UCM_01242]	Transition from <code>kVehicleChecking</code> state to <code>kIdle</code> state
[SWS_UCM_01243]	Transition from <code>kRollingBack</code> state to <code>kIdle</code> state
[SWS_UCM_01244]	Cancellation of an update campaign shall be possible
[SWS_UCM_01245]	Cancellation during activation shall be possible
[SWS_UCM_01246]	Unreachable UCM during update campaign
[SWS_UCM_01247]	Method to read History Report
[SWS_UCM_01248]	Content of History Report





Number	Heading
[SWS_UCM_01301]	Vehicle Package authentication
[SWS_UCM_01302]	Vehicle Package authentication failure
[SWS_UCM_01303]	Dependencies between Software Packages
[SWS_UCM_01304]	Confidential information protection
[SWS_UCM-CONSTR_00001]	

Table F.1: Added Specification Items in R19-11

F.1.2 Changed Specification Items in R19-11

Number	Heading
[SWS_UCM_00003]	Cancelling the package processing
[SWS_UCM_00017]	Sequential Software Package Processing
[SWS_UCM_00018]	Providing Progress Information
[SWS_UCM_00027]	Delta Package activation
[SWS_UCM_00071]	SwNameType table
[SWS_UCM_00081]	Processing state of Package Management
[SWS_UCM_00082]	Exit from Processing state of Package Management
[SWS_UCM_00102]	Update state
[SWS_UCM_00103]	Update to older Software Cluster version than currently present
[SWS_UCM_00104]	Consistency Check of processed Package
[SWS_UCM_00111]	Entering the Rolled-back state
[SWS_UCM_00112]	Software Cluster and version
[SWS_UCM_00126]	Entering the RollingBack state after a Rollback call
[SWS_UCM_00130]	Software Cluster and version error
[SWS_UCM_00146]	Entering the Cleaning-up state after a Finish call
[SWS_UCM_00149]	Return to the Idle state from Processing state
[SWS_UCM_00151]	Entering the Ready state of Package Management after a Cancel call
[SWS_UCM_00155]	Entering the RollingBack state after a failure in the Verifying state

Table F.2: Changed Specification Items in R19-11

F.1.3 Deleted Specification Items in R19-11

Number	Heading
[SWS_UCM_00012]	Log message retrieving
[SWS_UCM_00114]	ActivateOptionType table
[SWS_UCM_00144]	Log error

Table F.3: Deleted Specification Items in R19-11

F.1.4 Added Constraints in R19-11

none

F.1.5 Changed Constraints in R19-11

none

F.1.6 Deleted Constraints in R19-11

none

F.2 Constraint and Specification Item History of this document according to AUTOSAR Release R20-11.

F.2.1 Added Specification Items in R20-11

Number	Heading
[SWS_UCM_00184]	Persistent data clean-up after Software Cluster removal
[SWS_UCM_00185]	Provide SoftwareCluster general information
[SWS_UCM_00186]	
[SWS_UCM_00187]	
[SWS_UCM_00190]	Reinstallation of older Software Cluster version than previously removed
[SWS_UCM_00191]	Software Cluster life-cycle state kAdded
[SWS_UCM_00192]	Software Cluster life-cycle state transition from kAdded to kPresent
[SWS_UCM_00193]	Software Cluster life-cycle state transition from kUpdating to kPresent
[SWS_UCM_00194]	Software Cluster life-cycle state transition from kRemoved to kPresent
[SWS_UCM_00195]	Software Cluster life-cycle state kUpdating





Number	Heading
[SWS_UCM_00196]	Software Cluster life-cycle state kRemoved
[SWS_UCM_00197]	End of Software Cluster life-cycle state from state kAdded
[SWS_UCM_00198]	End of Software Cluster life-cycle state from state kRemoved
[SWS_UCM_00199]	Reporting of Software Cluster reaching end of life-cycle
[SWS_UCM_00200]	Failing authentication
[SWS_UCM_00201]	Delta Package dependency error
[SWS_UCM_00202]	Trusted Platform compliance
[SWS_UCM_00203]	TransferData InvalidTransferId
[SWS_UCM_00204]	TransferData IncorrectBlock
[SWS_UCM_00205]	TransferData IncorrectSize
[SWS_UCM_00206]	TransferData InsufficientMemory
[SWS_UCM_00207]	TransferData BlockInconsistent
[SWS_UCM_00208]	TransferData OperationNotPermitted
[SWS_UCM_00209]	TransferData PackageInconsistent
[SWS_UCM_00211]	TransferData TransferInterrupted
[SWS_UCM_00212]	TransferExit InvalidTransferId
[SWS_UCM_00213]	TransferExit InvalidPackageManifest
[SWS_UCM_00214]	DeleteTransfer InvalidTransferId
[SWS_UCM_00215]	DeleteTransfer OperationNotPermitted
[SWS_UCM_00216]	Validity of TransferId
[SWS_UCM_00217]	ProcessSwPackage InsufficientMemory
[SWS_UCM_00218]	ProcessSwPackage InvalidTransferId
[SWS_UCM_00219]	ProcessSwPackage OperationNotPermitted
[SWS_UCM_00220]	GetSwProcessProgress InvalidTransferId
[SWS_UCM_00230]	ProcessSwPackage AuthenticationFailed
[SWS_UCM_00231]	ProcessSwPackage IncompatibleDelta
[SWS_UCM_00232]	ProcessSwPackage
[SWS_UCM_00233]	Cancel Operation CancelFailed
[SWS_UCM_00234]	Cancel OperationNotPermitted
[SWS_UCM_00235]	Cancel InvalidTransferId
[SWS_UCM_00236]	RevertProcessedSwPackages NotAbleToRevertPackages
[SWS_UCM_00237]	RevertProcessedSwPackages OperationNotPermitted
[SWS_UCM_00238]	Rollback NotAbleToRollback
[SWS_UCM_00239]	Rollback OperationNotPermitted
[SWS_UCM_00240]	Finish OperationNotPermitted
[SWS_UCM_00241]	Activate OperationNotPermitted
[SWS_UCM_00242]	Activate PreActivationFailed
[SWS_UCM_00243]	Too big block size received by UCM





Number	Heading
[SWS_UCM_00245]	Software Cluster category
[SWS_UCM_00250]	TransferData AuthenticationFailed
[SWS_UCM_00251]	
[SWS_UCM_00252]	
[SWS_UCM_00253]	
[SWS_UCM_00254]	
[SWS_UCM_00255]	
[SWS_UCM_00256]	
[SWS_UCM_00257]	Update session
[SWS_UCM_00258]	Update session rejected
[SWS_UCM_00259]	Ending the update session
[SWS_UCM_00260]	PrepareUpdate, VerifyUpdate and PrepareRollback orders
[SWS_UCM_00261]	PrepareUpdate, VerifyUpdate and PrepareRollback synchronous calls
[SWS_UCM_00262]	Update preparation rejected
[SWS_UCM_00263]	Update preparation failure
[SWS_UCM_00264]	Update verification rejected
[SWS_UCM_01011]	TransferVehiclePackage InsufficientMemory
[SWS_UCM_01012]	TransferVehiclePackage InsufficientComputationPower
[SWS_UCM_01013]	Too big block size received by UCM Master
[SWS_UCM_01014]	Packages transferring sequence
[SWS_UCM_01015]	Invalid Vehicle Package manifest
[SWS_UCM_01016]	Invalid Package Manifest
[SWS_UCM_01017]	RequestedPackage field
[SWS_UCM_01117]	UCM Master SafetyState field
[SWS_UCM_01118]	UCM Master waiting for vehicle driver approval
[SWS_UCM_01119]	Report information of Software Packages
[SWS_UCM_01120]	Provide Software Packages general information
[SWS_UCM_01121]	Adaptive Platform interface provided for Flashing Adapter
[SWS_UCM_01122]	Supported physical layers by D-PDU API implementation
[SWS_UCM_01123]	Supported application layers by D-PDU API implementation
[SWS_UCM_01124]	Supported protocols by D-PDU API implementation
[SWS_UCM_01125]	Separation of D-PDU API -Software with the MVCI protocol module firmware
[SWS_UCM_01126]	Root description file (RDF)
[SWS_UCM_01127]	Module Description File (MDF)
[SWS_UCM_01128]	Symbolic names and IDs
[SWS_UCM_01129]	SAE J2534-1 and RP 1210a compatibility
[SWS_UCM_01130]	ComPrimitives in RawMode
[SWS_UCM_01131]	PDUIoCtl(PDU_IOCTL_RESET)



△

Number	Heading
[SWS_UCM_01132]	PDUIoCtl(PDU_IOCTL_START_MSG_FILTER), PDUIoCtl(PDU_IOCTL_CLEAR_MSG_FILTER), PDUIoCtl(PDU_IOCTL_STOP_MSG_FILTER)
[SWS_UCM_01133]	PDUIoCtl(PDU_IOCTL_SEND_BREAK)
[SWS_UCM_01134]	Not used D-PDU API function return codes
[SWS_UCM_01178]	
[SWS_UCM_01265]	TransferState field
[SWS_UCM_01266]	Subordinate Not Available On The Network
[SWS_UCM_01267]	Vehicle State Manager Communication Error
[SWS_UCM_01268]	Vehicle Driver Interface Communication Error
[SWS_UCM_01269]	Campaign cancellation history
[SWS_UCM_01270]	New campaign disabling
[SWS_UCM_01271]	New campaign enabling
[SWS_UCM_01305]	Vehicle Package format
[SWS_UCM_01306]	TransferExit Invalid package manifest
[SWS_UCM - CONSTR_00002]	UCM confidential information handling
[SWS_UCM - CONSTR_00003]	Exclusive use of Vehicle Driver Interface
[SWS_UCM - CONSTR_00004]	Unsupported safety policy by Vehicle driver interface
[SWS_UCM - CONSTR_00005]	Safety state change
[SWS_UCM - CONSTR_00006]	Exclusive use of Vehicle State Manager
[SWS_UCM - CONSTR_00007]	Unsupported safety policy by Vehicle State Manager
[SWS_UCM - CONSTR_00008]	Switching vehicle into update mode
[SWS_UCM - CONSTR_00009]	Safety policy change
[SWS_UCM - CONSTR_00010]	UCM Client update sequence
[SWS_UCM - CONSTR_00011]	Flashing Adapter provided interface

Table F.4: Added Specification Items in R20-11

F.2.2 Changed Specification Items in R20-11

Number	Heading
[SWS_UCM_00018]	Providing Progress Information
[SWS_UCM_00020]	Finishing the packages activation
[SWS_UCM_00025]	Activation of SoftwareClusters
[SWS_UCM_00026]	Dependency Check
[SWS_UCM_00027]	Delta Package activation
[SWS_UCM_00028]	Software Package Authentication
[SWS_UCM_00029]	Consistency Check of Manifest
[SWS_UCM_00031]	
[SWS_UCM_00032]	
[SWS_UCM_00038]	
[SWS_UCM_00039]	
[SWS_UCM_00040]	
[SWS_UCM_00044]	
[SWS_UCM_00069]	Report information on Software Packages
[SWS_UCM_00071]	
[SWS_UCM_00073]	
[SWS_UCM_00077]	
[SWS_UCM_00078]	
[SWS_UCM_00079]	
[SWS_UCM_00084]	Entering the kActivating state of Package Management
[SWS_UCM_00085]	Entering the kActivated state of Package Management
[SWS_UCM_00088]	Preparation of data transfer
[SWS_UCM_00092]	Software Package integrity
[SWS_UCM_00098]	Software Package Authentication failure
[SWS_UCM_00107]	Activated state
[SWS_UCM_00110]	Rolling-back the software update
[SWS_UCM_00111]	Entering the kRolled-Back state
[SWS_UCM_00112]	Software Cluster and version
[SWS_UCM_00115]	History
[SWS_UCM_00126]	Entering the kRolling-Back state after a Rollback call
[SWS_UCM_00130]	Software Cluster and version error
[SWS_UCM_00131]	
[SWS_UCM_00132]	
[SWS_UCM_00133]	
[SWS_UCM_00134]	
[SWS_UCM_00135]	





Number	Heading
[SWS_UCM_00136]	
[SWS_UCM_00137]	Processing several update Software Packages
[SWS_UCM_00145]	Sequential order of data transfer
[SWS_UCM_00147]	Return to the Idle state from Cleaning-up state
[SWS_UCM_00148]	Transfer sequence order
[SWS_UCM_00149]	Return to the Idle state from Processing state
[SWS_UCM_00151]	Entering the Ready state of Package Management after a Cancel call
[SWS_UCM_00153]	Action in kActivating state of Package Management
[SWS_UCM_00154]	Entering the Verifying state of Package Management
[SWS_UCM_00155]	Entering the kRolling-Back state after a failure in the kVerifying state
[SWS_UCM_00158]	Cleanup of interrupted actions
[SWS_UCM_00162]	Entering the Cleaning-up state after a RevertProcessedSwPackages call
[SWS_UCM_00165]	Processing from stream
[SWS_UCM_00166]	Processing from stream state
[SWS_UCM_00167]	Cancelling streamed packages
[SWS_UCM_00168]	Transferring while processing from stream
[SWS_UCM_00169]	Finishing transfer while processing from stream
[SWS_UCM_00173]	
[SWS_UCM_00174]	
[SWS_UCM_00175]	
[SWS_UCM_00176]	
[SWS_UCM_00177]	
[SWS_UCM_00178]	
[SWS_UCM_00179]	
[SWS_UCM_00180]	
[SWS_UCM_00181]	
[SWS_UCM_00182]	
[SWS_UCM_00183]	
[SWS_UCM_00210]	Transferring of software packages on kProcessing state
[SWS_UCM_01003]	UCM Master checks states of UCM subordinates
[SWS_UCM_01006]	Start transfer of a Vehicle Package to UCM Master
[SWS_UCM_01007]	Start transfer of a Software Package to UCM Master
[SWS_UCM_01008]	Transfer data of a Vehicle Package or Software Package to UCM Master
[SWS_UCM_01009]	Exit the transfer of a Vehicle Package or Software Package to UCM Master
[SWS_UCM_01010]	Delete a Vehicle Package transferred to UCM Master
[SWS_UCM_01101]	Provide information of installed Software Clusters in vehicle
[SWS_UCM_01102]	Get information of available Software Clusters in Backend





Number	Heading
[SWS_UCM_01103]	Inform Backend of needed Software Clusters for an update
[SWS_UCM_01105]	Interaction of UCM Master with Vehicle Driver
[SWS_UCM_01107]	UCM Master provides progress information to Vehicle Driver
[SWS_UCM_01109]	UCM Master provides a safety policy interface
[SWS_UCM_01110]	UCM Master SafetyState method
[SWS_UCM_01114]	
[SWS_UCM_01177]	
[SWS_UCM_01203]	CampaignState field
[SWS_UCM_01207]	Trigger on kSoftwarePackage_Transferring state
[SWS_UCM_01221]	Transition from kVehiclePackageTransferring state to kIdle state
[SWS_UCM_01222]	Transition from kVehiclePackageTransferring state to kSoftwarePackage_Transferring state
[SWS_UCM_01227]	Transition from kSoftwarePackage_Transferring state to kIdle state
[SWS_UCM_01228]	Transition from kSoftwarePackage_Transferring state to kProcessing state
[SWS_UCM_01229]	SafetyPolicy while processing stream
[SWS_UCM_01234]	Transition from kProcessing state to kActivating state
[SWS_UCM_01236]	Transition from kProcessing state to kIdle state
[SWS_UCM_01239]	Transition from kActivating state to kCancelling state
[SWS_UCM_01240]	Transition from kActivating state to kVehicleChecking state
[SWS_UCM_01244]	Cancellation of an update campaign shall be possible
[SWS_UCM_01245]	Cancellation during activation shall be possible
[SWS_UCM_01246]	Unreachable UCM during update campaign
[SWS_UCM_01247]	Method to read History Report
[SWS_UCM_01302]	Vehicle Package authentication failure
[SWS_UCM_01304]	Confidential information protection
[SWS_UCM_CONSTR_00001]	

Table F.5: Changed Specification Items in R20-11

F.2.3 Deleted Specification Items in R20-11

Number	Heading
[SWS_UCM_00011]	Updating persisted data
[SWS_UCM_00041]	LogLevelType table
[SWS_UCM_00042]	LogEntryType table
[SWS_UCM_00043]	LogVectorType table





Number	Heading
[SWS_UCM_00082]	Exit from Processing state of Package Management
[SWS_UCM_00091]	Successful data transfer
[SWS_UCM_00096]	Entering the Rolled-back state
[SWS_UCM_00102]	Update state
[SWS_UCM_00105]	UCM confidential information handling
[SWS_UCM_00108]	Execution of the update software
[SWS_UCM_00113]	Rollback of persisted data
[SWS_UCM_00124]	Verify State
[SWS_UCM_00128]	
[SWS_UCM_00141]	UCM insufficient memory for parallel data transfer
[SWS_UCM_00142]	Prevent software from blocking the Rollback operation
[SWS_UCM_00143]	Log level setting
[SWS_UCM_00156]	Procurement of Checksum
[SWS_UCM_00170]	Log message retrieving
[SWS_UCM_00171]	Log level changing
[SWS_UCM_00172]	Log messages removing
[SWS_UCM_01002]	UCM Master shall provide UCM services
[SWS_UCM_01106]	Exclusive use of Vehicle Driver Interface
[SWS_UCM_01108]	Unsupported safety policy by Vehicle driver interface
[SWS_UCM_01111]	Exclusive use of Vehicle State Manager
[SWS_UCM_01112]	Unsupported safety policy by Vehicle State Manager
[SWS_UCM_01113]	Switching vehicle into update mode
[SWS_UCM_01115]	VehicleStateManagerErrorDomain
[SWS_UCM_01116]	VehicleDriverApplicationErrorDomain
[SWS_UCM_01206]	Trigger on kTransferApproving state
[SWS_UCM_01208]	Trigger on kProcessApproving state
[SWS_UCM_01211]	Trigger on kActivateApproving state
[SWS_UCM_01223]	Transition from kVehiclePackageTransferring state to kTransferApproving state
[SWS_UCM_01224]	Transition from kTransferApproving state to kTransferring state
[SWS_UCM_01225]	Transition from kTransferApproving state to kIdle state
[SWS_UCM_01226]	Transition from kTransferring state to kTransferApproving state
[SWS_UCM_01230]	Transition from kTransferring state to kProcessApproving state
[SWS_UCM_01231]	Transition from kProcessApproving state to kProcessing state
[SWS_UCM_01232]	Transition from kProcessApproving state to kIdle state
[SWS_UCM_01233]	Transition from kProcessing state to kProcessApproving state
[SWS_UCM_01235]	Transition from kProcessing state to kActivateApproving state
[SWS_UCM_01237]	Transition from kActivateApproving state to kActivating state





Number	Heading
[SWS_UCM_01238]	Transition from kActivateApproving state to kIdle state

Table F.6: Deleted Specification Items in R20-11

F.2.4 Added Constraints in R20-11

none

F.2.5 Changed Constraints in R20-11

none

F.2.6 Deleted Constraints in R20-11

none

F.3 Constraint and Specification Item History of this document according to AUTOSAR Release R21-11.

F.3.1 Added Specification Items in R21-11

Number	Heading
[SWS_UCM_00265]	state transition due to ProcessSwPackage error
[SWS_UCM_00266]	OperationNotPermitted error and UCM state
[SWS_UCM_00267]	Error when checksum is not recognised at processing time
[SWS_UCM_00268]	
[SWS_UCM_00269]	
[SWS_UCM_00270]	UCM internal state persistency
[SWS_UCM_00271]	Keeping history of failure error code
[SWS_UCM_00272]	Transfer block size
[SWS_UCM_00273]	Persistent data clean-up after Software Cluster update that removes a process
[SWS_UCM_00274]	UCM initialization
[SWS_UCM_00275]	TransferData error handling order
[SWS_UCM_00276]	TransferExit error handling order
[SWS_UCM_00277]	ProcessSwPackage error handling order





Number	Heading
[SWS_UCM_00278]	Cancel error handling order
[SWS_UCM_00279]	RevertProcessedSwPackages error handling order
[SWS_UCM_00280]	Activate VerificationFailed
[SWS_UCM_00281]	Activate error handling order
[SWS_UCM_00282]	Rollback error handling order
[SWS_UCM_00283]	DeleteTransfer error handling order
[SWS_UCM_00285]	Removing or updating a Software Cluster not existing in the Machine
[SWS_UCM_00286]	Software Cluster life-cycle state transition from kRemoved to kPresent in case of Finish call
[SWS_UCM_00287]	End of Software Cluster life-cycle state from state kAdded in case of Finish call
[SWS_UCM_00288]	
[SWS_UCM_00289]	TransferData TransferFailed
[SWS_UCM_01018]	TransferVehiclePackage BusyWithCampaign
[SWS_UCM_01019]	UCM Master initialization
[SWS_UCM_01135]	Get Software Clusters descriptions from a vehicle
[SWS_UCM_01136]	
[SWS_UCM_01137]	
[SWS_UCM_01138]	
[SWS_UCM_01272]	VehicleCheck call not permitted
[SWS_UCM_01273]	CancelCampaign CancelFailed error
[SWS_UCM_01274]	CancelCampaign OperationNotPermitted error
[SWS_UCM_CONSTR_00012]	
[SWS_UCM_CONSTR_00013]	Confidential information protection
[SWS_UCM_CONSTR_00014]	Software Package and Software Cluster shortNames
[SWS_UCM_CONSTR_00015]	Trigger on kVehicleChecking state

Table F.7: Added Specification Items in R21-11

F.3.2 Changed Specification Items in R21-11

Number	Heading
[SWS_UCM_00004]	Report software information
[SWS_UCM_00009]	UCM exposing its identifier
[SWS_UCM_00017]	Sequential Software Package Processing





Number	Heading
[SWS_UCM_00020]	Finishing the packages activation
[SWS_UCM_00030]	Report changes
[SWS_UCM_00039]	
[SWS_UCM_00044]	
[SWS_UCM_00078]	
[SWS_UCM_00080]	Idle state of Package Management
[SWS_UCM_00081]	Processing state of Package Management
[SWS_UCM_00083]	Entering the Ready state of Package Management after a successful processing operation
[SWS_UCM_00084]	Entering the kActivating state of Package Management
[SWS_UCM_00085]	Entering the kActivated state of Package Management
[SWS_UCM_00092]	Software Package integrity
[SWS_UCM_00103]	Update to older <i>Software Cluster</i> version than currently present
[SWS_UCM_00104]	Integrity Check of processed Package
[SWS_UCM_00107]	Activated state
[SWS_UCM_00110]	Rolling-back the software update
[SWS_UCM_00111]	Entering the kRollingBack state
[SWS_UCM_00115]	History
[SWS_UCM_00126]	Entering the kRollingBack state after a Rollback call
[SWS_UCM_00127]	Finishing update sequence
[SWS_UCM_00130]	<i>Software Cluster</i> and version error
[SWS_UCM_00131]	
[SWS_UCM_00133]	
[SWS_UCM_00134]	
[SWS_UCM_00136]	
[SWS_UCM_00146]	Entering the Cleaning-up state after a Finish call
[SWS_UCM_00147]	Return to the Idle state from Cleaning-up state
[SWS_UCM_00149]	Return to the Idle state from Processing state
[SWS_UCM_00151]	Entering the Ready state of Package Management after a Cancel call
[SWS_UCM_00152]	Entering the Ready state of Package Management after a missing dependency
[SWS_UCM_00153]	Action in kActivating state of Package Management
[SWS_UCM_00154]	Entering the Verifying state of Package Management
[SWS_UCM_00155]	Entering the kRolling-Back state after a failure in the kVerifying state
[SWS_UCM_00162]	Entering the Cleaning-up state after a <i>RevertProcessedSwPackages</i> call
[SWS_UCM_00163]	Action in Cleaning-up state
[SWS_UCM_00164]	Cleaning up of Software Packages
[SWS_UCM_00166]	Processing from stream state
[SWS_UCM_00167]	Cancelling streamed packages





Number	Heading
[SWS_UCM_00168]	Transferring while processing from stream
[SWS_UCM_00169]	Finishing transfer while processing from stream
[SWS_UCM_00176]	
[SWS_UCM_00181]	
[SWS_UCM_00182]	
[SWS_UCM_00183]	
[SWS_UCM_00185]	Provide <code>SoftwareCluster</code> general information
[SWS_UCM_00186]	
[SWS_UCM_00190]	Reinstallation of older <code>Software Cluster</code> version than previously removed
[SWS_UCM_00191]	<code>Software Cluster</code> life-cycle state <code>kAdded</code>
[SWS_UCM_00192]	<code>Software Cluster</code> life-cycle state transition from <code>kAdded</code> to <code>kPresent</code>
[SWS_UCM_00193]	<code>Software Cluster</code> life-cycle state transition from <code>kUpdating</code> to <code>kPresent</code>
[SWS_UCM_00194]	<code>Software Cluster</code> life-cycle state transition from <code>kRemoved</code> to <code>kPresent</code> in case of <code>RevertProcessedSwPackages</code> call
[SWS_UCM_00195]	<code>Software Cluster</code> life-cycle state <code>kUpdating</code>
[SWS_UCM_00196]	<code>Software Cluster</code> life-cycle state <code>kRemoved</code>
[SWS_UCM_00197]	End of <code>Software Cluster</code> life-cycle state from state <code>kAdded</code> in case of <code>RevertProcessedSwPackages</code> call
[SWS_UCM_00198]	End of <code>Software Cluster</code> life-cycle state from state <code>kRemoved</code>
[SWS_UCM_00200]	Failing authentication
[SWS_UCM_00209]	<code>TransferData</code> <code>PackageInconsistent</code>
[SWS_UCM_00210]	Transferring of software packages on <code>kProcessing</code> state
[SWS_UCM_00213]	<code>TransferExit</code> <code>InvalidPackageManifest</code>
[SWS_UCM_00214]	<code>DeleteTransfer</code> <code>InvalidTransferId</code>
[SWS_UCM_00215]	<code>DeleteTransfer</code> <code>OperationNotPermitted</code>
[SWS_UCM_00220]	<code>GetSwProcessProgress</code> <code>InvalidTransferId</code>
[SWS_UCM_00237]	<code>RevertProcessedSwPackages</code> <code>OperationNotPermitted</code>
[SWS_UCM_00239]	<code>Rollback</code> <code>OperationNotPermitted</code>
[SWS_UCM_00240]	<code>Finish</code> <code>OperationNotPermitted</code>
[SWS_UCM_00241]	<code>Activate</code> <code>OperationNotPermitted</code>
[SWS_UCM_00242]	<code>Activate</code> <code>PreActivationFailed</code>
[SWS_UCM_00243]	Too big block size received by UCM
[SWS_UCM_00251]	
[SWS_UCM_00252]	
[SWS_UCM_00253]	
[SWS_UCM_00254]	
[SWS_UCM_00255]	





Number	Heading
[SWS_UCM_00257]	Update session
[SWS_UCM_00258]	Update session rejected
[SWS_UCM_00259]	Ending the update session
[SWS_UCM_00260]	PrepareUpdate, VerifyUpdate and PrepareRollback orders
[SWS_UCM_00261]	PrepareUpdate, VerifyUpdate and PrepareRollback synchronous calls
[SWS_UCM_00262]	Update preparation rejected
[SWS_UCM_00263]	Update preparation failure
[SWS_UCM_00264]	Update verification rejected
[SWS_UCM_01003]	UCM Master checks states of UCM subordinates
[SWS_UCM_01011]	TransferVehiclePackage InsufficientMemory
[SWS_UCM_01015]	Invalid Vehicle Package manifest
[SWS_UCM_01016]	Invalid Package Manifest
[SWS_UCM_01103]	Inform Backend of needed Software Packages for an update
[SWS_UCM_01109]	UCM Master provides a safety interface
[SWS_UCM_01114]	
[SWS_UCM_01117]	UCM Master SafetyState field
[SWS_UCM_01118]	UCM Master waiting for vehicle driver approval
[SWS_UCM_01203]	CampaignState field
[SWS_UCM_01204]	Initial state
[SWS_UCM_01207]	Trigger on kSoftwarePackage_Transferring state
[SWS_UCM_01209]	Trigger on kProcessing state
[SWS_UCM_01212]	Trigger on kActivating state
[SWS_UCM_01214]	Final action on kVehicleChecking state
[SWS_UCM_01215]	Trigger on kCancelling state
[SWS_UCM_01216]	Final action on kCancelling state
[SWS_UCM_01217]	Monitoring of UCM subordinates
[SWS_UCM_01218]	Transition from kIdle state to kSyncing state
[SWS_UCM_01219]	Transition from kSyncing state to kIdle state
[SWS_UCM_01220]	Transition from kIdle state to kVehiclePackageTransferring state
[SWS_UCM_01221]	Transition from kVehiclePackageTransferring state to kIdle state
[SWS_UCM_01222]	Transition from kVehiclePackageTransferring state to kSoftwarePackage_Transferring state
[SWS_UCM_01227]	Transition from kSoftwarePackage_Transferring state to kIdle state
[SWS_UCM_01228]	Transition from kSoftwarePackage_Transferring state to kProcessing state
[SWS_UCM_01229]	SafetyConditions while processing stream
[SWS_UCM_01234]	Transition from kProcessing state to kActivating state
[SWS_UCM_01236]	Transition from kProcessing state to kCancelling state
[SWS_UCM_01239]	Transition from kActivating state to kCancelling state





Number	Heading
[SWS_UCM_01240]	Transition from <code>kActivating</code> state to <code>kVehicleChecking</code> state
[SWS_UCM_01241]	Transition from <code>kVehicleChecking</code> state to <code>kCancelling</code> state
[SWS_UCM_01242]	Transition from <code>kVehicleChecking</code> state to <code>kIdle</code> state
[SWS_UCM_01243]	Transition from <code>kCancelling</code> state to <code>kIdle</code> state
[SWS_UCM_01244]	Cancellation of an update campaign shall be possible
[SWS_UCM_01246]	Unreachable UCM during update campaign
[SWS_UCM_01247]	Method to read History Report
[SWS_UCM_01265]	TransferState field
[SWS_UCM_01270]	New campaign disabling
[SWS_UCM_CONSTR_00002]	UCM confidential information handling
[SWS_UCM_CONSTR_00004]	Unsupported safety by Vehicle driver interface
[SWS_UCM_CONSTR_00005]	Safety state change
[SWS_UCM_CONSTR_00006]	Exclusive use of Vehicle State Manager
[SWS_UCM_CONSTR_00007]	Unsupported safety conditions by Vehicle State Manager
[SWS_UCM_CONSTR_00008]	Switching vehicle into update mode
[SWS_UCM_CONSTR_00009]	Safety condition change

Table F.8: Changed Specification Items in R21-11

F.3.3 Deleted Specification Items in R21-11

Number	Heading
[SWS_UCM_00093]	Transfer sequence
[SWS_UCM_00201]	Delta Package dependency error
[SWS_UCM_00211]	<code>TransferData</code> <code>TransferInterrupted</code>
[SWS_UCM_00230]	<code>ProcessSwPackage</code> <code>AuthenticationFailed</code>
[SWS_UCM_00232]	<code>ProcessSwPackage</code>
[SWS_UCM_00233]	<code>Cancel</code> <code>Operation CancelFailed</code>
[SWS_UCM_00250]	<code>TransferData</code> <code>AuthenticationFailed</code>
[SWS_UCM_01001]	<code>UCM Master</code> processes <code>Vehicle Package</code>
[SWS_UCM_01004]	Only one <code>UCM Master</code> shall be active per network domain
[SWS_UCM_01006]	Start transfer of a <code>Vehicle Package</code> to <code>UCM Master</code>





Number	Heading
[SWS_UCM_01007]	Start transfer of a Software Package to UCM Master
[SWS_UCM_01008]	Transfer data of a Vehicle Package or Software Package to UCM Master
[SWS_UCM_01009]	Exit the transfer of a Vehicle Package or Software Package to UCM Master
[SWS_UCM_01010]	Delete a Vehicle Package transferred to UCM Master
[SWS_UCM_01012]	TransferVehiclePackage InsufficientComputationPower
[SWS_UCM_01102]	Get information of available Software Clusters in Backend
[SWS_UCM_01213]	Trigger on kVehicleChecking state
[SWS_UCM_01245]	Cancellation during activation shall be possible
[SWS_UCM_01304]	Confidential information protection
[SWS_UCM_CONSTR_00010]	UCM Client update sequence

Table F.9: Deleted Specification Items in R21-11

F.3.4 Added Constraints in R21-11

none

F.3.5 Changed Constraints in R21-11

none

F.3.6 Deleted Constraints in R21-11

none

F.4 Constraint and Specification Item History of this document according to AUTOSAR Release R22-11.

F.4.1 Added Specification Items in R22-11

Number	Heading
[SWS_UCM_00290]	
[SWS_UCM_00291]	
[SWS_UCM_00292]	History elements ordering





Number	Heading
[SWS_UCM_00293]	VerifyUpdate method
[SWS_UCM_00294]	Unsupported package format for UCM
[SWS_UCM_00296]	
[SWS_UCM_00297]	Retry Strategy for ServiceBusy
[SWS_UCM_00298]	Retry Strategy for UpdateSessionRejected
[SWS_UCM_00299]	Verify rolled back Software Clusters
[SWS_UCM_00300]	Software Cluster failing to rollback
[SWS_UCM_00301]	Retry ro Rollback again when UCM is in kRollingBackFailed state
[SWS_UCM_00302]	Rollback failing is triggering production error
[SWS_UCM_00303]	failing to record history
[SWS_UCM_01020]	Retry Strategy for BlockInconsistent
[SWS_UCM_01275]	Safety conditions during activation
[SWS_UCM_01307]	Vehicle Package format not supported
[SWS_UCM_01308]	Check Vehicle Package version compatibility against UCM Master version
[SWS_UCM_CONSTR_00016]	OTA Client use of RequestedPackage field
[SWS_UCM_CONSTR_00017]	Interaction of UCM Master with Vehicle Driver

Table F.10: Added Specification Items in R22-11

F.4.2 Changed Specification Items in R22-11

Number	Heading
[SWS_UCM_00001]	Starting the package processing
[SWS_UCM_00003]	Cancelling the package processing
[SWS_UCM_00004]	Report software information
[SWS_UCM_00005]	Rollback to the software prior to Finish the update process
[SWS_UCM_00008]	Executing the data transfer
[SWS_UCM_00018]	Providing Progress Information
[SWS_UCM_00022]	Activation of Software Clusters
[SWS_UCM_00025]	Activation of SoftwareClusters
[SWS_UCM_00026]	Dependency Check
[SWS_UCM_00027]	Delta Package version applicability
[SWS_UCM_00030]	Report changes
[SWS_UCM_00031]	
[SWS_UCM_00032]	
[SWS_UCM_00038]	





Number	Heading
[SWS_UCM_00039]	
[SWS_UCM_00040]	
[SWS_UCM_00044]	
[SWS_UCM_00069]	Report information on Software Packages
[SWS_UCM_00071]	
[SWS_UCM_00073]	
[SWS_UCM_00077]	
[SWS_UCM_00078]	
[SWS_UCM_00079]	
[SWS_UCM_00085]	Entering the kActivated state of Package Management
[SWS_UCM_00087]	Insufficient amount of data transferred
[SWS_UCM_00092]	Software Package integrity
[SWS_UCM_00098]	Software Package Authentication failure
[SWS_UCM_00099]	Update of Adaptive Application
[SWS_UCM_00103]	Update to older Software Cluster version than currently present and than previously removed
[SWS_UCM_00104]	Integrity Check of processed Package
[SWS_UCM_00107]	Activated state
[SWS_UCM_00111]	Entering the kRollingBack state
[SWS_UCM_00120]	Runtime dependencies check
[SWS_UCM_00131]	
[SWS_UCM_00132]	
[SWS_UCM_00133]	
[SWS_UCM_00134]	
[SWS_UCM_00135]	
[SWS_UCM_00136]	
[SWS_UCM_00137]	Processing several update Software Packages
[SWS_UCM_00151]	Entering the Ready state of Package Management after a Cancel call
[SWS_UCM_00154]	Entering the Verifying state of Package Management
[SWS_UCM_00155]	Entering the kRolling-Back state after a failure in the kVerifying state
[SWS_UCM_00160]	Processing results records
[SWS_UCM_00161]	Check Software Package version compatibility against UCM version
[SWS_UCM_00167]	Cancelling streamed packages
[SWS_UCM_00173]	
[SWS_UCM_00175]	
[SWS_UCM_00176]	
[SWS_UCM_00177]	
[SWS_UCM_00178]	





Number	Heading
[SWS_UCM_00179]	
[SWS_UCM_00180]	
[SWS_UCM_00181]	
[SWS_UCM_00182]	
[SWS_UCM_00183]	
[SWS_UCM_00184]	Persistent data clean-up after Software Cluster removal
[SWS_UCM_00185]	Provide SoftwareCluster general information
[SWS_UCM_00186]	
[SWS_UCM_00187]	
[SWS_UCM_00193]	Software Cluster life-cycle state transition from kUpdating to kPresent
[SWS_UCM_00195]	Software Cluster life-cycle state kUpdating
[SWS_UCM_00200]	Failing authentication
[SWS_UCM_00202]	Trusted Platform compliance
[SWS_UCM_00203]	TransferData InvalidTransferId
[SWS_UCM_00204]	TransferData IncorrectBlock
[SWS_UCM_00205]	TransferData IncorrectSize
[SWS_UCM_00206]	TransferData InsufficientMemory
[SWS_UCM_00207]	TransferData BlockInconsistent
[SWS_UCM_00208]	TransferData OperationNotPermitted
[SWS_UCM_00219]	ProcessSwPackage OperationNotPermitted
[SWS_UCM_00231]	ProcessSwPackage IncompatibleDelta
[SWS_UCM_00242]	Activate PrepareUpdateFailed
[SWS_UCM_00245]	Software Cluster category
[SWS_UCM_00251]	
[SWS_UCM_00252]	
[SWS_UCM_00253]	
[SWS_UCM_00254]	
[SWS_UCM_00255]	
[SWS_UCM_00256]	
[SWS_UCM_00257]	Update session
[SWS_UCM_00258]	Update session rejected
[SWS_UCM_00262]	Update preparation rejected
[SWS_UCM_00263]	Update preparation failure
[SWS_UCM_00264]	Update verification rejected
[SWS_UCM_00265]	state transition due to ProcessSwPackage error
[SWS_UCM_00266]	OperationNotPermitted error and UCM state
[SWS_UCM_00268]	
[SWS_UCM_00269]	





Number	Heading
[SWS_UCM_00270]	UCM internal state persistency
[SWS_UCM_00272]	Transfer block size
[SWS_UCM_00273]	Persistent data clean-up after Software Cluster update that removes a process
[SWS_UCM_00274]	UCM initialization
[SWS_UCM_00275]	TransferData error handling order
[SWS_UCM_00276]	TransferExit error handling order
[SWS_UCM_00277]	ProcessSwPackage error handling order
[SWS_UCM_00280]	Activate VerificationFailed
[SWS_UCM_00281]	Activate error handling order
[SWS_UCM_00282]	Rollback error handling order
[SWS_UCM_00285]	Removing or updating a Software Cluster not existing in the Machine
[SWS_UCM_00286]	Software Cluster life-cycle state transition from kRemoved to kPresent in case of Finish call
[SWS_UCM_00287]	End of Software Cluster life-cycle state from state kAdded in case of Finish call
[SWS_UCM_00288]	
[SWS_UCM_01017]	RequestedPackage field
[SWS_UCM_01018]	TransferVehiclePackage BusyWithCampaign
[SWS_UCM_01101]	Provide information of installed Software Clusters in vehicle
[SWS_UCM_01105]	Interaction of UCM Master with Vehicle Driver
[SWS_UCM_01109]	UCM Master provides a safety interface
[SWS_UCM_01114]	
[SWS_UCM_01117]	UCM Master SafetyState field
[SWS_UCM_01118]	UCM Master waiting for vehicle driver approval
[SWS_UCM_01119]	Report information of Software Packages
[SWS_UCM_01120]	Provide Software Packages general information
[SWS_UCM_01135]	Get Software Clusters descriptions from a vehicle
[SWS_UCM_01136]	
[SWS_UCM_01137]	
[SWS_UCM_01138]	
[SWS_UCM_01177]	
[SWS_UCM_01178]	
[SWS_UCM_01203]	CampaignState field
[SWS_UCM_01212]	Trigger on kActivating state
[SWS_UCM_01214]	Final action on kVehicleChecking state
[SWS_UCM_01215]	Trigger on kCancelling state
[SWS_UCM_01216]	Final action on kCancelling state
[SWS_UCM_01218]	Transition from kIdle state to kSyncing state





Number	Heading
[SWS_UCM_01219]	Transition from <code>kSyncing</code> state to <code>kIdle</code> state
[SWS_UCM_01220]	Transition from <code>kIdle</code> state to <code>kVehiclePackageTransferring</code> and <code>kTransferring</code> states
[SWS_UCM_01221]	Transition from <code>kVehiclePackageTransferring</code> state and <code>kTransferring</code> state to <code>kCancelling</code> state
[SWS_UCM_01227]	Transition from <code>kSoftwarePackage_Transferring</code> state and <code>kTransferring</code> state to <code>kCancelling</code> state
[SWS_UCM_01228]	Transition from <code>kSoftwarePackage_Transferring</code> state and <code>kTransferring</code> state to <code>kProcessing</code> state and <code>kUpdating</code> state
[SWS_UCM_01234]	Transition from <code>kProcessing</code> state to <code>kActivating</code> state
[SWS_UCM_01236]	Transition from <code>kProcessing</code> state and <code>kUpdating</code> state to <code>kCancelling</code> state
[SWS_UCM_01239]	Transition from <code>kActivating</code> state and <code>kUpdating</code> state to <code>kCancelling</code> state
[SWS_UCM_01241]	Transition from <code>kVehicleChecking</code> state and <code>kUpdating</code> state to <code>kCancelling</code> state
[SWS_UCM_01242]	Transition from <code>kVehicleChecking</code> state and <code>kUpdating</code> state to <code>kIdle</code> state
[SWS_UCM_01243]	Transition from <code>kCancelling</code> state to <code>kIdle</code> state
[SWS_UCM_01244]	Cancellation of an update campaign shall be possible
[SWS_UCM_01246]	Unreachable UCM during update campaign
[SWS_UCM_01248]	Content of History Report
[SWS_UCM_01266]	Subordinate Not Available On The Network
[SWS_UCM_01267]	Vehicle State Manager Communication Error
[SWS_UCM_01268]	Vehicle Driver Interface Communication Error
[SWS_UCM_01271]	New campaign enabling
[SWS_UCM_01272]	<code>VehicleCheck</code> call not permitted
[SWS_UCM_01301]	<code>Vehicle Package</code> authentication
[SWS_UCM_01305]	<code>Vehicle Package</code> format
[SWS_UCM_01306]	<code>TransferExit</code> Invalid package manifest
[SWS_UCM_CONSTR_00004]	Unsupported safety by Vehicle driver interface
[SWS_UCM_CONSTR_00005]	Safety state change
[SWS_UCM_CONSTR_00006]	Exclusive use of Vehicle State Manager
[SWS_UCM_CONSTR_00007]	Unsupported safety conditions by Vehicle State Manager
[SWS_UCM_CONSTR_00009]	Safety condition change
[SWS_UCM_CONSTR_00013]	Confidential information protection





Number	Heading
[SWS_UCM_CONSTR_00015]	Trigger on <code>kVehicleChecking</code> state

Table F.11: Changed Specification Items in R22-11

F.4.3 Deleted Specification Items in R22-11

Number	Heading
[SWS_UCM_00009]	<code>UCM</code> exposing its identifier
[SWS_UCM_00028]	<code>Software Package</code> Authentication
[SWS_UCM_00086]	Unsupported method calls
[SWS_UCM_00174]	
[SWS_UCM_00209]	<code>TransferData</code> <code>PackageInconsistent</code>
[SWS_UCM_00238]	<code>Rollback</code> <code>NotAbleToRollback</code>
[SWS_UCM_01107]	<code>UCM Master</code> provides progress information to Vehicle Driver
[SWS_UCM_01110]	<code>UCM Master</code> <code>SafetyState</code> method

Table F.12: Deleted Specification Items in R22-11

F.4.4 Added Constraints in R22-11

none

F.4.5 Changed Constraints in R22-11

none

F.4.6 Deleted Constraints in R22-11

none

F.5 Constraint and Specification Item History of this document according to AUTOSAR Release R23-11.

F.5.1 Added Specification Items in R23-11

Number	Heading
[SWS_UCM_00305]	Persistent data uri at Software Cluster installation
[SWS_UCM_00306]	Persistent data uri change at update
[SWS_UCM_00309]	Definition of ImplementationDataType UCMLIdentifierAndVersionType
[SWS_UCM_00311]	Provide SoftwareCluster general information
[SWS_UCM_00312]	Definition of ImplementationDataType SwClusterManifestInfoType
[SWS_UCM_00313]	Definition of ImplementationDataType DependencyVectorType
[SWS_UCM_00314]	Definition of ImplementationDataType DependencyType
[SWS_UCM_00315]	Definition of ImplementationDataType DependencyCompareConditionType
[SWS_UCM_00316]	Definition of ImplementationDataType DependencyOperatorType
[SWS_UCM_00317]	Definition of ImplementationDataType LogicalOperationType
[SWS_UCM_00318]	Definition of ImplementationDataType DependencyRoleType
[SWS_UCM_00319]	Semantic versionning
[SWS_UCM_00320]	Diagnostic Event: History recording failed
[SWS_UCM_00321]	Diagnostic Event: Update session with SM rejected
[SWS_UCM_00322]	Diagnostic Event: PrepareUpdate call to SM failed
[SWS_UCM_00323]	Diagnostic Event: RollBack failed
[SWS_UCM_00324]	Diagnostic Event: Verification with SM at activation failed
[SWS_UCM_00325]	Diagnostic Event: Campaign cancelling failed
[SWS_UCM_00326]	Diagnostic Event: Activation not possible because of missing dependencies
[SWS_UCM_00327]	Diagnostic Event: Installing old software is not allowed
[SWS_UCM_00329]	Activate kPersistencyAllocationFailed

Table F.13: Added Specification Items in R23-11

F.5.2 Changed Specification Items in R23-11

Number	Heading
[SWS_UCM_00026]	Dependency Check
[SWS_UCM_00038]	Definition of ImplementationDataType SwPackageStateType
[SWS_UCM_00039]	Definition of ImplementationDataType SwPackageInfoType
[SWS_UCM_00077]	Definition of ImplementationDataType SwClusterStateType
[SWS_UCM_00078]	Definition of ImplementationDataType SwClusterInfoType
[SWS_UCM_00131]	Definition of ServiceInterface PackageManagement





Number	Heading
[SWS_UCM_00132]	Definition of ImplementationDataType ActionType
[SWS_UCM_00133]	Definition of ImplementationDataType ResultType
[SWS_UCM_00134]	Definition of ImplementationDataType HistoryType
[SWS_UCM_00135]	Definition of ImplementationDataType HistoryVectorType
[SWS_UCM_00136]	Definition of Application Error Domain of functional cluster UCM
[SWS_UCM_00161]	Check Software Package version compatibility against UCM version
[SWS_UCM_00175]	Definition of ImplementationDataType StrongRevisionLabelString
[SWS_UCM_00184]	Persistent data clean-up after Software Cluster removal
[SWS_UCM_00185]	Provide SoftwareCluster general information
[SWS_UCM_00203]	TransferData InvalidTransferId
[SWS_UCM_00236]	RevertProcessedSwPackages NotAbleToRevertPackages
[SWS_UCM_00245]	Software Cluster category
[SWS_UCM_00262]	Update preparation rejected
[SWS_UCM_00264]	Update verification rejected
[SWS_UCM_00266]	OperationNotPermitted error and UCM state
[SWS_UCM_00271]	Keeping history of failure error code
[SWS_UCM_00273]	Persistent data clean-up after Software Cluster update that removes a process
[SWS_UCM_00274]	UCM initialization
[SWS_UCM_00278]	Cancel error handling order
[SWS_UCM_00279]	RevertProcessedSwPackages error handling order
[SWS_UCM_00281]	Activate error handling order
[SWS_UCM_00292]	History elements ordering

Table F.14: Changed Specification Items in R23-11

F.5.3 Deleted Specification Items in R23-11

Number	Heading
[SWS_UCM_00120]	Runtime dependencies check
[SWS_UCM_00177]	
[SWS_UCM_00178]	
[SWS_UCM_00179]	
[SWS_UCM_00180]	
[SWS_UCM_00181]	
[SWS_UCM_00182]	
[SWS_UCM_00183]	





Number	Heading
[SWS_UCM_00186]	
[SWS_UCM_00187]	
[SWS_UCM_00210]	Transferring of software packages on <code>kProcessing</code> state
[SWS_UCM_00251]	
[SWS_UCM_00252]	
[SWS_UCM_00253]	
[SWS_UCM_00254]	
[SWS_UCM_00255]	
[SWS_UCM_00256]	
[SWS_UCM_00268]	
[SWS_UCM_00269]	
[SWS_UCM_00290]	
[SWS_UCM_00291]	
[SWS_UCM_00296]	
[SWS_UCM_00297]	Retry Strategy for <code>ServiceBusy</code>
[SWS_UCM_00298]	Retry Strategy for <code>UpdateSessionRejected</code>
[SWS_UCM_01003]	<code>UCM Master</code> checks states of <code>UCM</code> subordinates
[SWS_UCM_01005]	<code>UCM Master</code> is discovering <code>UCMs</code> in vehicle
[SWS_UCM_01011]	<code>TransferVehiclePackage</code> <code>InsufficientMemory</code>
[SWS_UCM_01013]	Too big block size received by <code>UCM Master</code>
[SWS_UCM_01014]	Packages transferring sequence
[SWS_UCM_01015]	Invalid Vehicle Package manifest
[SWS_UCM_01016]	Invalid Package Manifest
[SWS_UCM_01017]	<code>RequestedPackage</code> field
[SWS_UCM_01018]	<code>TransferVehiclePackage</code> <code>BusyWithCampaign</code>
[SWS_UCM_01019]	<code>UCM Master</code> initialization
[SWS_UCM_01020]	Retry Strategy for <code>BlockInconsistent</code>
[SWS_UCM_01101]	Provide information of installed <code>Software Clusters</code> in vehicle
[SWS_UCM_01103]	Inform <code>Backend</code> of needed <code>Software Packages</code> for an update
[SWS_UCM_01105]	Interaction of <code>UCM Master</code> with Vehicle Driver
[SWS_UCM_01109]	<code>UCM Master</code> provides a safety interface
[SWS_UCM_01114]	
[SWS_UCM_01117]	<code>UCM Master</code> <code>SafetyState</code> field
[SWS_UCM_01118]	<code>UCM Master</code> waiting for vehicle driver approval
[SWS_UCM_01119]	Report information of <code>Software Packages</code>
[SWS_UCM_01120]	Provide <code>Software Packages</code> general information
[SWS_UCM_01121]	Adaptive Platform interface provided for Flashing Adapter
[SWS_UCM_01122]	Supported physical layers by <code>D-PDU API</code> implementation





Number	Heading
[SWS_UCM_01123]	Supported application layers by D-PDU API implementation
[SWS_UCM_01124]	Supported protocols by D-PDU API implementation
[SWS_UCM_01125]	Separation of D-PDU API -Software with the MVCI protocol module firmware
[SWS_UCM_01126]	Root description file (RDF)
[SWS_UCM_01127]	Module Description File (MDF)
[SWS_UCM_01128]	Symbolic names and IDs
[SWS_UCM_01129]	SAE J2534-1 and RP 1210a compatibility
[SWS_UCM_01130]	ComPrimitives in RawMode
[SWS_UCM_01131]	PDUIoCtl(PDU_IOCTL_RESET)
[SWS_UCM_01132]	PDUIoCtl(PDU_IOCTL_START_MSG_FILTER), PDUIoCtl(PDU_IOCTL_CLEAR_MSG_FILTER), PDUIoCtl(PDU_IOCTL_STOP_MSG_FILTER)
[SWS_UCM_01133]	PDUIoCtl(PDU_IOCTL_SEND_BREAK)
[SWS_UCM_01134]	Not used D-PDU API function return codes
[SWS_UCM_01135]	Get Software Clusters descriptions from a vehicle
[SWS_UCM_01136]	
[SWS_UCM_01137]	
[SWS_UCM_01138]	
[SWS_UCM_01177]	
[SWS_UCM_01178]	
[SWS_UCM_01201]	Sequential orchestration of campaigns
[SWS_UCM_01203]	CampaignState field
[SWS_UCM_01204]	Initial state
[SWS_UCM_01205]	UCM Master internal state persistency
[SWS_UCM_01207]	Trigger on kSoftwarePackage_Transferring state
[SWS_UCM_01209]	Trigger on kProcessing state
[SWS_UCM_01212]	Trigger on kActivating state
[SWS_UCM_01214]	Final action on kVehicleChecking state
[SWS_UCM_01215]	Trigger on kCancelling state
[SWS_UCM_01216]	Final action on kCancelling state
[SWS_UCM_01217]	Monitoring of UCM subordinates
[SWS_UCM_01218]	Transition from kIdle state to kSyncing state
[SWS_UCM_01219]	Transition from kSyncing state to kIdle state
[SWS_UCM_01220]	Transition from kIdle state to kVehiclePackageTransferring and kTransferring states
[SWS_UCM_01221]	Transition from kVehiclePackageTransferring state and kTransferring state to kCancelling state
[SWS_UCM_01222]	Transition from kVehiclePackageTransferring state to kSoftwarePackage_Transferring state



△

Number	Heading
[SWS_UCM_01227]	Transition from kSoftwarePackage_Transferring state and kTransferring state to kCancelling state
[SWS_UCM_01228]	Transition from kSoftwarePackage_Transferring state and kTransferring state to kProcessing state and kUpdating state
[SWS_UCM_01229]	SafetyConditions while processing stream
[SWS_UCM_01234]	Transition from kProcessing state to kActivating state
[SWS_UCM_01236]	Transition from kProcessing state and kUpdating state to kCancelling state
[SWS_UCM_01239]	Transition from kActivating state and kUpdating state to kCancelling state
[SWS_UCM_01240]	Transition from kActivating state to kVehicleChecking state
[SWS_UCM_01241]	Transition from kVehicleChecking state and kUpdating state to kCancelling state
[SWS_UCM_01242]	Transition from kVehicleChecking state and kUpdating state to kIdle state
[SWS_UCM_01243]	Transition from kCancelling state to kIdle state
[SWS_UCM_01244]	Cancellation of an update campaign shall be possible
[SWS_UCM_01246]	Unreachable UCM during update campaign
[SWS_UCM_01247]	Method to read History Report
[SWS_UCM_01248]	Content of History Report
[SWS_UCM_01265]	TransferState field
[SWS_UCM_01266]	Subordinate Not Available On The Network
[SWS_UCM_01267]	Vehicle State Manager Communication Error
[SWS_UCM_01268]	Vehicle Driver Interface Communication Error
[SWS_UCM_01269]	Campaign cancellation history
[SWS_UCM_01270]	New campaign disabling
[SWS_UCM_01271]	New campaign enabling
[SWS_UCM_01272]	VehicleCheck call not permitted
[SWS_UCM_01273]	CancelCampaign CancelFailed error
[SWS_UCM_01274]	CancelCampaign OperationNotPermitted error
[SWS_UCM_01275]	Safety conditions during activation
[SWS_UCM_01301]	Vehicle Package authentication
[SWS_UCM_01302]	Vehicle Package authentication failure
[SWS_UCM_01303]	Dependencies between Software Packages
[SWS_UCM_01305]	Vehicle Package format
[SWS_UCM_01306]	TransferExit Invalid package manifest
[SWS_UCM_01307]	Vehicle Package format not supported
[SWS_UCM_01308]	Check Vehicle Package version compatibility against UCM Master version

Table F.15: Deleted Specification Items in R23-11

F.5.4 Added Constraints in R23-11

none

F.5.5 Changed Constraints in R23-11

none

F.5.6 Deleted Constraints in R23-11

Number	Heading
[SWS_UCM_CONSTR_00003]	Exclusive use of Vehicle Driver Interface
[SWS_UCM_CONSTR_00004]	Unsupported safety by Vehicle driver interface
[SWS_UCM_CONSTR_00005]	Safety state change
[SWS_UCM_CONSTR_00006]	Exclusive use of Vehicle State Manager
[SWS_UCM_CONSTR_00007]	Unsupported safety conditions by Vehicle State Manager
[SWS_UCM_CONSTR_00008]	Switching vehicle into update mode
[SWS_UCM_CONSTR_00009]	Safety condition change
[SWS_UCM_CONSTR_00011]	Flashing Adapter provided interface
[SWS_UCM_CONSTR_00013]	Confidential information protection
[SWS_UCM_CONSTR_00015]	Trigger on <code>kVehicleChecking</code> state
[SWS_UCM_CONSTR_00016]	OTA Client use of RequestedPackage field





Number	Heading
[SWS_UCM_CONSTR_00017]	Interaction of UCM Master with Vehicle Driver

Table F.16: Deleted Constraints in R23-11

F.6 Constraint and Specification Item History of this document according to AUTOSAR Release R24-11.

F.6.1 Added Specification Items in R24-11

Number	Heading
[SWS_UCM_00330]	GetSwPackages method at Software Packages kTransferring state
[SWS_UCM_00331]	Delete Software Package at kCleaningUp
[SWS_UCM_00332]	Software Package transfer - Log successful Software Package transfer
[SWS_UCM_00333]	Software Package transfer - Log failure of Software Package transfer
[SWS_UCM_00334]	Software Package processing - Log successful Software Package processing
[SWS_UCM_00335]	Software Package processing - Log failure of Software Package processing
[SWS_UCM_00336]	Software Cluster activation - Log installation of new Software Cluster
[SWS_UCM_00337]	Software Cluster activation - Log update of existing Software Cluster
[SWS_UCM_00338]	Software Cluster activation - Log removal of existing Software Cluster
[SWS_UCM_00339]	Software Cluster activation failure - Log failure of Software Cluster activation
[SWS_UCM_00340]	Software Cluster rollback - Log rollback of Software Cluster
[SWS_UCM_00341]	Definition of ImplementationDataType ProgressInformationType
[SWS_UCM_00342]	Update configuration only
[SWS_UCM_00343]	Definition of Method PackageManagement.GetId
[SWS_UCM_00344]	Definition of Method PackageManagement.RegisterSoftwarePackage
[SWS_UCM_00345]	Definition of Method PackageManagement.TransferStart
[SWS_UCM_00346]	Definition of Method PackageManagement.TransferData
[SWS_UCM_00347]	Definition of Method PackageManagement.TransferExit
[SWS_UCM_00348]	Definition of Method PackageManagement.DeleteTransfer
[SWS_UCM_00349]	Definition of Method PackageManagement.ProcessSwPackage
[SWS_UCM_00350]	Definition of Method PackageManagement.RevertProcessedSwPackages





Number	Heading
[SWS_UCM_00351]	Definition of Method PackageManagement.Cancel
[SWS_UCM_00352]	Definition of Method PackageManagement.Activate
[SWS_UCM_00353]	Definition of Method PackageManagement.Rollback
[SWS_UCM_00354]	Definition of Method PackageManagement.Finish
[SWS_UCM_00355]	Definition of Method PackageManagement.GetHistory
[SWS_UCM_00356]	Definition of Method PackageManagement.GetSwClusterChangeInfo
[SWS_UCM_00357]	Definition of Method PackageManagement.GetSwClusterInfo
[SWS_UCM_00358]	Definition of Method PackageManagement.GetSwClusterManifestInfo
[SWS_UCM_00359]	Definition of Method PackageManagement.GetSwPackages
[SWS_UCM_00360]	Definition of Method PackageManagement.GetProgress
[SWS_UCM_00361]	Definition of Field PackageManagement.CurrentStatus
[SWS_UCM_00362]	Definition of ImplementationDataType SwPackageNameType
[SWS_UCM_00363]	Missing dependencies is triggering production error
[SWS_UCM_00364]	Update session failure is triggering production error
[SWS_UCM_00366]	Diagnostic Event: for UCM
[SWS_UCM_00367]	Diagnostic Event: for UCM
[SWS_UCM_00368]	UCM FAILED PREPAREROLLBACK is triggering production error
[SWS_UCM_00369]	UCM REJECTED PREPAREROLLBACK is triggering production error
[SWS_UCM_00370]	UCM FAILED Verification is triggering production error
[SWS_UCM_00371]	UCM rollback after failed Machine restart
[SWS_UCM_00372]	Cancel Failing is triggering production error
[SWS_UCM_00373]	Update preparation rejected
[SWS_UCM_00374]	Diagnostic Event: Update verification rejected
[SWS_UCM_00375]	Diagnostic Event: Update session with SM rejected
[SWS_UCM_00376]	LogMessage SoftwarePackageReceived
[SWS_UCM_00377]	LogMessage SoftwarePackageTransferFailed
[SWS_UCM_00378]	LogMessage SoftwarePackageProcessed
[SWS_UCM_00379]	LogMessage SoftwarePackageProcessingFailed
[SWS_UCM_00380]	LogMessage SoftwareClusterInstalled
[SWS_UCM_00381]	LogMessage SoftwareClusterUpdated
[SWS_UCM_00382]	LogMessage SoftwareClusterRemoved
[SWS_UCM_00383]	LogMessage SoftwareClusterActivationFailed
[SWS_UCM_00384]	LogMessage SoftwareClusterRolledback
[SWS_UCM_00385]	Suspend and resume support
[SWS_UCM_00386]	Suspend and resume not support
[SWS_UCM_00387]	Suspend the execution of potentially long running Update States
[SWS_UCM_00388]	Resume the execution of potentially long running Update States
[SWS_UCM_00389]	Error behaviour for resume





Number	Heading
[SWS_UCM_00390]	Error behaviour of service interface during suspension
[SWS_UCM_00391]	Processing a Software Package
[SWS_UCM_00392]	Failed processing a Software Package
[SWS_UCM_00393]	Definition of ImplementationDataType TransferStateType
[SWS_UCM_00394]	Definition of ImplementationDataType ProcessingStateType
[SWS_UCM_00395]	Definition of ImplementationDataType RunningStateType
[SWS_UCM_00396]	Definition of ImplementationDataType UpdateStateType
[SWS_UCM_00397]	Definition of Method PackageManagement.Suspend
[SWS_UCM_00398]	Definition of Method PackageManagement.Resume
[SWS_UCM_00399]	SEV SW UPDATE FAILED
[SWS_UCM_00400]	SEV SW UPDATE SUCCESS
[SWS_UCM_00401]	string in the context data is shorter than SecurityEventContextData Element.maxLength
[SWS_UCM_00402]	string in the context data is longer than SecurityEventContextData Element.maxLength
[SWS_UCM_00403]	Security events for UCM
[SWS_UCM_00404]	Security event context data definition: SEV_SW_UPDATE_FAILED
[SWS_UCM_00405]	Security event context data definition: SEV_SW_UPDATE_SUCCESS
[SWS_UCM_00407]	Mapping of context data elements for SEV SW UPDATE FAILED
[SWS_UCM_00408]	Mapping of context data elements for SEV SW UPDATE SUCCESS

Table F.17: Added Specification Items in R24-11

F.6.2 Changed Specification Items in R24-11

Number	Heading
[SWS_UCM_00003]	Cancelling the package processing
[SWS_UCM_00008]	Executing the data transfer
[SWS_UCM_00010]	End of data transfer
[SWS_UCM_00017]	Sequential <i>Software Package</i> Processing
[SWS_UCM_00026]	Dependency Check
[SWS_UCM_00029]	Consistency Check of Manifest
[SWS_UCM_00031]	Definition of ImplementationDataType TransferIdType
[SWS_UCM_00032]	Definition of ImplementationDataType ByteVectorType
[SWS_UCM_00038]	Definition of ImplementationDataType SwPackageStateType
[SWS_UCM_00039]	Definition of ImplementationDataType SwPackageInfoType
[SWS_UCM_00040]	Definition of ImplementationDataType SwPackageInfoVectorType
[SWS_UCM_00044]	Definition of ImplementationDataType CurrentStatusType





Number	Heading
[SWS_UCM_00071]	Definition of ImplementationDataType SwClusterNameType
[SWS_UCM_00073]	Definition of Port PackageManagement provided by functional cluster UCM
[SWS_UCM_00078]	Definition of ImplementationDataType SwClusterInfoType
[SWS_UCM_00079]	Definition of ImplementationDataType SwClusterInfoVectorType
[SWS_UCM_00080]	Default state of Package Management
[SWS_UCM_00081]	Processing of Software Packages.
[SWS_UCM_00083]	Entering the Ready state of Package Management after a successful processing operation
[SWS_UCM_00085]	Entering the kActivated Update State (updateState) of Package Management
[SWS_UCM_00088]	Preparation of data transfer
[SWS_UCM_00098]	Software Package Authentication failure
[SWS_UCM_00100]	Update of Functional Clusters
[SWS_UCM_00101]	Update of Host
[SWS_UCM_00103]	Update to Software Cluster version which is not newer than currently present and than previously removed
[SWS_UCM_00104]	Integrity Check of processed Package
[SWS_UCM_00127]	Finishing update sequence
[SWS_UCM_00131]	Definition of ServiceInterface PackageManagement
[SWS_UCM_00132]	Definition of ImplementationDataType ActionType
[SWS_UCM_00134]	Definition of ImplementationDataType HistoryType
[SWS_UCM_00135]	Definition of ImplementationDataType HistoryVectorType
[SWS_UCM_00136]	Definition of Application Error Domain of functional cluster UCM
[SWS_UCM_00147]	Return to the Prparing state from Cleaning-up state
[SWS_UCM_00149]	Stay in Preparing state
[SWS_UCM_00150]	Cancellation of a Software Package processing
[SWS_UCM_00151]	Entering the Ready state of Package Management after a Cancel call
[SWS_UCM_00152]	Entering the Preparing state of Package Management after a missing dependency
[SWS_UCM_00161]	Check Software Package version compatibility against UCM version
[SWS_UCM_00162]	Entering the Cleaning-up state after a RevertProcessedSwPackages call
[SWS_UCM_00166]	Processing from stream states
[SWS_UCM_00167]	Cancelling streamed packages
[SWS_UCM_00168]	Transferring while processing from stream
[SWS_UCM_00169]	Finishing transfer while processing from stream
[SWS_UCM_00173]	Definition of ImplementationDataType UCMIIdentifierType
[SWS_UCM_00176]	Definition of ImplementationDataType SwNameVersionType
[SWS_UCM_00202]	Trusted Platform compliance
[SWS_UCM_00207]	TransferData BlockInconsistent





Number	Heading
[SWS_UCM_00217]	ProcessSwPackage InsufficientMemory
[SWS_UCM_00218]	ProcessSwPackage InvalidTransferId
[SWS_UCM_00219]	ProcessSwPackage OperationNotPermitted
[SWS_UCM_00231]	ProcessSwPackage IncompatibleDelta
[SWS_UCM_00239]	Rollback OperationNotPermitted
[SWS_UCM_00241]	Activate OperationNotPermitted
[SWS_UCM_00258]	Update session rejected
[SWS_UCM_00260]	PrepareUpdate, VerifyUpdate and PrepareRollback orders
[SWS_UCM_00262]	Diagnostic Event: Update preparation rejected
[SWS_UCM_00263]	Update preparation failure
[SWS_UCM_00264]	Update verification rejected
[SWS_UCM_00265]	state transition due to ProcessSwPackage error
[SWS_UCM_00266]	OperationNotPermitted error and UCM state
[SWS_UCM_00267]	Error when checksum is not recognised at processing time
[SWS_UCM_00271]	Keeping history of failure error code
[SWS_UCM_00272]	Transfer block size
[SWS_UCM_00273]	Persistent data clean-up after Software Cluster update that removes a process
[SWS_UCM_00274]	UCM initialization
[SWS_UCM_00275]	TransferData error handling order
[SWS_UCM_00276]	TransferExit error handling order
[SWS_UCM_00277]	ProcessSwPackage error handling order
[SWS_UCM_00283]	DeleteTransfer error handling order
[SWS_UCM_00285]	Removing or updating a Software Cluster not existing in the Machine
[SWS_UCM_00288]	Definition of Port UpdateRequest required by functional cluster UCM
[SWS_UCM_00289]	TransferData TransferFailed
[SWS_UCM_00293]	VerifyUpdate method
[SWS_UCM_00294]	Unsupported package format for UCM
[SWS_UCM_00299]	Verify rolled back Software Clusters
[SWS_UCM_00300]	Software Cluster failing to rollback
[SWS_UCM_00301]	Retry ro Rollback again when UCM is in kRollingBackFailed state
[SWS_UCM_00302]	Rollback failing is triggering production error
[SWS_UCM_00303]	failing to record history
[SWS_UCM_00305]	Persistent data uri at Software Cluster installation
[SWS_UCM_00306]	Persistent data uri change at update
[SWS_UCM_00309]	Definition of ImplementationDataType UCMIIdentifierAndVersionType
[SWS_UCM_00311]	Provide SoftwareCluster general information
[SWS_UCM_00312]	Definition of ImplementationDataType SwClusterManifestInfoType





Number	Heading
[SWS_UCM_00313]	Definition of ImplementationDataType DependencyVectorType
[SWS_UCM_00314]	Definition of ImplementationDataType DependencyType
[SWS_UCM_00315]	Definition of ImplementationDataType DependencyCompareConditionType
[SWS_UCM_00316]	Definition of ImplementationDataType DependencyOperatorType
[SWS_UCM_00317]	Definition of ImplementationDataType LogicalOperationType
[SWS_UCM_00318]	Definition of ImplementationDataType DependencyRoleType
[SWS_UCM_00321]	Diagnostic Event: Update session with SM rejected
[SWS_UCM_00329]	Activate kPersistencyAllocationFailed

Table F.18: Changed Specification Items in R24-11

F.6.3 Deleted Specification Items in R24-11

none

F.6.4 Added Constraints in R24-11

none

F.6.5 Changed Constraints in R24-11

Number	Heading
[SWS_UCM_CONSTR_00002]	UCM confidential information handling
[SWS_UCM_CONSTR_00012]	
[SWS_UCM_CONSTR_00014]	Software Package and Software Cluster shortNames

Table F.19: Changed Constraints in R24-11

F.6.6 Deleted Constraints in R24-11

none

F.7 Constraint and Specification Item History of this document according to AUTOSAR Release R25-11.

F.7.1 Added Specification Items in R25-11

Number	Heading
[APMC_UCM_00001]	Definition of ApmcFunctionalClusterDef UcmInstantiation
[APMC_UCM_00002]	Definition of ApmcParamConfContainerDef UcmSubordinateInstantiation
[APMC_UCM_00003]	Definition of ApmcParamConfContainerDef SoftwareClusterCollection
[APMC_UCM_00004]	Definition of ApmcParamConfContainerDef SoftwareCluster
[APMC_UCM_00005]	Definition of ApmcParamConfContainerDef ProcessRefCollection
[APMC_UCM_00006]	Definition of ApmcParamConfContainerDef FunctionalClusterRefContainer
[APMC_UCM_00007]	Definition of ApmcForeignReferenceDef AdaptiveModuleInstantiationForeignRef
[APMC_UCM_00008]	Definition of ApmcParamConfContainerDef ARElementRefCollection
[APMC_UCM_00009]	Definition of ApmcForeignReferenceDef ARElementRef
[APMC_UCM_00010]	Definition of ApmcForeignReferenceDef LicenseRef
[APMC_UCM_00011]	Definition of ApmcForeignReferenceDef ReleaseNotesRef
[APMC_UCM_00012]	Definition of ApmcForeignReferenceDef VendorSignatureRef
[APMC_UCM_00015]	Definition of ApmcParamConfContainerDef DesignRefCollection
[APMC_UCM_00016]	Definition of ApmcUriForeignReferenceDef DesignRef
[APMC_UCM_00017]	Definition of ApmcParamConfContainerDef SoftwareClusterFunctionGroupRefCollection
[APMC_UCM_00018]	Definition of ApmcForeignReferenceDef ClaimedFunctionGroupForeignRef
[APMC_UCM_00019]	Definition of ApmcIntegerParamDef UcmSubordinateMaxAvailablePersistencyStorageSpace
[APMC_UCM_00020]	Definition of ApmcParamConfContainerDef UcmSubordinatePrepareRollback
[APMC_UCM_00021]	Definition of ApmcParamConfContainerDef UcmSubordinatePrepareUpdate
[APMC_UCM_00022]	Definition of ApmcParamConfContainerDef UcmSubordinateVerifyUpdate
[APMC_UCM_00023]	Definition of ApmcParamConfContainerDef UcmRetryStrategy
[APMC_UCM_00024]	Definition of ApmcIntegerParamDef MaximumNumberOfRetries
[APMC_UCM_00025]	Definition of ApmcFloatParamDef RetryIntervalTime
[APMC_UCM_00026]	Definition of ApmcParamConfContainerDef UcmInstantiationGeneral
[APMC_UCM_00027]	Definition of ApmcStringParamDef UcmIdentifier
[APMC_UCM_00028]	Definition of ApmcIntegerParamDef MaxBlockSize
[APMC_UCM_00029]	Definition of ApmcStrongRevisionLabelParamDef Version
[APMC_UCM_00030]	Definition of ApmcForeignReferenceDef DiagnosticPropsForeignRef
[APMC_UCM_00032]	Definition of ApmcStringParamDef TypeApproval
[APMC_UCM_00033]	Definition of ApmcIntegerParamDef VendorId





Number	Heading
[APMC_UCM_00034]	Definition of ApmcStrongRevisionLabelParamDef SoftwareClusterVersion
[APMC_UCM_00035]	Definition of ApmcEnumerationParamDef SoftwareClusterCategory
[APMC_UCM_00036]	Definition of ApmcParamConfContainerDef ArtifactLocatorCollection
[APMC_UCM_00037]	Definition of ApmcParamConfContainerDef ArtifactLocator
[APMC_UCM_00038]	Definition of ApmcStringParamDef URI
[APMC_UCM_00039]	Definition of ApmcForeignReferenceDef RepresentedModelElementRef
[APMC_UCM_00040]	Definition of ApmcForeignReferenceDef FunctionalClusterForeignRef
[APMC_UCM_00041]	Definition of ApmcForeignReferenceDef ProcessForeignRef
[APMC_UCM_00042]	Definition of ApmcStringParamDef ArtifactChecksum
[APMC_UCM_00043]	Definition of ApmcParamConfContainerDef ArtifactChecksumCollection
[APMC_UCM_00044]	Definition of ApmcParamConfContainerDef ArtifactChecksum
[SWS_UCM_00409]	Cleanup failing is triggering production error
[SWS_UCM_00410]	Diagnostic Event: Cleanup failing is triggering production error

Table F.20: Added Specification Items in R25-11

F.7.2 Changed Specification Items in R25-11

Number	Heading
[SWS_UCM_00218]	ProcessSwPackage InvalidTransferId
[SWS_UCM_00245]	Software Cluster category

Table F.21: Changed Specification Items in R25-11

F.7.3 Deleted Specification Items in R25-11

Number	Heading
[SWS_UCM_00001]	Starting the package processing
[SWS_UCM_00004]	Report software information
[SWS_UCM_00005]	Rollback to the software prior to Finish the update process
[SWS_UCM_00018]	Providing Progress Information
[SWS_UCM_00025]	Activation of SoftwareClusters
[SWS_UCM_00027]	Delta Package version applicability
[SWS_UCM_00069]	Report information on Software Packages
[SWS_UCM_00083]	Entering the Ready state of Package Management after a successful processing operation





Number	Heading
[SWS_UCM_00151]	Entering the Ready state of Package Management after a Cancel call
[SWS_UCM_00185]	Provide SoftwareCluster general information
[SWS_UCM_00220]	GetSwProcessProgress InvalidTransferId
[SWS_UCM_00265]	state transition due to ProcessSwPackage error

Table F.22: Deleted Specification Items in R25-11

F.7.4 Added Constraints in R25-11

none

F.7.5 Changed Constraints in R25-11

none

F.7.6 Deleted Constraints in R25-11

none